



Nabz documentation

Plans, product, engineering and operations for the Nabz home-healthcare marketplace, in one document.

10 October 2026 · branch feature/nabz-care-marketplace

Contents

PART 1: PLAN, PRODUCT AND OPERATIONS

1. Improvement plan	docs/NABZ_IMPROVEMENT_PLAN.md
2. What's built in Nabz	docs/NABZ_WHAT_WE_BUILT.md
3. Operations runbook (staging)	docs/NABZ_OPERATIONS_RUNBOOK.md

PART 2: PRODUCT PLANS

4. Provider marketplace plan	docs/product/PROVIDER_MARKETPLACE_PLAN.md
5. Admin and ads guide	docs/product/ADMIN_AND_ADS_GUIDE.md
6. Book for someone else	docs/product/BOOK_FOR_SOMEONE_ELSE_PLAN.md
7. Pharmacy marketplace plan	docs/MEDRUSH_PHARMACY_MARKETPLACE_PLAN.md
8. Pharmacy store network plan	docs/pharmacy/STORE_NETWORK_PLAN.md
9. Revenue model	docs/NABZ_REVENUE_MODEL.md
10. Phase 1.1 rollout	docs/plans/PHASE_1_1_ROLLOUT.md
11. Website and app imagery	docs/plans/2026-09-24-website-and-app-patient-imagery.md

PART 3: ENGINEERING

12. Getting started	docs/MEDRUSH_GETTING_STARTED.md
13. Data model	docs/MEDRUSH_DATA_MODEL.md
14. AWS deployment	docs/MEDRUSH_AWS_DEPLOYMENT.md
15. Staging infrastructure (Terraform)	terraform/apprunner-staging/README.md
16. Mobile apps	frontends/mobile/README.md
17. Website	frontends/web/README.md
18. Code context	docs/product/CODE_CONTEXT.md
19. Project context (handoff brief)	context.md

PART 4: AUDITS AND WORKING NOTES

20. Audit of all roles (24 Sep 2026)	docs/AUDIT_2026-09-24_ALL_ROLES.md
--------------------------------------	------------------------------------

21. Build prompts

[docs/product/PROMPTS.md](#)

22. Features, models and edge cases (earlier PDF)

[attached at the end](#)

23. Testing and security report (earlier PDF)

[attached at the end](#)

PART 1

Plan, product and operations

Written in October 2026: what to improve next, what is built, and how to run it.

Improvement plan

October 2026. Covers UI/UX, features and architecture, ordered by what matters most before real customers use Nabz.

Recommended next five

1. **Phone OTP sign-in** with an SMS provider.
2. **Payments end to end** (Razorpay test mode, then live).
3. **A real domain** and Google sign-in moved to production.
4. **App builds in the cloud**, with over-the-air updates.
5. **One booking flow** for every service.

Together these make Nabz launchable and remove the biggest source of confusion. After step 4, most later items reach installed apps through over-the-air updates, without a new APK.

Phase 1: Launch blockers (weeks 1–2)

These stop Nabz from working for real users, whatever the UI looks like.

#	What	Why	Effort
1	Turn on payments: Razorpay test mode, then live	Staging runs with <code>RAZORPAY_ENABLED=false</code> . Online payment, refunds and payouts have not run end to end against Razorpay.	Medium
2	Phone OTP sign-in (MSG91 or Twilio)	Switched off on staging because no SMS provider is set. Many elderly users know their phone number but not a Gmail address or a password.	Small
3	Real domain (e.g. <code>nabz.in</code>) behind CloudFront	Google sign-in cannot leave Testing mode (100 test users) on an <code>awsapprunner.com</code> address. A domain also gives a proper support email, privacy-policy links and faster loading.	Small–Medium
4	Cloud app builds (EAS Build or CodeBuild), AAB files for the Play Store, EAS Update	Local builds take 30–70 minutes and stop when the PC sleeps. EAS Update delivers most UI fixes to installed apps in minutes.	Medium
5	Housekeeping	Rotate the exposed Google web client secret. Fix the two failing tests: the shared-package export count, and the bot test that misfires around midnight.	Small

Phase 2: UI/UX (weeks 2–5)

1. **One booking flow everywhere.** Nursing, Physio & care and Lab tests each book differently today. Use one pattern for all of them: *what* → *when* → *where* → *pay*, with the same steps and the same bill card. This is the biggest single fix for confusion, especially for elderly users.

2. **One search on Home.** Search services, medicines, lab tests and shops together. Make it tolerant of typos ("paracetamol") and of Hindi typed in English. MongoDB's plain text search, used today, misses typos. Atlas Search fixes this.
3. **Real product photos** in the pharmacy grid, instead of one generic icon. This is what builds trust on quick-commerce apps.
4. **Live medicine delivery tracking for customers:** the rider on a map, with an arrival time. Riders and stores already have tracking; customers don't.
5. **Finish accessibility:**
 - Easy mode on the website.
 - Hindi on every screen; about 14 of 45 app screens still have English-only text.
 - Read-aloud (deferred earlier).
6. **Video thumbnails** for partner posts, which show a plain play icon today.
7. **One notifications inbox** in the app for orders, visits, substitutions and offers.

Phase 3: Features (month 2)

Pharmacy owners

- Barcode scanning with the phone camera to receive stock and update counts.
- Reorder lists to send to distributors.
- GST invoices as PDFs.

Partners

- An availability calendar.
- Monthly earnings statements as PDFs.
- Instant payouts through RazorpayX, instead of admins marking each one paid.

Customers

- A prescription vault: upload once, reuse on every order.
- Photos in reviews.
- Lab report PDFs in the app (images only today).

Admin

- Automatic screening of partner photos and videos (AWS Rekognition). Anything flagged waits for a person to review it instead of going live.
- A daily dashboard: sign-ups, bookings, orders, cancellations, revenue.

Phase 4: Architecture (in parallel, ongoing)

Area	Today	Change to
Deploys	Manual: zip from a laptop, then CodeBuild	GitHub Actions runs tests on every pull request; merging to <code>develop</code> deploys staging automatically.

Area	Today	Change to
Background work	Rider assignment and family notifications start inside the request (<code>setImmediate</code> , a save hook), so a server restart can lose them	A job queue (SQS + worker) so jobs retry and survive restarts. Scheduled jobs already hold a MongoDB lock and are fine.
Rate limits and cache	Redis is off, so each of up to 4 servers counts limits separately	ElastiCache Redis, so limits are shared across servers.
Media	Original files through signed S3 links	CloudFront, with resized photo thumbnails and compressed videos: faster, and lighter on mobile data.
Duplicated code	<code>apps/patient-health</code> copies the booking code, kept in sync by hand	Finish that split, or delete the copy.
Crash visibility	Server logs only	Sentry in the app and website, so crashes customers hit are visible.
Testing	Strong server tests; the website only has a public-page check	Automated app tests (Maestro): sign in → book → pay → track. A load test before launch.
Security	The website's Content Security Policy still allows inline scripts (<code>'unsafe-inline'</code>)	A nonce-based policy.

What's built in Nabz

State of the `feature/nabz-care-marketplace` branch, October 2026.

Nabz is a home-healthcare marketplace in Jaipur. It has four surfaces:

- **Nabz app** (Android) for customers.
- **Nabz Partner app** (Android) for nurses, physiotherapists, labs, pharmacies and riders.
- **Website** (Next.js) with the same features as the apps.
- **Admin panel** on the website.

Every customer and partner feature ships in both an app and the website. The API is a Node/Express monolith on MongoDB, served at `/api/v1`.

Customers

Home

- Address and search at the top, then a one-line count of verified staff online nearby.
- "Nurse at home" services in a 4 × 2 grid, with **See all** for the rest.
- Tiles for Pharmacy, Lab tests and Physio & care, then offers and **Book again**.
- Tapping a service asks **Book now** or **Pick a time** first.
- The full-screen map is one tap away. It has a pin for each service type and sponsored pins (labelled "Ad") that admin controls.

Booking

- Home nursing: now or scheduled, with supplies the nurse can bring.
- Physio, home care (the same caregiver throughout) and labs: compare providers by price, distance, rating and reliability, Zomato-style.
- Multi-visit plans, partners' plan proposals, and booking for a family member.

Pharmacy

- Medicines from nearby stores in a 3-column grid, cart, cash on delivery or online payment, and riders.
- **Substitution with consent:** a store can offer the same medicine from another maker. The customer compares them side by side and accepts or removes it within 15 minutes.
- Refill reminders with one-tap reorder.

Lab tests

- A 3-column test grid, then labs compared for home collection or a lab visit.
- Pay online and get reports.

Care Circle

- Family members see each other's visits and care logs.
- Nurses write a care log during each visit (vitals and notes).

Easy mode (app)

- Large text, fewer choices and big buttons for elderly users.
- A "Call me back" request goes to support.

Sign-in

- Email and password.
- Google: native account picker on Android, Google's button on the website.
- Phone OTP is built but switched off until an SMS provider is set up.

Partners

Nurses and physios: visits (accept, start, care log, complete), plus availability and reliability scores.

Care shops: service menu, prices, plans, offers and ads.

Riders: online switch, batched trips of up to 3 nearby drops, cash collection and earnings.

Pharmacy store management:

- **Today:** open/closed switch, sales against yesterday, new orders, and alerts for low, out-of-stock, expiring and off-sale stock.
- **Orders:** accept, verify prescriptions, pack, suggest substitutes, hand over with the customer's delivery code.
- **Stock:** search, filters, editing prices and counts, batch and expiry tracking, receiving stock, adding products, CSV import, the Schedule H1 register.
- **Money:** a sales statement (sales, commission, earnings, cash collected) with withdrawals.
- **Shop settings:** opening hours, delivery distance, fees, minimum order, packing time.

All partners

- Account, documents and verification, payouts and withdrawals, profile photo.
- **Photos and videos:** posts shown on their shop page.

Admin

- Care marketplace: shops, settings, call-backs and refunds.
- Ads and settings, with a propose → approve flow.
- Users, documents, staff checks and partner applications.
- Payments, withdrawals, campaigns and live logs.
- **Partner posts:** hide a post with a reason.
- Admin accounts require two-step sign-in (authenticator app).

Security and data

Uploads

- File types are checked from the file's contents, not its name.
- Partner media upload straight to private S3 through a one-time signed link fixed to that file's type and size. The server checks the stored file before publishing it.

Releases and secrets

- Android releases are signed with Nabz's own private key, not the public React Native debug key.
- The key file and its password are in AWS Secrets Manager.

Other protections

- The website sends security headers, including a Content-Security Policy, and blocks being framed by other sites.
- AWS WAF protects the API.
- Sign-in has rate limits and per-account logout.

Tests

- Integration tests run against a real MongoDB (local replica set) for:
 - store management;
 - substitution;
 - riders and refills;
 - the care marketplace;
 - call-backs;
 - Care Circle;
 - reliability;
 - partner posts.
- The fast suite (`npm test`) runs about 1,430 tests.

Operations runbook (staging)

How to deploy, build the Android apps, and manage sign-in and signing keys. This document lists where secrets are kept, never the secrets themselves.

Addresses

What	URL
API	<code>https://tiuh3tvxsa.ap-south-1.awsapprunner.com</code> (health: <code>/api/v1/health</code> shows <code>deploymentCommit</code>)
Website	<code>https://79fkmxu8w3.ap-south-1.awsapprunner.com</code>
Admin panel	website <code>/admin/login</code> ; requires an authenticator app
Pharmacy store dashboard	website <code>/vendor/today</code> , or the Nabz Partner app

AWS account `554433963412` , region `ap-south-1` . Always use the AWS CLI profile `nabz` (IAM user `nabz-deployer`). The default profile has no App Runner access.

Deploy the API and website

1. Commit and push the branch (`feature/nabz-care-marketplace`).
2. Upload the source and start both image builds:

```
git archive --format=zip -o source.zip HEAD
aws s3 cp source.zip s3://nabz-staging-build-554433963412/source.zip --profile nabz
C=$(git rev-parse HEAD)
aws codebuild start-build --project-name nabz-staging-images --profile nabz \
  --environment-variables-override name=TARGET,value=api,type=PLAINTEXT
name=DEPLOYMENT_COMMIT,value=$C,type=PLAINTEXT
aws codebuild start-build --project-name nabz-staging-images --profile nabz \
  --environment-variables-override name=TARGET,value=web,type=PLAINTEXT
name=DEPLOYMENT_COMMIT,value=$C,type=PLAINTEXT \
  name=API_ORIGIN,value=https://tiuh3tvxsa.ap-south-1.awsapprunner.com,type=PLAINTEXT
```

3. App Runner deploys each new image automatically, in about 10 minutes. Check that `/api/v1/health` shows the new `deploymentCommit` .

Infrastructure (Terraform)

- Folder: `terraform/apprunner-staging/` . State is in S3 (`backend.hcl`).
- `staging.auto.tfvars` is loaded automatically. It keeps `create_api = true` , `create_web = true` and the Google client IDs, so a plain `terraform plan / apply` can't delete the services or drop Google sign-in.
- **Always run** `terraform plan` , **read it, and apply that saved plan**. Never apply blind.

- The private uploads bucket has default encryption, public access blocked, CORS for the website's direct uploads, and a rule that deletes unpublished partner uploads (`partner-posts-pending/`) after one day.

Secrets (where they live)

Secret	Where
MongoDB connection string	AWS Secrets Manager <code>nabz/staging/MONGODB_URI</code> (pipe it into a process; never print it)
Generated app secrets (JWT etc.)	Secrets Manager, created by Terraform
Android release key file	<code>C:\mrb\keys\nabz-release.jks</code> , backed up in Secrets Manager <code>nabz/android/release-keystore</code>
Android key password	Secrets Manager <code>nabz/android/release-keystore-password</code>
Staging test accounts	<code>C:\mrb\nabz-test-accounts.txt</code> (local only, not in git)

Android apps

Build both APKs

```
powershell -NoProfile -ExecutionPolicy Bypass -File apk-build.ps1 # wraps C:\mrb\build-cloud.ps1
```

- The script copies `frontends/mobile` and `frontends/shared` to `C:\mrb`.
- It builds **Nabz** (customer) and **Nabz Partner** against the staging API, signs both with the release key, and saves `Nabz.apk` and `Nabz-Partner.apk` to the Desktop. The old APKs are replaced only after both builds succeed.
- It keeps the PC awake while it runs. A full build takes 30–70 minutes; the log is at `C:\mrb\build-cloud.log`.
- To rebuild only one app, call `build-cloud.ps1 -WebUrl <website> -Variants customer` (or `partner`).
- If a screen file is moved or renamed, delete the old copy under `C:\mrb\frontends\mobile\app`. The copy step adds and updates files but never removes them.

Signing key

- Release key: alias `nabz-release` , RSA 4096.
- SHA-1: `05:84:A4:CE:3B:F9:D1:D1:53:6B:0B:86:81:46:06:BF:29:70:6D:EA`
- SHA-256:
`34:3B:32:B8:E6:6C:E0:7F:C5:77:34:15:71:41:CB:96:68:42:1B:AB:59:3A:76:35:39:96:6D:8D:58:22:84:C0`
- The build signs through Gradle's `android.injected.signing.*` settings and reads the password from Secrets Manager at build time.
- **Never lose this key:** Play Store updates and Google sign-in depend on it. When the app moves to the Play Store, enrol in Play App Signing and keep this key as the upload key.

Google sign-in

Google Cloud project: **Nabz**. The app is in Testing mode, so only accounts listed as **test users** can sign in.

Client	ID	Settings
Web	542534084220-dkulc04guq24btt9vc7b37gom7h2df0.apps.googleusercontent.com	JavaScript origins: the website URL and http://localhost:3000
Android	542534084220-8eh8ru5aeum5in1esendnuv4hlcvbjk3.apps.googleusercontent.com	Package app.medrush.mobile, the SHA-1 above

- The API accepts ID tokens for both clients (`GOOGLE_OAUTH_CLIENT_IDS`) and gives the website its client ID (`GOOGLE_WEB_CLIENT_ID`).
- **Android** uses native sign-in (`@react-native-google-signin`). The token is issued for the **web** client ID, and Google checks the app's package and signing key against the Android client. Browser redirects with a custom scheme are blocked for new Android clients (Error 400: `invalid_request`).
- **Website** uses Google Identity Services in a popup. The Content Security Policy allows `accounts.google.com/gsi` only.
- **Before launch:** move the consent screen to production. That needs a real domain, a support email and a privacy-policy URL.
- **Pending:** rotate the web client secret that was exposed in a screenshot. Nabz doesn't use it, but rotate it anyway.

Tests

```
# Local MongoDB replica set on port 28017 (rs: testset), then:
MONGODB_URI="mongodb://127.0.0.1:28017/<db>?replicaSet=testset" npx jest <files> --runInBand
npm test          # fast suite (jest.fast.config.js)
npm run lint
```

PART 2

Product plans

Provider marketplace plan

Physio and lab marketplace: plan

7 Oct 2026. For the Nabz team: the new way customers choose and book physiotherapists and path labs, the edge cases, and what else should change.

The idea in one line

Physios and labs work like restaurants on Zomato or Swiggy. Each one has its own **menu** (rate card) with its own prices. The customer compares them and picks one. Nabz runs the shared catalog, the booking, the payment and the trust layer.

Zomato / Swiggy	Nabz
Restaurant	Physio (on their own), physio clinic, or path lab
Restaurant outlet	Branch (a clinic or lab can have several)
Dish	Service on the rate card: "Knee pain therapy, 45 min", "CBC test"
Same dish, different price per restaurant	Same service, each provider sets their own price
Delivery vs dine-in	Home visit vs at the clinic (or lab)
Delivery fee by distance	Travel fee, ₹10–15 per km
Quantity	Number of sessions
One restaurant per cart	One provider per booking
Restaurant "closed now"	Provider paused, on leave, or fully booked

Where we are today

Area	Today	Gap
Physio	One Nabz price per service for everyone. "Book now" offers the visit to the nearest online physio (Uber style). The customer can name a professional, but the price stays the same. Packages are fixed products: 5, 10 or 15 sessions.	No per-physio prices, no clinic visits, no travel fee, sessions are fixed products instead of a number the customer picks.
Path labs	Lab partner login and a "coming soon" page. A separate branch (<code>feature/medrush-core</code>) has a test catalog with one Nabz price per test .	Everything for a marketplace. That branch's test details (sample type, fasting, report time) can be reused; its pricing cannot.
Addresses	The server can store several addresses per customer.	The apps and website do not offer them. Home visits and travel fees depend on the address, so this now matters.

How it works for the customer

1. **Pick what you need.** Search or browse services ("Back pain", "Thyroid test") or browse providers near you.
2. **Compare providers.** Each card shows the price for that service, home or clinic, distance, rating, reviews, gender, languages, experience, next free slot, and the travel fee to your saved address.
3. **Open a provider.** You see their full rate card, like a restaurant menu: services, price at the clinic, price at home, session length, and multi-session discounts.
4. **Choose home or clinic.**
 - **Clinic:** you see the address, a map and the distance. No travel fee.
 - **Home:** pick a saved address or add one. Nabz works out the distance and shows the travel fee. If the address is beyond the physio's home-visit radius, that physio shows "Clinic only for this address".
5. **Choose the number of sessions:** 1 to 30. For more than one, pick the days of the week and the time. The physio's discount for more sessions applies automatically, for example "10% off from 10 sessions".
6. **See the full bill before paying:** service × sessions, discount, travel fee × home sessions, Nabz fee, tax, total. Then pay: the whole plan now, or session by session.
7. **During the plan:** the same physio every session. Reschedule a session, change the address for one session (the travel fee is worked out again), or change physio for the rest of the sessions.

Labs work the same way. The customer picks tests or a checkup package, compares labs (price, NABL accreditation, report time, home collection), picks home collection or a walk-in visit, and books. A cart holds tests from **one lab** only, because the samples go to one lab.

How it works for the provider

- **Shop profile:** name, photos, qualification, council registration, clinic address and hours, languages, bio.
- **Rate card:** chosen from the Nabz catalog (no made-up services). For each service:
 - offered at the clinic: yes or no, and the price;
 - offered at home: yes or no, and the price (defaults to the clinic price);
 - session length;
 - optional discounts for more sessions, for example 5+ sessions 5% off, 10+ sessions 10% off.
- **Home visits:** how far they travel (radius, up to 25 km), their rate per km (from ₹10 to ₹15), and where trips start from (the clinic or their home base).
- **Calendar:** weekly hours, separately for clinic and home; how many patients the clinic can see at once (beds or physios); leave days; a buffer between home visits for travel.
- **Pause switch:** "Not taking new bookings", like a restaurant closing for the day. Existing bookings stay.
- **Labs also set:** report time per test, which tests can be collected at home, the home collection fee (the same per-km rule, or free above a basket value), and their phlebotomists.

Prices and fees

The **travel fee**, worked out by Nabz, never typed in by the provider or the app:

travel fee = road km from the provider's start point to the visit address (rounded up to the next km) × the provider's ₹/km, with a minimum fee (Nabz setting, default ₹30)

- Road km is the straight-line distance × 1.3 until a maps service (Google Distance Matrix or OSRM) is connected; then it is the real road distance.
- It is charged once per home session: each session is a separate trip.
- It is one-way, not a round trip.
- The travel fee goes to the provider in full, with no Nabz commission on it, because it covers their fuel and time.

Worked example: 10 home sessions of knee therapy with Dr. A

Line	Amount
Knee therapy at home, ₹600 × 10	₹6,000
Dr. A's discount for 10+ sessions, 10%	– ₹600
Travel: 6.2 km → 7 km × ₹12 = ₹84 × 10	₹840
Nabz fee (on the service, not the travel)	as set in <code>config/revenue.js</code>
Tax	see "Tax" below

Price guardrails. Each catalog service has a Nabz price floor and ceiling, for example knee therapy from ₹300 to ₹2,500 per session.

- The floor stops ₹1 listings meant to win the ranking, followed by a request for cash at the door.
- The ceiling stops price gouging.
- An admin can approve a price outside the band (for example, a senior sports physio).

Locked prices. The bill the customer sees is a **quote**, held for 15 minutes. Booking uses the quote, not the prices the app sends. If the provider changes a price before the customer pays, the customer sees "price changed" and confirms again. Once booked, every session of the plan keeps the booked price, even if the rate card changes later.

Paying for several sessions

- **Pay as you go:** each session is paid after it happens (cash or online), as today.
- **Pay for the plan now:** the customer pays the whole plan upfront and gets the multi-session discount. Nabz holds the money and pays the physio **after each completed session**, through the settlement ledger that already exists. If a physio stops coming, the unused money is still there to refund.
- **Refund of unused sessions:** what was paid, minus the completed sessions at the **non-discounted** price, minus any cancellation fees, and never below zero. This stops "buy 10 for the discount, use 3, refund 7 at the discounted rate".

- **Plans expire:** sessions must be used within the number of sessions × 2 weeks, with at least 4 weeks. The customer can ask for one extension. Unused sessions after expiry are refunded by the rule above.

Edge cases and how we handle them

Prices and rate cards

Situation	What happens
The physio changes a price while the customer is checking out	The quote has expired or no longer matches, so the customer is shown "Price changed from ₹600 to ₹650" and confirms again.
The physio changes a price in the middle of a 10-session plan	The remaining sessions keep the booked price.
The physio stops offering a service at home while home sessions are booked	Booked sessions stand. The provider is warned while editing: "You have 6 upcoming home sessions for this. They stay booked." To drop them, the provider must release them, and the customer is offered a clinic visit, another physio or a refund.
A price outside the floor and ceiling	Rejected with the allowed range, unless an admin has approved it.
The same person opens two shops to appear twice in search	One shop per council registration number (unique); a clinic lists its physios inside one shop.

Home visits and distance

Situation	What happens
The address is outside the physio's radius	Home is not offered for that address. The card says "Clinic only for this address" or the physio is hidden from home-visit results.
A customer fakes the map pin near the physio to pay less travel	The pin must fall inside the area of the pincode they typed (within about 15 km of the pincode centre). At arrival, the physio's phone location is recorded with the visit code. If the real visit place is more than 2 km from the booked pin, the visit is flagged and ops can charge the difference.
A river or one-way streets make the road much longer than the straight line	The 1.3 multiplier covers most of it. The provider can raise a distance dispute from the visit. Moving to a maps service removes the issue.
The patient moves in the middle of a plan (for example to a daughter's house after surgery)	Change the address for the remaining sessions or for one session. The travel fee is worked out again; if the new address is out of range, offer clinic sessions, another physio or a refund for the rest.
Two family members need physio at the same address back to back	The second session at the same address, next to the first, has no travel fee (one trip).
The physio is stuck in traffic	Live ETA (exists today). A 15-minute grace period. After 30 minutes late, the customer can cancel for free and the physio gets a reliability strike.

Sessions, plans and calendars

Situation	What happens
Two customers grab the same slot at the same moment	Slots are reserved with an atomic counter that has the provider's capacity as its limit (1 for a solo home physio, N for a clinic with N beds). The second customer gets "That time was just taken" and the next free times.
A home session runs late and the next home visit is across town	Home slots include the travel buffer the physio sets (default 30 minutes). Slots that cannot be reached in time are not offered.
The physio takes leave during a plan	Sessions on leave days are flagged. The customer chooses: move them, take a substitute for those days only, or refund those sessions.
The physio is suspended or loses verification in the middle of a plan	Sessions are released (exists today). The customer gets a choice of another physio at the same or lower price, or a refund of the unused sessions.
The customer misses a clinic session (no-show)	That session is used up. For pay-as-you-go, a no-show fee as in the cancellation rules.
The customer is not home for a home visit	The travel fee is not refunded, plus the late-cancellation fee that exists today.
The physio does not turn up	Full refund for that session, a ₹100 credit to the customer, and a reliability strike for the physio. Three strikes in 30 days: paused from new bookings.
After the first visit the physio says another service is needed (booked "back pain", needs "neuro rehab")	The physio sends a plan proposal from their app: service, number of sessions, price. The customer accepts and pays the difference, or declines. Nothing changes without the customer's consent.
The physio wants to charge an assessment first	"Assessment" is a catalog service. Many physios will list it, and a plan usually starts with it.
The session runs over time	No automatic extra charge. The physio can add an extension the customer approves in the app.
The plan expires with sessions left	One extension on request, otherwise a refund by the rule above.
A female patient wants a female physio at home	A gender filter on search, plus the "preferred gender" that exists today.
A son abroad books for his mother	"Book for someone else" (already planned): the visit code and updates go to the mother's phone, and the son pays.

Labs

Situation	What happens
The customer wants 5 tests and the chosen lab offers only 4	The lab card says "4 of your 5 tests", with the missing one named. The customer can choose another lab or remove the test.
A test cannot be collected at home (for example a glucose tolerance test needs 3 draws over 2 hours)	Marked "lab visit only" in the catalog. A cart with it offers only a lab visit, or splits into two bookings with the customer's consent.
A fasting test	Only morning collection slots (6 to 10 am) are offered, with a reminder the evening before.
The sample is rejected (clotted, too little)	Free re-collection booked by the lab. The report time restarts.

Situation	What happens
The report is later than the promised time	Automatic credit to the customer. It counts against the lab's rating.
A test that needs a doctor's prescription or a consent form	The catalog marks it; checkout asks for the upload or consent first.
A checkup package compared with buying the tests one by one	The package card shows "You save ₹X" against the same lab's test prices.
Who sees the report	The patient. The booker only with the patient's consent (see the "book for someone else" plan).

Payments and trust

Situation	What happens
The provider asks for extra cash at the door	All prices come from the rate card. The customer has a "Was asked to pay extra" button on the visit. Confirmed cases mean a strike and possible suspension.
The customer and physio continue outside the app after the first visit (common in physio)	Make staying worth it: plan discounts, session notes and an exercise plan in the app, receipts that work for insurance, the visit code and tracking for safety, and a lower Nabz commission for repeat patients of the same physio. Phone numbers stay masked.
Fake reviews	Only completed bookings can be rated (exists today). Ratings are kept per provider and per service.
Rounding	All amounts are worked out in paise (whole numbers) and shown in rupees.

Tax: please confirm with your CA

Today Nabz adds 18% GST on the visit price **and** the Nabz fee. Under GST exemption notification 12/2017 (entry 74), health care by clinical establishments, registered practitioners and paramedics (physiotherapy and lab diagnostics are normally included) is **exempt**. If that holds, customers are overpaying 18% on the care itself today. The usual setup is:

- no GST on the physio's or lab's service or on the travel fee;
- 18% GST on the Nabz platform fee only.

This should be a setting, switched only after the CA confirms.

What changes in the system

New

Part	What it holds
Provider shop (CareStore)	Type (solo physio, physio clinic, path lab), owner, name, photos, council registration or NABL number, address and map point, hours, home radius, ₹/km, travel start point, capacity, pause switch, approval status, rating. Built like the pharmacy store model, which already works.

Part	What it holds
Rate card item (RateCardItem)	Shop + catalog service; clinic on/off and price; home on/off and price; session length; multi-session discounts; for labs, report time and home collection. Built like the pharmacy's store-inventory model (store + product + price).
Quote	The locked bill: lines, totals, expiry, a fingerprint of the prices used. Booking requires a valid quote.
Care plan	One booking of N sessions: provider, service, home or clinic, sessions booked and used, locked prices, payment mode, expiry, refund state. Each session stays a visit as today, linked to the plan.
Slot reservation	One record per provider and time slot, with a counter and capacity, so two customers can never take the last place at the same time.
Address book (UI)	Saved addresses on app and website: label ("Mom's home"), map pin, default address. The server part exists.

Changed

- **Catalog:** each service gets a price floor and ceiling, a "can be done at home" flag, and (for labs) sample type, fasting, report time and "needs prescription". Fixed package products (5, 10 and 15 sessions) are retired: the number of sessions becomes a choice, not a product.
- **Visit (booking):** gains shop, practitioner, home or clinic, travel details (km, ₹/km, fee), price snapshot, plan link, and the quote it came from.
- **Search:** the nearest shops offering a service, filtered by home or clinic, distance, price, rating, gender and language. Uses a map index on the shop location.
- **Payouts:** prepaid plans are paid out to the provider per completed session. Travel fees go to the provider with no commission.

Suggestions for the rest of Nabz

1. **One booking engine for every service.** Nursing, physio, labs and (later) consults all become "a booking with line items from a provider". Today the booking has a hard-coded list of service names, and each new service needs a code change. Services should come from the catalog. One engine means one place for payments, cancellations, refunds, the visit code, tracking and reviews.
2. **One partner structure for everyone:** organisation (legal entity, KYC, bank account, GSTIN) → branches (location, hours, radius) → people (verification) → rate card. Today pharmacies, nurses and physios each have their own version of KYC, payouts and verification. One structure removes the duplicates and lets a clinic run nursing, physio and a pharmacy under one account.
3. **Keep Uber-style matching for urgent nursing.** An injection or IV drip "now" is about who is nearest and free, not about choosing. Physio, labs and planned nursing suit the marketplace. Both can share the same shops and rate cards; "Book now" just picks the best nearby shop automatically.
4. **Quotes everywhere.** Pharmacy checkout, nursing and labs should all book from a locked quote, so the app can never decide the price and a price change mid-checkout is handled the same way everywhere.
5. **Money in paise.** All amounts as whole numbers of paise, to end rounding drift between the bill, the payment and the payout.

6. **Real road distance.** Connect a maps service (Google Distance Matrix, or OSRM run by Nabz) for travel fees and ETAs, cached by area.
7. **Fix GST** as above, after the CA confirms.
8. **Ratings per service and per practitioner.** A clinic can be great at sports injuries and average at neuro rehab.

Build order

Step	What	Where
1	Shops, rate cards, price guardrails, quote and travel fee engine, search, care plans with N sessions, slot reservation, refunds of unused sessions. Bot tests for every edge case above.	Server
2	Customer: browse and compare physios, provider menu, home or clinic, address book, sessions picker, bill, plan page.	Website and apps
3	Partner: shop setup, rate card editor, home radius and ₹/km, calendar and leave, plan proposals.	Website and partner app
4	Labs on the same engine: lab shops, test rate cards, one-lab cart, home collection, sample tracking, reports.	Server, website and apps
5	Move existing physio services and providers over; retire the fixed package products.	Server (migration script)

Build status (7 Oct 2026)

Step 1 (server) is built and tested: 19 unit tests and 27 end-to-end tests against a real database. All of it lives under `/api/v1/marketplace`:

- **Shops:**
 - verified physios go live at once; labs and unverified professionals wait for review;
 - one shop per registration number;
 - only safe fields are editable by the shop.
- **Rate cards:**
 - clinic and home prices inside the catalog floor and ceiling (an admin can approve a different range);
 - multi-session discounts;
 - every price change is versioned.
- **Search and compare:**
 - home results only show shops whose area covers the address, with the travel fee;
 - sort by recommended, price, distance or rating;
 - no contact details or private home base in public results.
- **Quotes:**
 - the full bill with travel ₹/km, locked for 15 minutes;
 - GST switch (off until the CA confirms).

- **Plans of 1–30 sessions:**
 - pay per session or prepaid (discount);
 - all sessions' times reserved atomically (two customers can never take the last slot, a 2-bed clinic takes exactly 2);
 - a double tap books once;
 - a price change asks for confirmation.
- **During a plan:**
 - move a session;
 - change the address (travel re-priced, out of range refused);
 - cancel a session or the plan;
 - the professional releasing a session, leave days and suspension all hand the session back to the customer (not to dispatch);
 - short-notice releases add a strike, and three strikes pause the shop.
- **Money:**
 - travel goes to the professional with no commission;
 - prepaid plans pay the professional per completed session;
 - unused sessions are refunded at the list price (the discounted price if the provider was at fault);
 - unpaid holds expire;
 - a late payment is fully refunded.
- **Reports:** "asked to pay extra" and "didn't come" flag the visit for ops.

Built since (8 Oct 2026): steps 2–4 are live on staging.

- **Labs** on the same engine: one-lab cart, compare, fasting mornings, collection code, sample steps, report upload and private report links, rejected samples with free re-collection, late-report credit, pay at collection or online.
- **Home care** with the same named caregiver every day, night shifts across midnight, live-in care with travel charged once, agency teams.
- **Customer screens** on the website and both apps: care hub, compare, shop menu, booking with a live bill, plans, lab tests, saved addresses, Nabz credit, plan suggestions.
- **Partner screens** on the website and the partner app: My Shop (profile, hours, rate card, team, leave and pause, plans), ads, lab desk, "Suggest a plan" after a visit.
- **Admin** on the website: shop review, catalog, refunds, reports, needs-action queue, sessions and lab boards, offers, ads and settings (propose → approve).
- **Also done:** arrival location check, free second trip for family at one address, no-show credit through the customer wallet, online payment for plans, lab tests and ad top-ups (Razorpay; needs the live keys).

Still open:

Item	Note
Moving existing physio services and providers over; retiring the fixed packages	Step 5

Item	Note
Pin-vs-pincode check at address entry	The arrival check covers fake pins today
Changing physio in the middle of a plan	Today: cancel the rest (refunded), then book the new physio
Real road distance (maps service) and the GST switch after the CA confirms	Settings ready

Admin and ads guide

Admin controls and ads: guide

7 Oct 2026. For the Nabz team: everything the admin panel should control, and how to place ads the way Swiggy and Zomato do, adapted for health care.

Part 1 lists what the admin must be able to do: what exists today and what to add, in priority order. Part 2 is the ads guide: the ad products, where they go, how they are ranked, the health-care rules, and the admin controls over all of it.

Priority labels:

- **Launch:** needed before real customers.
 - **Soon:** within the first months.
 - **Later:** when the volume justifies it.
-

Part 1: What the admin controls

Today's admin panel

These already exist:

- **Live logs.**
- **Payments.**
- **Users:** search, with masked details.
- **Partner documents:** each view is logged.
- **Verification:** ID, police check and council.
- **Partner applications.**
- **Withdrawals.**
- **Revenue.**
- **Push campaigns.**
- **Security:** sensitive actions need a fresh two-step code, and an audit log records who did what.
- **Pharmacy vendor approval:** server side.
- **Marketplace (new today, server only, no admin screen yet):** approve or suspend a shop, approve an out-of-band price, add a reliability strike.

Rules for every admin action

1. **Least privilege:** use separate admin roles instead of one all-powerful admin.
 - Super admin.
 - Operations: bookings, partners.
 - Finance: refunds, payouts.

- Support: customers.
- Content: banners, ads, catalog.

Today there is only `platform_admin`. **Launch.**

2. **Fresh two-step code** for anything touching money, partner trust or personal data. This exists today; keep it on every new action.
3. **Maker-checker for money and prices:** one admin proposes, a second approves. This covers refunds above ₹5,000, fee changes, commission changes and manual payouts. **Soon.**
4. **Everything is logged:** who, what, before and after, why (a reason is required). The audit log exists today; extend it to every new action.
5. **Changes never rewrite the past.** New fees and prices apply to new bookings only. Booked visits and plans keep their locked prices (the marketplace already works this way).
6. **Personal data stays masked** until a reason is given. The view is logged.

What to add, by area

1. Shops (physios, clinics, labs, pharmacies)

Control	Why	Priority
Review queue for new shops: approve, reject (with reason), ask for changes	Labs and unverified professionals wait for review (built on the server; needs the screen)	Launch
Suspend / reinstate a shop	Booked sessions are released to their customers automatically (built)	Launch
See a shop's rate card, price history and upcoming bookings	Answer customer complaints; spot price games	Launch
Approve a price outside the catalog band for one shop	For example, a senior sports physio above the ceiling (built on the server)	Launch
Reliability: strikes, no-shows, late releases, cancellation rate; three strikes in 30 days auto-pauses the shop (built); admin can lift the pause	Keep quality up without manual policing	Launch
Licence and accreditation expiry (council registration, NABL, drug licence) with automatic pause on expiry	Legal requirement; pharmacy already does this	Soon
Mark a lab "NABL accredited" (badge) after checking the certificate	Badge must be earned, not self-declared	Launch
Duplicate detection: same registration number, phone, bank account or PAN across shops	One person must not list twice	Soon
Add or remove a clinic's practitioners	Clinics with several physios	Soon
Lower commission for a shop for a period (launch partner deals)	Onboarding incentives	Later

2. Catalog (services, tests, packages)

Control	Why	Priority
Add / edit / retire catalog services: name, description, category, session length	Shops can only list catalog services	Launch
Price floor and ceiling per service (later per city)	Stops ₹1 listings and gouging	Launch
Home allowed / clinic allowed per service	Some services must not be done at home	Launch
Lab tests: sample type, fasting hours, home collection possible, default report time, needs prescription or consent	Drives slots, reminders and checkout rules	Launch (with labs)
Checkup packages: which tests are inside	"You save ₹X" must be true	Launch (with labs)
Sort order, "popular" flag, icons and images	Home and browse pages	Soon
Suggest-a-service queue (shops request a service that isn't in the catalog)	Grow the catalog safely	Later

3. Prices, fees and rules

Today these live in `config/revenue.js` and need an environment-variable change and a redeploy. Move them to an admin settings page with a history, maker-checker and a start date.

Setting	Today's value	Priority
Nabz fee on care visits	15%	Launch
Provider commission tiers	20% / 15% / 12% by monthly volume	Launch
Travel band: min and max ₹/km, minimum fee, road factor, max radius	₹10–15, ₹30, ×1.3, 25 km	Launch
Plans: max sessions, weeks per session, quote validity, prepaid hold time	30, 2 weeks, 15 min, 20 min	Soon
Late cancellation fee and grace period	₹100	Launch
Delivery fee, free-delivery threshold, night and surge charges (pharmacy)	₹25, ₹499, ₹20, ×1.5 / ×2	Soon
Nabz Plus price and trial length	₹149, 7 days	Soon
GST: "health care exempt" switch (only after the CA confirms)	Off	Launch

4. Bookings and plans (daily operations)

Control	Why	Priority
Live board: today's visits by status, late arrivals, visits that haven't started on time	Ops sees problems before customers call	Launch
"Needs action" queue: sessions a provider released (marketplace) and visits with no staff (dispatch)	Call the customer, suggest another provider	Launch
Reported problems: "asked to pay extra", "didn't come"; resolve with a strike, refund or credit	Built on the server as flags; needs the screen and the outcomes	Launch

Control	Why	Priority
SOS queue with live location and one-tap call	Safety (SOS exists; needs a dedicated screen)	Launch
Cancel any visit or plan with a reason (free for the customer when Nabz or the provider is at fault)	Clean exits	Launch
Move a plan's remaining sessions to another provider, with the customer's consent and the price difference handled	Provider suspended in the middle of a plan	Soon
Extend a plan's expiry once	Customer in hospital, travelling	Soon
Labs: sample tracking board (collected, in transit, at the lab, report ready), late reports, rejected samples and free re-collection	Lab operations	Launch (with labs)

5. Money

Control	Why	Priority
Refund queue: prepaid plan refunds (the server already calculates them), pharmacy and visit refunds; approve, send through Razorpay, mark processed	Customers' money	Launch
Payouts and withdrawals (exists) plus a hold on a partner's payout during a dispute	Fraud and complaints	Launch
Cash mismatches: "cash short" visits (already flagged) and partners' cash balance owed to Nabz	Pay-after-visit leaks	Launch
Settlement ledger per partner and per booking (exists on the server)	Answer "why was I paid ₹X"	Soon
Customer dues: view, waive (with reason)	Goodwill after a bad experience	Soon
Credits / wallet for compensation (for example ₹100 after a provider no-show)	Promised in the marketplace plan; needs a wallet	Soon
GST and TDS reports, invoices, monthly statements for partners	Compliance	Soon
Chargebacks and disputes	Payment gateway disputes	Later

6. Customers

Control	Why	Priority
Search, view (masked), booking history (exists)	Support	Launch
Block / unblock (abuse, fraud) with reason	Safety for partners	Launch
Grant or extend Nabz Plus	Goodwill, partnerships	Soon
Data deletion and data export requests (DPDP Act)	Legal	Launch
Consent records (book for someone else, report sharing)	Legal	Soon

7. Quality and trust

Control	Why	Priority
Review moderation: hide abusive or fake reviews (never hide a review just because it is negative)	Fair ratings	Soon
Suspicious patterns: many 5-star reviews from new accounts, the same device for the customer and the provider, refunds abuse	Rating and money fraud	Later
Strike and auto-pause thresholds	Tune the reliability rules	Soon

8. Areas and launch

Control	Why	Priority
Cities and areas on or off per service (physio, labs, pharmacy, nursing)	Launch city by city	Launch
Area stress lever (exists for pharmacy): shrink delivery radius in rain or rider shortage	Keep promises realistic	Exists
Max home-visit radius per city	Big cities vs small towns	Soon

9. Content and messages

Control	Why	Priority
Push campaigns (exists)	Re-engagement	Exists
Home banners managed by admin (today they are written in code)	See Part 2	Soon
Announcements inside the apps (outage, new city)	Operations	Soon
Terms, privacy policy and FAQ versions, with customers asked to accept new terms	Legal	Soon

10. Ads

All the controls are in Part 2, section 8.

Part 2: Ads guide

1. How Swiggy and Zomato do it

- **Sponsored listings:**
 - Restaurants pay to appear higher in search and category results, marked "Ad" or "Promoted".
 - They pay per click or per order.
 - An ad only appears if the restaurant is open, delivers to you and matches what you searched.
- **Home carousel and banners:** big tiles on the home screen, sold per week per city, often for brand campaigns.
- **Offer badges:** "50% off up to ₹100", paid for by the restaurant, shown on its card and in an "Offers" filter.

- **Featured collections:** "Top rated near you", curated by the platform; partly paid, mostly earned.
- **Sponsored products** (Instamart): brands pay to appear first in product search.
- **Self-serve for partners:** set a budget and bid, see clicks, orders and return on spend.

What makes it work: relevance comes first.

- An ad only appears where an organic result could have appeared.
- Only a limited number of ad slots exist.
- Bad partners (low rating, closed, out of range) can't buy their way in.

2. What is different for health care

Ads must never put a customer's health or trust at risk. These rules are not optional:

1. **Safety gates before money.** An ad is eligible only if the provider is:

- verified and approved, not paused or suspended;
- offering the exact service and mode the customer is looking at;
- in range of the customer's address;
- free in the next 7 days;
- rated at least 4.0 with 10 or more reviews (new providers have a separate free boost; see section 5).

2. **Clearly labelled** "Sponsored" on every ad. The ASCI code and the CCPA 2023 guidelines on dark patterns ban disguised ads.

3. **No ads in sensitive moments:**

- emergency or SOS;
- live visit tracking;
- payment and checkout;
- viewing lab reports or health records;
- right after a cancellation by the provider.

4. **No medicine promotion that the law forbids.** Under the Drugs and Magic Remedies Act and the Drugs and Cosmetics Rules:

- no ads for prescription medicines (Schedule H, H1, X);
- no "cure" claims.

Pharmacy ads are limited to OTC, wellness and devices.

5. **No targeting on health data without consent (DPDP Act).**

- Target by **context**, meaning what the customer is browsing right now. Example: on the knee pain page, show knee physios.
- Never target by their medical history, lab results or diagnoses.

6. **Price honesty:**

- The price on an ad must be the provider's real rate-card price.
- An offer badge must really apply at checkout.

Both are checked automatically against the rate card.

7. **Ads never outrank safety.** A sponsored provider never pushes out the nearest available one in an urgent "Book now" flow. Urgent nursing gets no ads at all.

3. The ad products

#	Product	What it is	How it's sold	Where
1	Sponsored listing	A physio, lab or nursing shop shown in a sponsored slot in search and category results	Cost per click (CPC) with a minimum bid; daily budget	Physio and lab search, service pages ("Knee pain therapy"), "Physios near you"
2	Offer badge	"10% off your first session", "Free home collection", funded by the provider	Free to run (the provider pays the discount), admin approved; shown in an "Offers" filter	On the provider card and shop page
3	Home spotlight tile	A tile in the home carousel for a provider or lab package	Fixed price per week per city area (cost per thousand views, CPM, later)	Customer home, max 1 of the 4 tiles
4	Category banner	A banner at the top of a service page ("Full body checkup from ₹999 at XYZ Labs")	CPM, one advertiser per page per week	Lab tests home, physio home
5	Sponsored product	An OTC or wellness product first in pharmacy search	CPC, brands or stores	Pharmacy search and categories, OTC only
6	Helpful cross-sell (house ad, not paid)	"Need a knee brace?" after booking knee physio	Free; Nabz's own pharmacy	Booking confirmation screen, one card

Not recommended:

- **Push or SMS ads:** they annoy users and cost uninstalls. Keep push for Nabz's own campaigns, which exist.
- **Pop-ups or full-screen ads:** never.

4. Where ads go (placement map)

Screen	Slots	Max ads	Format
Customer home	Carousel (4 tiles)	1 sponsored tile	Spotlight tile
Physio or lab search results	Positions 2 and 6	2 in the first 10, never position 1	Sponsored listing (same card as organic, plus "Sponsored")
Service page ("Knee pain therapy")	Top banner + positions 2 and 6	1 banner + 2 listings	Category banner, sponsored listing
Shop page	None	0	(No competitor ads on a provider's own page)

Screen	Slots	Max ads	Format
Pharmacy search	Positions 1 and 5	2 in the first 10	Sponsored product
Booking confirmation	One card below the details	1	House cross-sell only
Checkout, payment, tracking, SOS, reports, records	None	0	None, ever

Why position 1 is never sponsored for providers: the first result should be the best match for the patient. Swiggy also often keeps the top organic result.

5. How ads are ranked

Each sponsored slot runs a small auction among eligible ads (eligible means it passed the safety gates):

$\text{ad score} = \text{bid} \times \text{quality}$

$\text{quality} = \text{rating (from 0 to 1)} \times \text{completion rate} \times (1 - \text{cancellation rate}) \times \text{relevance}$

Relevance comes from distance (or home-radius fit), the service match and how soon the provider has free slots.

- **The winner pays just enough to beat the next ad** (second-price), never more than their bid. This stops bidding wars.
- **Minimum CPC:** for example ₹5 for physio, ₹3 for labs, ₹1 for products; admin can change it.
- **No double appearance:** if a provider is already in the organic top 3, their ad doesn't show in that list (the slot goes to the next ad).
- **Frequency cap:** the same ad at most 3 times per customer per day.
- **Pacing:** the daily budget is spread across the day, so an ad doesn't burn out by 10 am.
- **New provider boost (free, organic):** for their first 30 days, new verified providers get extra organic visibility. Otherwise only those who already have reviews can win, and new physios never get started.
- **Clicks that count:** the same customer clicking the same ad twice in a day counts once. Bots and the provider's own account never count.

Attribution: a booking within 7 days of clicking an ad counts as "from the ad", shown to the advertiser as orders and return on spend.

6. Self-serve for partners (partner app and website)

1. Pick a goal: "more bookings for knee therapy", "promote my full body checkup".
2. Pick the product: sponsored listing, offer badge, spotlight.
3. Pick the area: their city areas, by default within their home-visit radius.
4. Set a daily budget and a max CPC (Nabz suggests a bid).
5. Write the offer (if any), checked against their rate card.
6. Pay from an **ad wallet** (prepaid top-up, GST invoice). Spend is taken per click.
7. See results daily: views, clicks, bookings, spend, cost per booking.

Ads go live after an admin approval for the first campaign. Later campaigns from trusted advertisers go live automatically, with spot checks.

7. What the system needs

Part	What it holds
Ad campaign	Advertiser (shop or store), product, placement, target (city, areas, services, home or clinic), bid, daily and total budget, start and end, status (draft, in review, active, paused, rejected, ended), creative (title, image, offer)
Ad events	Views and clicks per ad per day (counted, not stored one by one forever), booking attribution
Ad wallet	Top-ups, spend per day, refunds of unused balance, GST invoices
Placement settings (admin)	Per placement: on or off, max ads, positions, min CPC, rating floor, cities
Ad server	One function each list page calls: "give me up to N ads for this context". It applies the safety gates, the auction, caps, pacing and logging, and returns the ads already labelled

The ad server sits **inside** the existing search (for example `careStoreService.searchStores`): organic results first, then ads merged into the allowed positions. That way an ad can never point to a provider who isn't open, in range or offering the service.

8. Admin controls for ads

Control	What it does
Kill switch	Turns all ads off instantly (global, per city, per placement)
Placement settings	Positions, max ads per list, min CPC, rating and review floors, frequency cap, which screens allow ads
Review queue	Approve or reject advertisers and creatives, with a reason; banned words list ("cure", "guaranteed", medicine brand names on care ads)
Pause any ad	One tap, with a reason, logged
Category rules	Which categories may advertise where (for example no ads on emergency; pharmacy OTC only)
Advertiser trust	Auto-approve trusted advertisers; block an advertiser
Budgets and billing	Wallet balances, top-ups, refunds, invoices, credit for disputed clicks
Fraud review	Click patterns per advertiser (same device, bursts), with refunds of invalid clicks
Reports	Per placement and per city: ad revenue, CTR, bookings from ads, and the important one: did ads lower booking rates? If customers book less where ads show, reduce the ad load
House ads	Admin's own cards (Nabz Plus, new city, a health-awareness week) using the same slots when no paid ad qualifies
Experiments	Show ads to a share of customers (for example 50%) and compare bookings before rolling out

9. Rollout plan

Phase	What	Why
A: Launch	House ads only (Plus, new services), offer badges funded by providers, admin-curated "Featured" (free). The kill switch and placement settings are built.	Build the slots and rules with no money at risk; learn what customers click
B: After about 1,000 bookings a month in a city	Sponsored listings with CPC, ad wallet, partner self-serve, admin review queue, reports	Enough traffic for ads to work for partners
C: Later	Home spotlight and category banners (CPM), pharmacy sponsored products (OTC brands)	Brand money once Nabz has reach

10. Numbers to watch

Metric	Healthy	Act if
Ads in the first 10 results	≤ 2	Never more
Booking rate on lists with ads vs without	Within 5%	Ads lower bookings more than 5%: reduce the ad load or raise the quality floor
Ad CTR (sponsored listing)	2–6%	Below 1%: ads are irrelevant; tighten targeting
Complaints mentioning "ad" / "sponsored"	Near zero	Any rise: review placements
Share of revenue from ads	Grows slowly	Never let ads replace earning trust

Book for someone else

28 Sep 2026. For the Nabz team: what exists, what is missing, and the build plan.

Many Nabz customers book for someone else: a son in Bengaluru booking a nurse for his mother in Jaipur, a daughter ordering her father's medicines. The person who books and pays (the **booker**) is not the person who receives care (the **recipient**). Every service needs to handle that the same way, in the apps and on the website.

Where we are today

Service	Today	Gap
Home nurse / physio visit	"Me / Someone else" on web and app. Recipient name, age, gender. On-site contact name and phone. App picks from phone contacts and remembers recent people on the phone. Family tracking link.	Bug fixed today: the recipient's name, age and gender were not being saved, so the professional only saw the account holder. The visit code still only reaches the booker's app, so a parent in another city cannot give it to the nurse.
Physio packages	Same visit form in the app. The same professional comes each session.	Website package booking is not built yet. Progress notes are not shared with the booker.
Pharmacy	App: deliver to someone else with receiver name and phone. Website: a contact phone on checkout.	No "someone else" choice on the website. The delivery code goes to the booker only. A prescription is not checked against the recipient's name.
Lab tests	Not built on this branch (module is on <code>feature/medrush-core</code>).	Everything.
Doctor consult	Not built on this branch.	Everything.
Emergency	Not built on this branch.	Everything.
Nabz Plus	Covers the account holder only.	No family cover.

The design: saved people ("Family & friends")

One list of people per customer account, stored on the server, used by every service, and the same on app and website.

A saved person has:

- name, relation (mother, father, spouse, child, friend, other), age or date of birth, gender;
- phone number (optional; needed for codes and updates);
- one or more addresses;
- care notes the professional should know: allergies, conditions, mobility, language;
- consent record: when the booker confirmed they have the person's permission.

Everywhere a service asks "Who is this for?": Me, one of my saved people, or add someone new (from phone contacts in the app, typed on the website). Recent picks come first.

How each service works

Service	Who receives codes and updates	What the professional sees	Special rules
Nurse / physio visit	The visit code and live tracking link go by SMS to the on-site contact (usually the recipient). The booker gets every status update in the app.	Recipient name, age, gender, relation, care notes, on-site contact.	Booker pays and can cancel. The professional calls the on-site contact, not the booker.
Physio package	Same as a visit, for every session.	Same, plus notes from earlier sessions.	The package belongs to the recipient. Changing the professional needs the booker. Session notes are shared with the booker if the recipient agrees.
Pharmacy	The delivery code goes to the receiver by SMS. The booker gets order updates.	Receiver name and phone.	A prescription must be in the recipient's name; the pharmacist checks it. Prepaid by the booker, or cash on delivery from the receiver.
Lab test	Collection time and fasting instructions to the on-site contact. The report goes to the recipient's number, and to the booker's account only if the recipient agrees.	Recipient name, age, gender; collection address.	The phlebotomist confirms identity at collection (name and age; photo ID where the test needs it).
Doctor consult	The video link goes to the recipient (or on-site contact). The booker can join.	Recipient details and care notes.	The recipient must be present. The prescription is issued in the recipient's name.
Emergency	Help goes to the recipient's address. The booker gets live updates.	Recipient details, care notes, on-site contact.	No consent step: in an emergency, acting fast comes first.
Nabz Plus	Not applicable.	Not applicable.	A family plan covering up to 4 saved people.

Consent and privacy (DPDP Act 2023)

- **Booker's confirmation:** when adding an adult, the booker ticks "I have their permission to book care for them". For a child or a dependent adult, the booker confirms they are the guardian.
- **Recipient is told:** an adult with their own phone gets an SMS: "Rohit booked a Nabz nurse for you on 2 Oct, 10 am. Not you? Tap to cancel."
- **Health information stays with the recipient:** lab reports and session notes reach the booker only with the recipient's consent.
- **Easy to remove:** the booker can delete a saved person at any time; their bookings keep only what the law requires.
- **Only what is needed:** each service asks for the minimum it needs.

Build plan

Phase A: visits and pharmacy (about 3 days)

1. Saved people on the server (API, app, website), replacing "recent people" stored on the phone.
2. "Who is this for?" on every booking and checkout, on the website as well as the app.
3. Visit and delivery codes plus the tracking link go to the on-site contact:
 - by SMS once an SMS provider is set up;
 - until then, a "Share with them" button sends the code and link over WhatsApp from the booker's phone.
4. The professional's view shows "For Kamla Devi (Mother, 72)" with care notes. (Name, age and gender work from today's fix.)

Phase B: pharmacy checks and family plan (about 2 days)

1. The pharmacist checks the prescription name against the recipient.
2. The delivery code goes to the receiver.
3. The Nabz Plus family plan.

Phase C: labs, consults, emergency (with each module)

1. Build "for someone else" into lab tests, consults and emergency as each module is ported from `feature/medrush-core`.

Edge cases to handle

- **Same person saved twice:** match on phone number and offer to merge.
- **No phone number:** the booker must give an on-site contact, and codes go to that contact.
- **Visit cancelled on site:** if the recipient declines, the professional marks it "Patient declined" and the booker is told. Cancellation fees follow the normal rules.
- **Shared phone:** a husband and wife share one number. Allow one phone on several saved people.
- **Minors:** a guardian must be present at the visit; the professional confirms this on arrival.
- **Recipient later makes their own account:** offer to link their history, with their consent.

Decisions needed

1. **SMS provider:** MSG91, Gupshup or Twilio India. In India, transactional SMS needs sender and template registration on the telecom DLT portal, which takes a few days.
2. **Limit on saved people:** we suggest 6 per account.
3. **Nabz Plus family plan price.**
4. **Report sharing:** should lab reports go to the booker by default when the recipient has no phone of their own?

Pharmacy marketplace plan

Revised implementation plan (supersedes `implementation_plan11.docx` / "MedRush MVP"). Date: 2026-09-22 · Owner: Chirayu Mishra · Stack decision: **Next.js (web) + React Native/Expo (mobile)** on the existing Nocturnal Node/Express/MongoDB backend.

Companion docs: [MEDRUSH DATA MODEL.md](#) — collections ("tables"), relationships, indexes, and how to add a model · [MEDRUSH GETTING STARTED.md](#) — simple architecture + how to run backend/web/mobile locally.

1. What changed vs. the original plan

The original "MedRush MVP" was a quick-commerce layer for **home medical services** (nursing, lab tests, doctor consults, emergency SOS) on company-dispatched staff. Three directives from the product owner reshape it:

1. **Fulfillment = marketplace of local pharmacy vendors, not dark stores.** Instead of owning inventory (Zepto/Blinkit/Flipkart-Minutes model), we **aggregate independent local pharmacies** (Swiggy/Zomato-style). This adds a *Pharmacy Vendor* org entity, per-vendor inventory & pricing, order routing to a chosen/nearest vendor, a vendor dashboard, and a delivery-partner (rider) role.
2. **Frontend is Next.js + React Native**, replacing the Vanilla-JS PWA for the new product surfaces. The existing Express API is kept as the backend.
3. **Scope = full breadth** (pharmacy + home nursing + lab tests + doctor consult + emergency SOS), **web + mobile built in parallel**, vendors **admin-seeded with a vendor dashboard**.

Corrections to inaccurate assumptions in the original doc

- ❌ "Socket.IO already in the codebase / WebSocket ready." **False** — there is no `socket.io` dependency and no references. Real-time (order/staff tracking, chat, dispatch) must be **added** (`socket.io` server + client) or done via polling for the first slice. Planned below.
- ⚠️ "~60% of the backend is already built." True for **auth, RBAC, Patient/User/NurseBooking/ServiceCatalog/Payment/Review models, uploads (S3/GCS), Razorpay, Firebase push, rate-limiting/security stack**. The pharmacy marketplace, lab, consult, emergency, and real-time layers are **new**.
- ✅ Reuse confirmed: `middleware/auth.js` (`protect` , `authorize(...roles)`), `middleware/patientAuth.js` , `constants/roles.js` + `constants/enums.js` as the single source of truth, `routes/v1/index.js` route composition, `utils/uploadEnhanced` , `models/payment.js` , `models/review.js` , `models/notification.js` .

2. Roles & auth

Role	Model	Auth	Onboarding
Patient	Patient	Phone OTP + optional email/password (existing patientAuth)	Self-register
Pharmacy Vendor (store staff)	User role pharmacy_vendor , scoped to a PharmacyVendor org	Email + password	Admin-seeded/approved (MVP), self-serve later
Delivery Partner / Rider	User role delivery_partner	Phone + password	Admin-verified
Medical Staff (nurse/paramedic)	User role nurse (existing) or new medical_staff	Phone + password + cert upload	Admin-verified
Phlebotomist (lab sample collection)	User role phlebotomist	Phone + password + cert	Admin-verified
Doctor	User role doctor (existing)	Email + password + MCI verify	Admin-approved
Admin / Platform Admin	User role admin / platform_admin (existing)	Email + password	—

New roles added to `constants/roles.js` (`ROLES` , `ALL_ROLES` , `ROLE_PERMISSIONS`) **and** `constants/enums.js` (`STAFF_ROLES`) so both `authorize()` and the `User` schema enum accept them.

3. Architecture & repo layout

Keep the Express monolith as the API. Add frontends in an **isolated top-level** `frontends/` **directory** that is *not* matched by the root `workspaces globs` (`packages/*` , `apps/*`) — this keeps the heavily CI-gated backend dependency tree, lint (`eslint` .), and deploy gates untouched by React/Next/Expo dependencies.

```
/ # existing Express backend (unchanged install/CI)
  routes/ controllers/ models/ constants/ middleware/ services/ ...
frontends/
  shared/ # @medrush/shared – TS domain types + typed API client (fetch), zero heavy
deps
  web/ # Next.js 15 (App Router, TS, Tailwind) – patient storefront +
vendor/doctor/admin dashboards
  mobile/ # Expo (React Native, TS, expo-router) – patient app + vendor app (role-
aware)
docs/
  MEDRUSH_PHARMACY_MARKETPLACE_PLAN.md # this file
```

- `frontends/shared` is the contract layer: one place for TS types (`PharmacyOrder` , `Medicine` , `LabTestBooking` , ...) and a typed `ApiClient` used by both web and mobile. Consumed via a local path dependency + `tsconfig` path alias.
- Root `.gitignore` and ESLint `ignores` updated so `frontends/**/node_modules` and build output don't enter backend tooling.
- Mobile stays on **Expo managed workflow** (fastest path, OTA, push, maps, camera for prescription capture). The existing Capacitor Android wrapper is retired for the new app.

Real-time strategy

- Add `socket.io` to the backend, mounted on the same HTTP server (`server.js`), namespaced: `/rt/orders` , `/rt/tracking` , `/rt/consult` , `/rt/emergency` . Rooms keyed by `orderId` / `bookingId` / `userId` . Auth via the existing access token in the socket handshake.
- Web uses `socket.io-client` ; mobile uses the same. **Fallback:** status polling every 5–10s where sockets are unavailable, so the product works before real-time lands.

4. Data model

New models

- `models/pharmacyVendor.js` — store profile: `name` , `owner` (User), `drugLicenseNumber` , `gstIn` , `address` + `location` (GeoJSON Point, 2dsphere), `serviceRadiusKm` , `operatingHours` , `status` (`PENDING` / `APPROVED` / `SUSPENDED` / `REJECTED`), `payout` details, `rating` , `isOpen` .
- `models/medicine.js` — global master catalog: `name` , `genericName` / `composition` , `brand` , `manufacturer` , `form` (`TABLET` / `SYRUP` / ...), `strength` , `packSize` , `scheduleType` (`OTC` / `PRESCRIPTION` / `SCHEDULE_H`), `category` , `hsn` , `images` , `isActive` .
- `models/vendorInventory.js` — per-vendor listing: `vendor` , `medicine` , `mrp` , `sellingPrice` , `stockQty` , `isAvailable` ; unique compound index { `vendor` , `medicine` } .
- `models/pharmacyOrder.js` — `patient` , `vendor` , `items[]` (medicine + qty + price snapshot), `prescription` (upload ref), `status` (`PLACED` → `ACCEPTED` → `PREPARING` → `READY_FOR_PICKUP` → `OUT_FOR_DELIVERY` → `DELIVERED` / `REJECTED` / `CANCELLED`), `deliveryAddress` + `location` , `rider` (User), `amounts` (items/delivery/tax/total), `payment` (Payment ref), `timeline[]` , `requiresPrescription` .
- `models/labTest.js` + `models/labTestBooking.js` — catalog (category, sample type, pricing, prep, TAT, partner) + booking (tests, collection slot, phlebotomist, `sampleStatus` , `reportUrl` , doctor ref).
- `models/consultation.js` — `patient` , `doctor` , `type` (`CHAT` / `AUDIO` / `VIDEO`), `status` , `prescription` , linked `labTestBookings` , `fees` , `duration` .
- `models/emergencyBooking.js` — `patient` , live `location` , `urgencyLevel` , `condition` , assigned `staff` , `responseTimeMs` , emergency-contact notifications.

Modified models (additive only)

- `models/user.js` — `pharmacyVendor` ref, `currentLocation` (GeoJSON Point, 2dsphere), `isOnline` , `isAvailable` , `servicesOffered[]` , `vehicle` (rider), `averageResponseTime` .

- `models/patient.js` — `savedAddresses[]` (label + address + GeoJSON), `defaultAddressId`, `emergencyContacts[]`, `preferredLanguage`.
- `constants/enums.js` — new role arrays; `MEDICINE_FORMS`, `MEDICINE_SCHEDULE_TYPES`, `MEDICINE_CATEGORIES`, `PHARMACY_ORDER_STATUSES`, `PHARMACY_VENDOR_STATUSES`, `DELIVERY_STATUSES`, `LAB_TEST_CATEGORIES`, `SAMPLE_STATUSES`, `CONSULTATION_TYPES`, `CONSULTATION_STATUSES`.

ERD (additions)

```

Patient ||--o{ PharmacyOrder : places
Patient ||--o{ LabTestBooking : orders
Patient ||--o{ Consultation : requests
Patient ||--o{ EmergencyBooking : triggers
Patient ||--o{ NurseBooking : books          (existing)

PharmacyVendor ||--o{ VendorInventory : stocks
PharmacyVendor ||--o{ PharmacyOrder : fulfills
Medicine ||--o{ VendorInventory : listed_as
Medicine ||--o{ PharmacyOrder.items : ordered_in

User(delivery_partner) ||--o{ PharmacyOrder : delivers
User(doctor) ||--o{ Consultation : conducts
User(phlebotomist) ||--o{ LabTestBooking : collects
User(nurse/medical_staff) ||--o{ NurseBooking : serves

PharmacyOrder ||--o| Payment : generates
LabTestBooking ||--o| Payment : generates
Consultation ||--o| Payment : generates

```

5. API surface (mounted under `/api/v1` via `routes/v1/index.js`)

Pharmacy (patient)

- `GET /pharmacy/vendors/nearby?lat&lng` — open vendors within radius
- `GET /pharmacy/medicines/search?q&vendorId` — search catalog / a vendor's stock
- `POST /pharmacy/orders` — place order (with prescription upload ref)
- `GET /pharmacy/orders/:id` / `GET /pharmacy/orders` — track / list
- `POST /pharmacy/orders/:id/cancel`

Pharmacy (vendor dashboard) — `authorize('pharmacy_vendor')`

- `GET/POST/PATCH /pharmacy/vendor/inventory` — manage stock & pricing
- `GET /pharmacy/vendor/orders` — incoming orders
- `PATCH /pharmacy/vendor/orders/:id/status` — accept/reject/prepare/ready
- `PATCH /pharmacy/vendor/profile` — hours, open/closed

Delivery — `authorize('delivery_partner')`

- `GET /delivery/tasks`, `PATCH /delivery/tasks/:id/status`, `PATCH /delivery/location`

Lab tests — `/lab-tests/search`, `/lab-tests/book`, `PATCH /lab-tests/:id/status`, `GET /lab-tests/:id/report` **Consultations** — `POST /consultations`, `PATCH /consultations/:id`, chat via socket **Emergency** — `POST /emergency`, `PATCH /emergency/:id` **Staff** — `PATCH /staff/location`, `GET /staff/nearby`, `PATCH /staff/availability` **Admin** — `/admin/vendors` (approve/suspend), `/admin/medicines` (master catalog), `/admin/staff` (verify)

All new routers follow the existing pattern: per-domain file in `routes/`, wired into `routes/v1/index.js`, protected by `protect` / `authorize` or `patientAuth`, validated with `express-validator`, using `constants/enums.js` values.

6. Frontend surfaces

Next.js web (`frontends/web`)

- **Patient storefront:** `/` home (categories: Pharmacy · Nursing · Lab Tests · Consult · Emergency), `/pharmacy` vendor + medicine browse, `/cart`, `/checkout` (address + prescription upload + pay), `/orders/[id]` live tracking, `/lab-tests`, `/consult`, `/emergency`, `/account`.
- **Vendor dashboard:** `/vendor` (orders queue, accept/reject), `/vendor/inventory`, `/vendor/profile`.
- **Doctor portal:** `/doctor` consult queue + e-prescription.
- **Admin:** `/admin/vendors`, `/admin/medicines`, `/admin/staff`.
- SSR/SEO for storefront; auth via HTTP-only cookies against the Express API.

Expo mobile (`frontends/mobile`)

- **Patient app:** home, pharmacy browse/cart/checkout (camera prescription capture), order tracking (map), lab booking, consult (chat/audio), one-tap Emergency SOS with live location.
- **Vendor app:** order alerts (push), accept/prepare, inventory quick-edit.
- **Rider app (phase 2):** task list, navigation, status updates, live location publish.
- expo-router file-based navigation; role chosen at login; Firebase push via `expo-notifications`; maps via `react-native-maps`.

Both consume `@medrush/shared` for types + `ApiClient`.

7. Payments & delivery

- **Razorpay** (already integrated) for patient → platform; vendor payouts tracked on `PharmacyVendor.payout` (manual/settlement for MVP).
- **Payment timing:** prepaid for pharmacy/lab/consult; **post-pay** for emergency SOS (per original doc's recommendation).
- **Delivery (MVP):** vendor marks `READY_FOR_PICKUP`; a `delivery_partner` is assigned (manual/nearest) → `OUT_FOR_DELIVERY` → `DELIVERED`. Third-party logistics integration deferred.

8. Phased execution roadmap

Phase 1 — Foundations (backend + scaffolds)

- Roles + enums; new models (`pharmacyVendor` , `medicine` , `vendorInventory` , `pharmacyOrder`); additive `User / Patient` fields.
- Pharmacy routes/controllers (patient browse + order, vendor inventory + order status) wired into `/api/v1` .
- Scaffold `frontends/shared` , `frontends/web` (Next.js), `frontends/mobile` (Expo). Root ignores/lint updated.
- Seed script: sample vendors + medicines + inventory.

Phase 2 — Pharmacy end-to-end (web + mobile in parallel)

- Patient storefront (web) + patient app (mobile): browse → cart → prescription upload → checkout (Razorpay) → order.
- Vendor dashboard (web) + vendor app (mobile): order queue accept/reject/prepare/ready; inventory management.
- Order status tracking (polling first, then socket).

Phase 3 — Real-time + delivery

- Add `socket.io` ; live order/rider tracking on map; delivery partner app; push notifications for new orders.

Phase 4 — Medical services breadth

- Home nursing (reuse `NurseBooking`) UI; lab test catalog + booking + sample tracking + report delivery; doctor consultation (chat/audio, e-prescription) ; emergency SOS.

Phase 5 — Polish & launch

- Admin panels (vendor/staff verification, medicine master), analytics/monitoring, e2e tests, performance (Redis geo-cache), Play Store / Vercel deploy.

9. Testing & deploy gates

- Backend: extend Jest suites — model unit tests + integration tests for pharmacy order lifecycle, vendor authz, lab/consult/emergency. Respect `npm test` (fast config) vs `npm run test:all` .
- Frontends: component tests + Playwright e2e for the storefront funnel; Expo EAS build checks.
- Keep the existing deploy-gate contract tests green; add new contract tests rather than disabling any.
- `npm run verify:local` (lint + fast tests) before each PR; Conventional Commits; branch from `develop` .

10. Open decisions (resolved / remaining)

-  Frontend: Next.js + Expo.  Scope: full breadth.  Build order: parallel.  Vendors: admin-seeded + dashboard.
-  Lab partner strategy (own catalog for MVP vs. Thyrocare/1mg API) — **own catalog for MVP.**

- 🕒 Video consult in MVP — **chat + audio first**, video (Daily.co/Agora) in a later phase.
- 🕒 Emergency scope — limit to serviceable first-aid/IV/wound care + nearest-staff dispatch; ambulance integration later.
- 🕒 OTP provider (MSG91/Twilio) — wire when patient OTP is enabled; email/password works today.

Pharmacy store network plan

Pharmacy store network: plan, design and edge cases

Nabz is a marketplace of independent chemists. Stock is not in our dark stores (as with Blinkit or Zepto): it sits on shelves we don't control, and walk-in customers buy from them all day. Most failures come from the gap between what the app thinks a store has and what is actually on the shelf. This document covers how we close that gap, what is built, and what comes next.

Status key: **Built** = in this branch with tests · **Next** = planned for the next phase · **Later** = needed before scale, not yet planned in detail.

1. What we took from Blinkit, Zepto, Zomato, Swiggy and Uber Eats

Lesson	Where it comes from	What we did
A partner that doesn't respond must not stall the order. After a timeout the platform auto-rejects or reassigns and tells the customer.	Zomato POS integration docs, "Order Inaction" and "Restaurant Rejects Order" flows	3-minute accept window (configurable). Silence moves the order to the next best store. The reason a store gives decides the next step.
Stock accuracy comes first. Blinkit's case studies describe fixing store processes and discrepancy rates <i>before</i> launching customer-facing availability features.	Blinkit out-of-stock case study	Every listing carries <code>stockUpdatedAt</code> . Old counts are labelled "likely available" and ranked lower. Stores get a one-tap "my counts are right". A store that marks an item unavailable has that count zeroed.
"Listed" and "in stock" are different things. Running out must not delist an item.	Blinkit (already in <code>models/vendorInventory.js</code>)	Kept, and stock rules (expiry, bans) are layered on top.
Missing items are handled per item: replace (store picks), substitute (customer picked), or refund. The customer approves a replacement, and a rejected replacement is refunded automatically.	Uber Eats retail order fulfilment guide	Stores mark single items "not available": the item is dropped, the customer is refunded, and the rest of the order continues. Substitutes are offered to the customer and never swapped silently , because a prescription brand is a medical decision.
Separate "order received / paid" from "order confirmed", and make order creation idempotent.	Zepto on AWS (DynamoDB draft-order service)	Prepaid orders stay invisible to the store until paid (existing). Order create is idempotent (existing middleware). The store's acceptance clock starts at payment, not at checkout.
Refresh hot data in the background instead of waiting for cache misses.	Zepto performance engineering blog	Not needed yet at our volume. Listed under Later (availability cache per geohash).
Refunds for missing or damaged items; returns only for damaged or wrong items.	Swiggy / Instamart refund policy	Partial refunds are built. The returns flow is Later , and returned medicine never goes back to sellable stock.

Sources are listed at the end.

2. How it works now

```
customer searches → availability engine → stores that can sell it now, ranked
                                     ↳ same-salt substitutes if few or none
customer cart → cart planner → one store with everything
                                     ↳ else a 2-store split
                                     ↳ else best partial + substitutes for the rest
checkout → product rules + store gate + atomic reserve + price-changed guard
store turn → accept within SLA → ACCEPTED
        ↳ silent → next store (or cancel + refund), store auto-paused after
repeated misses
        ↳ rejects: stock/busy/closed → next store
        ↳ rejects: prescription → cancel + refund
        ↳ some items missing → drop + refund those, zero the count, carry on
```

2.1 One definition of "sellable"

`services/pharmacyAvailabilityService.js` and `services/serviceabilityService.js` are the only places that decide this. Search, cart planning, checkout and reassignment all use them, so they can't disagree.

A listing is sellable when **all** of these hold:

- **The store can trade now:** approved, active, the owner's switch is on, not auto-paused, drug licence not expired, and inside its opening hours in **India time** (overnight hours handled). Servers run in UTC, so a naive hour check would be 5½ hours off.
- **The store delivers to the customer:** zone-aware radius and stress multiplier (existing).
- **The product may be sold online:** not banned, not discontinued, not Schedule X.
- **Enough units** of stock, listed by the store.
- **Enough shelf life:** the listing's `expiryDate` is further away than `PHARMACY_MIN_SHELF_LIFE_DAYS` (default 30).
- **Cold-chain products** only from stores with `hasColdStorage`.
- **Prescription products** only from stores that take prescription orders.

2.2 Ranking

Lower score wins. Full cart coverage always beats partial.

```
score = ETA minutes + 0.05 × subtotal (₹20 ≈ 1 min) + 30 × (1 - acceptance rate) + 5 × stale items
acceptance rate = (accepted + 9) / (offered + 10)
```

The acceptance rate is smoothed so a new store starts near 90% and moves with evidence.

2.3 Acceptance SLA and reassignment

`services/pharmacyAssignmentService.js`, with a worker started from `server.js`:

- **COD orders** start the clock at checkout. **Prepaid orders** start it when payment lands (the store can't see them before that).
- **The worker** runs every 20 s. It moves overdue `PLACED` orders to the next store that has **every remaining item at no more than the price the customer agreed to**. If the new store is cheaper, the difference is refunded. At most 3 stores are tried (configurable), then the order is cancelled and refunded in full.
- **Store records:** `reliability.offered / accepted / rejected / timedOut / itemsMarkedUnavailable`. Three misses in a row, or a "we're closed" rejection, auto-pauses the store for 30 minutes. This uses `pausedUntil`, and never touches the owner's own open/closed switch.
- **Nurse supplies:** for home-care supply orders, the linked nurse booking is updated to the new store, or marked cancelled.
- **Customer updates:** the customer gets an in-app notification and a push for every move, missing item and cancellation.

2.4 Concurrency rule (the part that must not be broken)

Every order move is **one compare-and-set** on the order document, filtered on the state we read (status, current store, length of the attempts log). Whoever wins the update does the side effects: releasing or reserving stock, refunds, store records. A loser undoes only what it did itself, for example a reservation at the new store.

This fixes a real bug that existed before this work. `updateOrderStatus` and `cancelOrderByPatient` did read → check → restock → save. A store rejecting while the customer cancelled could restock twice, inflating the store's count, and try two refunds. Tests now race these cases and assert that stock moves exactly once.

2.5 Money

- **Partial refunds** (`refunds[]` on the order) are queued, locked per entry and sent to Razorpay with the entry id in `notes`. A retry first lists the payment's refunds and matches that id, so a refund whose database write was lost is recorded, not paid twice. Failed refunds are retried by the worker and raise a monitoring alert.
- **The money invariant:** `amounts.refunded + amounts.total = amounts.originalTotal`.
- **Store payout** uses `amounts.itemsSubtotal` at delivery, which already excludes dropped items.

3. Edge-case catalogue

Stock and catalogue

#	Case	Status	How
1	Walk-in sales make the app's count wrong	Built	Acceptance SLA, per-item "not available" zeroes the count, stock freshness labels, one-tap confirm
2	Two customers buy the last unit	Built (existing)	Atomic conditional <code>\$inc</code> reservation
3	Same medicine added twice to a cart	Built	Lines merged before reserving

#	Case	Status	How
4	Stock expiring soon	Built	Minimum shelf life on search, planning and reservation
5	Several batches with different expiries	Built	<code>InventoryBatch</code> per store and product: earliest expiry sold first, batch recorded on each order line, near-expiry batches quarantined by the worker, batch recalls
6	Banned, discontinued or Schedule X product	Built	Blocked everywhere, including items already in a cart and listings made before the ban
7	Cold-chain product (insulin)	Built	Only stores with a fridge
8	Misuse-prone products (codeine syrups, sedatives)	Built	Per order (<code>maxQtyPerOrder</code>), per patient per 30 days across all stores (<code>maxQtyPerMonth</code>), risk flags (<code>MANY_STORES</code> , <code>EARLY_REFILL</code> , <code>MONTHLY_LIMIT_NEAR</code>) with an admin review list
9	Brand out of stock, but the same salt is available	Built	Salt key; substitutes offered, never auto-swapped
10	The same product entered twice in the master list	Built	Admin merge moves listings, stock, batches, alerts and demand; refuses products with different compositions; open orders restock onto the surviving product
11	Selling price above MRP	Built (existing check, plus <code>MRP > 0</code>)	· Government price caps (DPCO/NPPA) Later
12	Price changed while the item sat in the cart	Built	Client sends <code>quotedSubtotal</code> ; the server returns 409 with the new price instead of charging a different amount
13	Price per tablet vs per strip	Built for comparison (<code>packUnits</code>) · selling loose strips Later	
14	Store lists a product it may not sell online	Built	Listing refused for banned and Schedule X

Stores

#	Case	Status	How
15	Store doesn't respond	Built	SLA, reassignment, auto-pause
16	Store forgot to switch off and rejects with "closed"	Built	Pause for 30 min
17	Opening hours, including past midnight, in India time	Built	<code>isWithinOperatingHours</code>
18	Drug licence expired	Built	Dropped from search and checkout automatically
19	Store doesn't take prescription orders	Built	Filtered in planning and checkout
20	Store accepts, then can't fulfil	Built	"Can't fulfil" with a reason: reassign or cancel and refund

#	Case	Status	How
21	Pharmacy chains with several branches and shared pricing	Later	Organization → stores; staff roles per branch
22	Holidays and festival closures	Later	Date-based closures on top of weekly hours

Orders and money

#	Case	Status	How
23	Some items missing	Built	Drop those lines, refund them, carry on
24	Reassigned store is dearer or cheaper	Built	Never dearer; cheaper refunds the difference
25	Customer cancel vs store decline, or store accept vs timeout worker	Built	Compare-and-set; race tests
26	Refund sent twice (retry after lost write)	Built	Per-entry lock + gateway lookup by entry id
27	Orders placed before this release	Built	All updates work when the attempts log is missing
28	Payment arrives after the order was cancelled	Built (existing)	Auto-refund
29	Cart split across two stores at checkout	Built (cash on delivery) · one online payment across stores Next	<code>POST /pharmacy/checkouts</code> creates child orders all-or-nothing
30	Supplies for a nurse visit reassigned	Built	Booking follows the order
31	Damaged or wrong item delivered	Later	Return flow; returned medicine is never restocked (ledger <code>RETURN_DAMAGED</code>)
32	Refund gateway down	Built	Queued, retried, alerted

Compliance

#	Case	Status	How
33	Pharmacist verifies the prescription; Schedule H1 register	Built	Packing is blocked until the pharmacist records the doctor, their registration number and the prescription date (max age <code>PHARMACY_RX_MAX_AGE_DAYS</code> , default 180, which is a business rule to confirm with your pharmacist); H1 register export per store (CSV, formula-safe) and for admins
34	Prescription reused for refills beyond the prescribed quantity	Later	Track dispensed quantity per prescription
35	Prescriptions stay private	Built (existing)	Private storage, owner-scoped links

Operations and scale

#	Case	Status	How
36	Several API instances run the worker at once	Built	Compare-and-set, plus an overlap guard in each process
37	Demand nobody nearby can meet	Built	<code>PharmacyDemandSignal</code> per medicine, ~5 km cell and day; "what customers near you couldn't find" for stores, and an admin view
38	"Notify me when back in stock"	Built	Alert fires once when a store that delivers to the customer goes from 0 to in stock; alerts lapse after 14 days
39	Stores keep stock in billing software (Marg, GoFrugal, Busy)	Built CSV upload: barcode or exact name applies, anything else waits in a review queue, and confirmed barcodes are learned · Later direct sync	
40	Availability load at scale	Later	Cache per geohash with background refresh (Zepto pattern)

Phase 2 notes

- **Batch-tracked items** keep the listing count equal to the sum of the ACTIVE batch counts. A product switches to batches only once its untracked count is zero, and after that the store updates batch counts, not the listing count. Units from a batch that was quarantined or recalled while an order held them are never put back on sale.
- **A recall** pulls the batch at every store. Open orders holding it get a timeline warning so the store swaps units, and customers who received it can optionally be notified.
- **Monthly caps** are checked at checkout. Two orders placed at the very same moment can both pass; the review flags and the H1 register catch that rare case (a hard guarantee needs a per-patient lock).
- **Split checkout** is cash on delivery only. One online payment across several stores needs a shared gateway order, which is still **Next**.

4. Code map

Piece	File
Rules and tunables	<code>config/pharmacyOps.js</code>
Store gate, hours (IST), trading filter	<code>services/serviceabilityService.js</code>
Availability, cart plan, substitutes, demand	<code>services/pharmacyAvailabilityService.js</code>
Accept, decline, missing items, timeouts, worker	<code>services/pharmacyAssignmentService.js</code>

Piece	File
Partial refunds + retry	services/pharmacyPaymentService.js
Checkout rules, compare-and-set cancel and progress, stock confirm	services/pharmacyService.js
Models	models/medicine.js (salt key, sale flags), models/vendorInventory.js (stockUpdatedAt), models/pharmacyVendor.js (licence, pharmacist, fridge, reliability, pause), models/pharmacyOrder.js (item status, acceptBy , attempts, refunds), models/pharmacyDemandSignal.js
API	GET /pharmacy/medicines/:id/availability , POST /pharmacy/cart/plan , POST /pharmacy/vendor/orders/:id/items/unavailable , POST /pharmacy/vendor/inventory/confirm ; the status update accepts reasonCode and unavailableMedicineIds ; order create accepts quotedSubtotal
Store screens	frontends/mobile/app/vendor.tsx , frontends/web/app/vendor/page.tsx : accept countdown, reject reasons, per-item "Not available", "Can't fulfil", confirm counts
Customer screens	App pharmacy tab and web /pharmacy : "Find nearby" (other stores and same-salt brands); checkout sends quotedSubtotal
Phase 2 services	services/pharmacyBatchService.js (batches, earliest-expiry-first, quarantine, recall), services/pharmacyComplianceService.js (monthly caps, risk flags, Rx verification, H1 register), services/pharmacyCatalogService.js (merge, CSV import, demand), services/pharmacyStockAlertService.js , services/pharmacyCheckoutService.js
Phase 2 API	Store: /vendor/inventory/batches (list, receive, recount), /vendor/inventory/import (CSV) and /vendor/inventory/imports/:id/rows/:line , /vendor/orders/:id/prescription/verify , /vendor/register/h1?format=csv , /vendor/demand . Customer: /medicines/:id/notify-me , /checkouts . Admin: /admin/medicines/:id/merge , /admin/recalls , /admin/demand , /admin/orders/flagged , /admin/register/h1
Tests	tests/integration/pharmacy-store-network.test.js and pharmacy-store-network-phase2.test.js (real MongoDB, including races), tests/unit/pharmacy/store-network-rules.test.js

5. Settings and rollout

Env var	Default	Meaning
PHARMACY_ACCEPT_SLA_SECONDS	180 (staging: 600 via Terraform)	Store acceptance window
PHARMACY_MAX_ASSIGNMENT_ATTEMPTS	3	Stores tried before cancelling
PHARMACY_AUTO_PAUSE_AFTER_MISSES / PHARMACY_AUTO_PAUSE_MINUTES	3 / 30	Auto-pause
PHARMACY_MIN_SHELF_LIFE_DAYS	30	Expiry cut-off for online sale

Env var	Default	Meaning
PHARMACY_STALE_STOCK_HOURS	48	When a count becomes "likely"
PHARMACY_MAX_SPLIT_STORES	2	Cart split limit
PHARMACY_ASSIGNMENT_WORKER_ENABLED	true	Set <code>false</code> to stop the worker (also runs batch quarantine and alert expiry every 10 min)
PHARMACY_RX_MAX_AGE_DAYS	180	Oldest prescription date a pharmacist can accept

Rollout steps:

1. Deploy the API.
2. Run `npm run db:indexes` (adds the demand-signal and order sweep indexes).
3. Run `npm run db:backfill:pharmacy-catalog -- --write` (salt keys for existing medicines).
4. Deploy the website and rebuild the apps.

Existing listings show as "likely available" until stores confirm or recount. This is intended.

Sources

- Zomato POS integration docs: [Order inaction](#), [Restaurant rejects order](#), [Zomato rejects order](#)
- [Uber Eats: retail order fulfilment](#)
- [How Zepto scales to millions of orders per day using Amazon DynamoDB \(AWS\)](#)
- [Performance engineering at Zepto, part 2](#)
- [Solving the out-of-stock problem: a product case study on Blinkit](#)
- [Swiggy refund and cancellation policy](#)

Revenue model

How Nabz makes money, the launch numbers, and where each rule lives in the code. Status: implemented on branch `cClaude/pharmacy-vendor-app-website-12957b` (2026-09-23). Launch city: **Jaipur**.

1. Summary

Nabz earns on both sides of every transaction, like Swiggy, Zomato and Urban Company:

#	Stream	Who pays	Launch rate	Live in code
1	Care commission	Nurse / physio (from the visit price)	20% of base price	✓
2	Care platform fee	Customer	15% of base price (waived for Plus)	✓
3	Pharmacy commission	Pharmacy (from the item value)	10% of items	✓
4	Delivery fee + surge + night	Customer	Store fee (default ₹25) × zone surge (1.5× high, 2× severe) + ₹20 after 10 pm	✓
5	Nabz Plus membership	Customer	₹149 / 30 days, 7-day free trial	✓
6	Care packages (10 physio sessions, 30-day elderly care, 14-day post-surgery)	Customer, upfront	Catalog package price	✓ (catalog)
7	B2B: hospitals (post-discharge care), insurers (home visits), corporates	Institutions	Contract	Planned
8	Sponsored listings for pharmacy brands	Brands	CPM / CPC	Planned

Core bet: home-care visits are the high-value, high-repeat product. Pharmacy delivery adds frequency and pays for the supplies nurses bring. Plus locks in the families who book often.

2. How each stream works

2.1 Home-care visits (nurse / physio)

Customer pays = base price + platform fee (15%) + GST on (base + fee)
 Provider earns = base price - commission (20%)
 Nabz keeps = commission + platform fee

Example (IM injection, base ₹299, non-member):

Line	Amount
Base price	₹299.00
Platform fee (15%)	₹44.85
GST 18% on ₹343.85	₹61.89
Customer pays	₹405.74
Provider payout (₹299 – 20%)	₹239.20
Nabz revenue (₹59.80 commission + ₹44.85 fee)	₹104.65

Plus members pay no platform fee: ₹299 + ₹53.82 GST = **₹352.82**. Nabz still earns the ₹59.80 commission.

Supplies the nurse brings ("staff brings it") are a separate pharmacy order with nurse pickup (no delivery fee). They follow §2.2.

2.2 Pharmacy orders

Customer pays = items + delivery fee
 Pharmacy earns = items – commission (10%)
 Nabz keeps = commission + delivery fee (funds riders)

Delivery fee rules (checked in this order):

1. **Nurse pickup** (visit supplies): ₹0.
2. **Plus member**: ₹0, always, including surge and night.
3. **Basket ≥ ₹499**: base fee waived. Surge and the night surcharge still apply, because riders cost more in rain and late at night.
4. **Otherwise**: store base fee (or the ₹25 platform default) × zone surge, plus ₹20 between 22:00 and 06:00 IST.

Surge follows the ops "stress" lever on each service zone: raise a zone to HIGH or SEVERE in heavy rain or during a rider shortage, set an expiry, and it reverts automatically.

Example (₹350 basket, store fee ₹25, normal time): customer pays ₹375, the pharmacy earns ₹315, Nabz keeps ₹35 + ₹25 = **₹60**.

2.3 Nabz Plus (₹149 / 30 days)

- Benefits: free delivery on every pharmacy order, no platform fee on visits, priority support.
- **One free 7-day trial per customer, ever.**
- Paid plan through Razorpay: the server checks the signature, re-fetches the payment and matches amount, currency and order before activating.
- Renewals stack after the current period, so no days are lost.
- **Break-even for the customer**: about 2 visits (2 × ₹44.85 saved) or 6 deliveries a month. That makes it attractive to families who book often, the customers worth keeping.

3. Unit economics (illustrative, Jaipur launch assumptions)

These are planning assumptions for review, not data. Revisit after 60 days of real orders.

Assumption	Value
Visits per active family per month	1.5
Pharmacy orders per active family per month	2
Average visit base price	₹400
Average basket	₹350
Plus adoption among active families	15%

Per active family per month (non-member):

- Visits: $1.5 \times (\text{₹80 commission} + \text{₹60 fee}) = \text{₹210}$
- Pharmacy: $2 \times (\text{₹35 commission} + \text{₹25 delivery}) = \text{₹120}$
- **About ₹330 of revenue per active family per month**, before rider, support and payment costs.

A Plus member gives up about ₹140 in fees but pays ₹149 and books more often. Watch the **visit frequency of Plus vs. non-Plus** customers as the key metric.

4. Money flow & settlement

Every completed event writes rows to the **settlement ledger** (`SettlementEntry` , append-only):

Event	Rows
Pharmacy order DELIVERED	<code>VENDOR_PAYOUT</code> (pending), <code>COMMISSION</code> , <code>DELIVERY_FEE</code>
Care visit COMPLETED	<code>PROVIDER_PAYOUT</code> (pending), <code>COMMISSION</code> , <code>PLATFORM_FEE</code>
Plus payment verified	<code>MEMBERSHIP_FEE</code>

- Recording is **idempotent**: a unique key on source and type means replays add nothing.
- Recording **never blocks** a delivery or a visit. A failure is logged for replay.
- Partner payouts start `PENDING` . A weekly payout run pays them and marks them `PAID` . `GET /admin/revenue/payouts` groups the pending amounts per partner.
- Cash on delivery: the rider or nurse collects the cash. The partner's net is what's owed after that cash is reconciled. Automating COD reconciliation is on the next-steps list (§7).

5. Where it lives in the code

What	File
All rates and fees (env-overridable)	<code>config/revenue.js</code>
Fee and split maths (pure functions)	<code>services/pricingService.js</code>

What	File
Membership (trial, Razorpay checkout and verify)	<code>services/membershipService.js</code> , <code>models/membership.js</code> , <code>routes/membership.js</code>
Ledger + reports	<code>services/settlementService.js</code> , <code>models/settlementEntry.js</code> , <code>routes/revenue.js</code>
Delivery fee applied at checkout	<code>services/pharmacyService.js#createOrder</code> (stored as <code>order.feeBreakdown</code>)
Visit price with member waiver	<code>services/bookingService.js#createBooking</code> ; preview on <code>GET /care/services</code> (<code>pricingPreview</code>)
Ledger hooks	order → DELIVERED (<code>pharmacyService</code>), visit → COMPLETED (<code>bookingService#completeService</code>)
Customer UI	web <code>/plus</code> ; app Account → Nabz Plus card; booking screens show member pricing
Admin UI	web <code>/admin</code> → Revenue panel (platform admins only)
Tests	<code>tests/unit/pharmacy/pricing-service.test.js</code> , <code>tests/integration/nabz-revenue-tracking.test.js</code>

Changing a number

Set the env var. No code change or deploy is needed beyond a task restart. On AWS, add it to the ECS task environment or Secrets Manager.

Env var	Default
<code>REVENUE_CARE_PROVIDER_COMMISSION_RATE</code>	<code>0.20</code>
<code>REVENUE_CARE_CUSTOMER_FEE_RATE</code>	<code>0.15</code>
<code>REVENUE_CARE_MEMBER_FEE_WAIVED</code>	<code>true</code>
<code>REVENUE_PHARMACY_COMMISSION_RATE</code>	<code>0.10</code>
<code>REVENUE_DELIVERY_FEE_DEFAULT</code>	<code>25</code>
<code>REVENUE_FREE_DELIVERY_ABOVE</code>	<code>499</code> (0 = off)
<code>REVENUE_SURGE_HIGH</code> / <code>REVENUE_SURGE_SEVERE</code>	<code>1.5</code> / <code>2</code>
<code>REVENUE_NIGHT_SURCHARGE</code> , <code>REVENUE_NIGHT_START_HOUR</code> , <code>REVENUE_NIGHT_END_HOUR</code>	<code>20</code> , <code>22</code> , <code>6</code>
<code>REVENUE_MEMBER_FREE_DELIVERY</code>	<code>true</code>
<code>REVENUE_PLUS_MONTHLY_PRICE</code> , <code>REVENUE_PLUS_TRIAL_DAYS</code>	<code>149</code> , <code>7</code>
<code>REVENUE_GST_RATE</code>	<code>0.18</code>

API

Method & path	Who
<code>GET /api/v1/membership/plans</code>	public

Method & path	Who
GET /api/v1/membership · POST /membership/trial · POST /membership/checkout · POST /membership/verify	customer
GET /api/v1/admin/revenue/summary?from&to · /payouts · /policy	platform_admin only (hospital admins are refused)

6. Open decisions / compliance (confirm before launch)

1. **GST:** the code applies 18% to base + platform fee, as the booking flow did before. Home nursing by qualified professionals may be GST-exempt while the platform fee is taxable. **Confirm the treatment with a CA** before charging real customers.
2. **Drugs & Cosmetics rules / e-pharmacy guidelines:** keep the pharmacy as the seller of record. Nabz charges a service commission and doesn't set medicine prices above MRP. Discounts must stay within the rules.
3. **Surge on healthcare:** capped at 2× on the delivery fee only. **Never surge the medicine price or the nurse's visit price.** This is deliberate, both for trust and to stay clear of regulators.
4. **Partner agreements** must state the commission rates, payout cycle (weekly) and COD reconciliation.
5. Plus **cancellation/refund policy** (e.g. refund the unused days within 48 hours) needs writing into the terms.

7. Next steps

- Automated weekly payouts (Razorpay Route / RazorpayX) that mark `PENDING` → `PAID`.
- COD cash reconciliation for riders and nurses.
- Promo codes and first-order discounts (the `discount` field already exists on orders and bookings).
- Plus checkout inside the mobile app (needs `react-native-razorpay`). The web checkout works today.
- B2B contracts and sponsored listings (§1, rows 7–8).

Commission tiers (decided 27 Sep 2026)

Home-care visits (nurse / physio) use monthly volume tiers that reset on the 1st of every month (IST). The job number is the professional's completed visits this month, including this one.

Jobs this month	Nabz commission	Professional keeps
1–10	20%	80%
11–30	15%	85%
31+	12%	88%

- Configure with `REVENUE_CARE_COMMISSION_TIERS` (default `10:0.20,30:0.15,0:0.12`).
- The rate is decided once, when the visit completes, and stamped on the booking (`commissionOverride`), so payouts, the ledger and replays always agree

(`services/commissionService.js`).

- Offers show the professional's real earnings at their current tier.
- **Referral credits:** a job uses 5% or the tier rate, whichever is lower; the credit is only spent when it lowers the rate.
- **Pharmacy:** flat 10% on items (`REVENUE_PHARMACY_COMMISSION_RATE` , keep within 8–12% because medicine margins are thin), with the same referral rule.
- Customer platform fees, delivery fees and Nabz Plus stay alongside commission.

Phase 1.1 rollout

Source: founders' Phase 1.1 note (Human track 1–7, Tech track 1–2). Built on what is live (see [CODE CONTEXT.md](#)).

Today, by 5 pm: ships to staging and both apps

#	Note item	What we build	Where
A	3, 4, 7	Partner types: phlebotomist, PRP technician, and hospital / nursing home join the partner application (waitlist, reviewed by admin), alongside nurse/physio, pharmacy, lab and delivery.	API, website /partners , Partner app "Join"
B	5	Physio gender: partner applications record gender, so ops can track the 2 : 8 M : F hiring target. Approved physios carry it into matching, where the customer's preferred gender already applies.	API, forms, admin panel (M : F count)
C	5 "book from distance"	Book for someone else: a toggle on the booking form. It takes the patient's name, age and gender plus the on-site contact person and phone. The professional sees that contact (inside the usual phone-visibility window) instead of the booker's.	API validation, website booking, customer app
D	Tech 1	Demo data: demo stores stocked with realistic common medicines; demo accounts for a nurse, a physio and a pharmacy, so the Partner and customer app demo end to end.	Staging database seed
E	—	Deploy the API and website, and rebuild both APKs. Run the Partner app on the emulator for joint testing.	Staging

Next: Phase 1.1b (needs a day or more, or a decision first)

#	Note item	Work	Blocked on
1	2 Labs (5 labs, common rate list)	Port the lab module from <code>feature/medrush-core</code> , then add: - An admin-managed common rate list (one price per test across partner labs). - Lab partner dashboard. - Home-collection booking.	Merge with the uncommitted lab work on <code>feature/medrush-core</code> . The rate list itself comes from lab negotiations (Human track).
2	3 Phlebotomist	Dispatch phlebotomists to lab bookings (reuse nurse dispatch), sample handover, and report upload.	Item 1
3	4 PRP technician	A PRP service only as part of a doctor-led visit.	Medical/legal decision: PRP injection is a medical procedure. Confirm who performs it and under which doctor.
4	1 Delivery (freelance)	Rider app flow: accept delivery, pick up from the store, hand over with the delivery code, earnings. Reuses the pharmacy order states.	Rider onboarding and payout terms

#	Note item	Work	Blocked on
5	6 Packages	Package purchase with scheduled sessions, sessions-left counter, reschedule, same professional preferred.	Package prices from the team
6	7 Hospital / nursing	Partner onboarding, referral of discharged patients to post-surgery packages.	Commercial terms
7	5 Book from distance, part 2	Pick the visit address on a map or by search (geocoding) instead of the booker's location.	Choose a maps provider (Google or Ola Maps) plus an API key
8	Tech 2 Domain and hosting	Attach <code>nabz.in</code> (or chosen domain) to App Runner (API <code>api.</code> , web <code>www.</code>), SSL, and update the app base URLs.	Buy the domain and give DNS access

Test script for the joint test (after 5 pm build)

1. **Partner app (emulator), signed in as the demo nurse:** go online. The app rings when a request arrives.
2. **Customer app (phone):** book a physio for "someone else" with an on-site contact. Prefer a female professional.
3. **Partner app:** accept the offer and check that the on-site contact shows. Move the visit to on the way. Start it with the customer's visit code. Complete it with the cash collected.
4. **Customer app:** order medicines from a demo store and pay by cash on delivery.
5. **Partner app, signed in as the demo pharmacy:** accept, pack, then mark delivered with the customer's delivery code.
6. **Website:** apply as a phlebotomist and as a physio (with gender). In the admin panel, check the queue and the M : F count.

Website and app imagery

For Claude: REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Transform the Nabz web landing page and mobile welcome screen from text-only mockups into high-trust, human-centric experiences featuring authentic patient and caregiver photography, verified customer testimonials, and an interactive onboarding care carousel.

Architecture:

1. Establish a high-fidelity image asset pipeline with responsive WebP and PNG assets for both web (`frontends/web/public/images/`) and mobile (`frontends/mobile/assets/images/`).
2. Upgrade `frontends/web/app/page.tsx` with a human-centered Hero featuring real patient care, a "Stories of Care" testimonial showcase, and a "Care in Action" visual gallery.
3. Upgrade `frontends/mobile/app/welcome.tsx` with an interactive, auto-advancing Care Story Carousel showcasing real patient care moments with contextual safety badges.

Tech Stack: Next.js 14 App Router, React 18, React Native (Expo Router), TypeScript, Lucide Icons, CSS Variables / Tailwind.

Task 1: Generate & Organize Patient & Caregiver Image Assets

Files:

- Create: `frontends/web/public/images/patients/elderly-care.webp` (and `.png`)
- Create: `frontends/web/public/images/patients/physio-recovery.webp` (and `.png`)
- Create: `frontends/web/public/images/patients/mother-child-care.webp` (and `.png`)
- Create: `frontends/web/public/images/patients/vitals-home.webp` (and `.png`)
- Create: `frontends/web/public/images/testimonials/avatar-ramesh.webp` (and `.png`)
- Create: `frontends/web/public/images/testimonials/avatar-neha.webp` (and `.png`)
- Create: `frontends/web/public/images/testimonials/avatar-vikram.webp` (and `.png`)
- Create: `frontends/web/public/images/testimonials/avatar-sunita.webp` (and `.png`)
- Create: `frontends/web/public/images/staff/nurse-asha.webp` (and `.png`)
- Create: `frontends/mobile/assets/images/welcome/slide-elderly.png`
- Create: `frontends/mobile/assets/images/welcome/slide-physio.png`
- Create: `frontends/mobile/assets/images/welcome/slide-safety.png`
- Create: `scripts/generate-brand-assets.js`

Step 1: Write asset generator script Generate authentic, warm Indian healthcare photographs using the image generation pipeline for four scenarios:

1. Senior smiling warmly while verified nurse checks blood pressure at home.
2. Physiotherapist guiding knee/arm mobility exercises for recovering patient.
3. Relieved mother with calm child during gentle home health checkup.
4. Close-up portrait of verified nurse Asha Verma with uniform and ID card.

Step 2: Run asset generation script Run: `node scripts/generate-brand-assets.js` Expected: High-resolution WebP and PNG assets saved to `frontends/web/public/images/` and `frontends/mobile/assets/images/`.

Step 3: Verify image dimensions and aspect ratios Run: `node -e "const fs = require('fs'); ['frontends/web/public/images/patients', 'frontends/mobile/assets/images/welcome'].forEach(d => console.log(d, fs.readdirSync(d)));" Expected: All required files exist and are non-empty (>50KB each).`

Step 4: Commit assets

```
git add frontends/web/public/images frontends/mobile/assets/images
git commit -m "assets: add high-fidelity patient and caregiver imagery for web and mobile"
```

Task 2: Build Web Social Proof & Patient Stories Components

Files:

- Create: `frontends/web/app/_components/PatientStories.tsx`
- Create: `frontends/web/app/_components/TrustBadgeBar.tsx`
- Create: `frontends/web/app/_components/CareGallery.tsx`
- Test: `tests/unit/web/patient-stories.test.tsx`

Step 1: Write unit test for PatientStories component

```
import { render, screen } from '@testing-library/react';
import { PatientStories } from '@app/_components/PatientStories';

describe('PatientStories', () => {
  it('renders verified customer testimonials with ratings and location', () => {
    render(<PatientStories />);
    expect(screen.getByText(/Stories of Care/i)).toBeInTheDocument();
    expect(screen.getByText(/Ramesh Kumar/i)).toBeInTheDocument();
    expect(screen.getByText(/C-Scheme, Jaipur/i)).toBeInTheDocument();
    expect(screen.getAllByText(/Verified Visit/i).length).toBeGreaterThan(0);
  });
});
```

Step 2: Run test to verify it fails Run: `npm --prefix frontends/web test -- tests/unit/web/patient-stories.test.tsx` Expected: FAIL (module not found).

Step 3: Implement `PatientStories.tsx`, `TrustBadgeBar.tsx`, and `CareGallery.tsx`

- `TrustBadgeBar.tsx` :
 - 4 customer micro-avatars stacked with overlap.
 - "4.9/5 from 12,500+ visits in Jaipur".
 - Partner hospital / pharmacy trust seals.
- `PatientStories.tsx` :
 - 3 rich testimonial cards with customer photo, verified badge, quote, service badge ("Post-Op Nursing", "Elderly Care", "Physiotherapy"), and star rating.
- `CareGallery.tsx` :

- 3-column photo grid highlighting "Care at Home", "Sterile Pharmacy Supplies", and "Doorstep Vitals Monitoring" with subtle hover zoom and warm typography.

Step 4: Run test to verify it passes Run: `npm --prefix frontends/web test -- tests/unit/web/patient-stories.test.tsx` Expected: PASS.

Step 5: Commit

```
git add frontends/web/app/_components/ tests/unit/web/
git commit -m "feat(web): add PatientStories, TrustBadgeBar and CareGallery components"
```

Task 3: Upgrade Web Landing Page (`frontends/web/app/page.tsx`)

Files:

- Modify: `frontends/web/app/page.tsx`
- Modify: `frontends/web/app/globals.css`
- Test: `tests/unit/web/landing-page.test.tsx`

Step 1: Write unit test for upgraded landing page

```
import { render, screen } from '@testing-library/react';
import Landing from '@app/page';

describe('Landing Page UX & Imagery', () => {
  it('renders social proof avatar bar and patient stories section', () => {
    render(<Landing />);
    expect(screen.getByText(/Trusted by 12,000\+ families/i)).toBeInTheDocument();
    expect(screen.getByText(/Real stories from real visits/i)).toBeInTheDocument();
    expect(screen.getByAltText(/Asha Verma/i)).toBeInTheDocument();
  });
});
```

Step 2: Run test to verify it fails Run: `npm --prefix frontends/web test -- tests/unit/web/landing-page.test.tsx` Expected: FAIL.

Step 3: Update `page.tsx` and `globals.css`

- In Hero:
 - Add `<TrustBadgeBar />` below the CTA buttons.
 - Replace abstract mockup avatar with realistic image of nurse Asha Verma (`/images/staff/nurse-asha.webp`).
 - Add photo background overlay to hero preview card showing real patient checkup moment.
- In Body:
 - Insert `<CareGallery />` right after the Service Grid.
 - Insert `<PatientStories />` right after "How It Works".
- In `globals.css` :
 - Add modern card styles, smooth hover transitions, glassmorphic badges, and avatar group CSS.

Step 4: Run test to verify it passes Run: `npm --prefix frontends/web test -- tests/unit/web/landing-page.test.tsx` Expected: PASS.

Step 5: Commit

```
git add frontends/web/app/page.tsx frontends/web/app/globals.css
git commit -m "feat(web): integrate patient photos, social proof bar and stories into landing page"
```

Task 4: Build Mobile App Care Carousel (`frontends/mobile/app/welcome.tsx`)

Files:

- Create: `frontends/mobile/lib/WelcomeCareCarousel.tsx`
- Modify: `frontends/mobile/app/welcome.tsx`
- Modify: `frontends/mobile/lib/i18n.ts` (add Hindi & English slide strings)
- Test: `tests/unit/mobile/welcome-carousel.test.tsx`

Step 1: Write test for WelcomeCareCarousel

```
import { render } from '@testing-library/react-native';
import { WelcomeCareCarousel } from '@lib/WelcomeCareCarousel';

describe('WelcomeCareCarousel', () => {
  it('renders 3 care slides with patient imagery and badges', () => {
    const { getByText } = render(<WelcomeCareCarousel />);
    expect(getByText(/Elderly Care at Home/i)).toBeTruthy();
    expect(getByText(/Police verified/i)).toBeTruthy();
  });
});
```

Step 2: Run test to verify it fails Run: `npm --prefix frontends/mobile test -- tests/unit/mobile/welcome-carousel.test.tsx` Expected: FAIL.

Step 3: Implement `WelcomeCareCarousel.tsx` and integrate into `welcome.tsx`

- Features:
 - 3 rich slides with patient imagery:
 1. *Elderly Care*: Photo of grandmother with nurse + "Gentle nursing & vitals monitoring at home".
 2. *Rehabilitation*: Photo of physiotherapy session + "Recover comfortably without hospital visits".
 3. *Verified Safety*: Photo of nurse with ID badge + "100% Police & nursing council verified".
 - Animated pagination indicator pills with smooth width transition.
 - Auto-advance every 4.5s with user drag/touch detection to pause.
 - Floating trust pills with backdrop blur (`4.9 ★ 15,000+ visits` , `Arrives in 15 min`).
- In `welcome.tsx` :
 - Replace the static circle halo in `stage` with `<WelcomeCareCarousel />` .

- Maintain the phone/Google/Email login buttons and exploration CTA below.

Step 4: Run test to verify it passes Run: `npm --prefix frontends/mobile test -- tests/unit/mobile/welcome-carousel.test.tsx` Expected: PASS.

Step 5: Commit

```
git add frontends/mobile/lib/WelcomeCareCarousel.tsx frontends/mobile/app/welcome.tsx
frontends/mobile/lib/i18n.ts
git commit -m "feat(mobile): add animated patient care photo carousel to welcome screen"
```

Task 5: End-to-End Build & Visual Regression Verification

Files:

- Test: `tests/e2e/landing-page-visual.spec.ts`

Step 1: Type check Web and Mobile applications Run:

```
npm --prefix frontends/web run lint
npx --prefix frontends/web tsc --noEmit
npx --prefix frontends/mobile tsc --noEmit
```

Expected: 0 errors.

Step 2: Run Next.js production build Run: `npm --prefix frontends/web run build` Expected: Successful build with static page generation.

Step 3: Visual inspection via browser subagent Capture full-page screenshot of:

- Desktop Web Landing Page (`http://localhost:3000`)
- Mobile Web Viewport (`390x844`)
- Confirm patient images load with high resolution, zero layout shift (CLS < 0.05), and correct contrast.

Step 4: Commit and finalize

```
git add .
git commit -m "chore: verify web and mobile patient imagery enhancements"
```

PART 3

Engineering

Getting started

The goal here is **easy to run**: one backend, one database, two frontends. Everything else in this repo (Docker, k8s, Grafana, RabbitMQ, replica sets) is **optional production tooling you can ignore** for local development.

1. The simple architecture



- **One process to run the server:** `npm run dev` (nodemon on port 5000).
- **One database:** local MongoDB (`nocturnal_dev`). No table creation step — collections appear on first insert; you only `seed` demo data.
- **Redis is off** unless you set `REDIS_ENABLED=true` ; the app degrades gracefully without it. Same for Razorpay, Firebase, S3 — all optional.
- Frontends never touch the database — they only call the API.

You need exactly **three things installed**: Node $\geq$ 22, MongoDB (local or Atlas), and npm. That's it for the full stack.

2. First-time setup (5 steps)

```
# 1. From the repo root – install backend deps
npm install

# 2. Create your .env (generates strong JWT_SECRET + ENCRYPTION_KEY for you)
npm run setup:env
# ...or copy .env.example to .env and set the two REQUIRED values:
#   JWT_SECRET      = 64+ random chars
#   ENCRYPTION_KEY  = 64 hex chars    -> openssl rand -hex 32
#   MONGODB_URI defaults to mongodb://localhost:27017/nocturnal_dev in dev.

# 3. Make sure MongoDB is running locally (or point MONGODB_URI at Atlas)

# 4. Create indexes + seed demo pharmacy data (vendors, medicines, inventory)
npm run db:indexes
npm run db:seed:pharmacy

# 5. Start the backend
npm run dev                # API on http://localhost:5000 (health: /api/v1/health)
```

The seed prints the demo logins. Set `SEED_DEMO_PASSWORD` in your local `.env` first; all demo accounts use it.

3. Run the frontends

Open two more terminals.

Web (Next.js):

```
cd frontends/web
cp .env.local.example .env.local    # NEXT_PUBLIC_API_BASE_URL=http://localhost:5000
npm install
npm run dev                         # http://localhost:3000
```

Mobile (Expo):

```
cd frontends/mobile
npm install
npx expo start                    # press w (web preview), a (Android), i (iOS)
```

On a real phone, set `EXPO_PUBLIC_API_BASE_URL=http://<your-LAN-IP>:5000`.

First time only: `cd frontends/shared && npm install` so the shared package's TypeScript is available to both apps (they import it directly from source).

4. Try the flow

1. Web → **Pharmacy** (/pharmacy): it geolocates (or falls back to the demo area), lists seeded vendors, shows a catalog, and builds a cart.
2. Web → **Vendor** (/vendor): paste a `pharmacy_vendor` token to see the order queue and move orders `ACCEPTED → PREPARING → READY → ...`.
3. API directly:

```
curl "http://localhost:5000/api/v1/pharmacy/vendors/nearby?lat=12.9352&lng=77.627&radiusKm=10"
```

Online payment (Razorpay, optional)

Without Razorpay keys, checkout offers **Cash on delivery** only. To test prepaid:

1. Create a Razorpay account → Dashboard in **Test mode** → *API Keys* → generate.
2. Add to the backend `.env` and restart `npm run dev`:

```
RAZORPAY_KEY_ID=rzp_test_XXXXXXX
RAZORPAY_KEY_SECRET=XXXXXXXXXXXXXXXXX
# optional: PHARMACY_PAYMENT_TTL_MINUTES=15 (how long unpaid orders hold stock)
```

3. Checkout now shows **Pay online**. Use Razorpay's test card / UPI (`success@razorpay`) to pay.

How it behaves:

- A prepaid order reserves stock and is **hidden from the pharmacy until paid**.
- Payment is verified server-side (signature + amount/currency/order re-fetched from Razorpay) before the order becomes `PAID`.
- If the patient closes the payment window, the order page shows **Pay now** until the hold expires; then a background sweeper cancels it and restocks (it asks Razorpay first, so a paid-but-unverified order is kept, not cancelled).
- Cancelling or rejecting a paid order refunds it automatically. If the refund call fails, the order stays `REFUND_PENDING` with `razorpay.refundError` set. Reconcile those by hand for now.

5. Everyday commands

Task	Command (repo root)
Backend dev server	<code>npm run dev</code>
Backend + web frontend together	<code>npm run dev:all</code>
Seed pharmacy demo data	<code>npm run db:seed:pharmacy</code>
Create DB indexes	<code>npm run db:indexes</code>
Lint	<code>npm run lint</code>
Fast tests	<code>npm test</code>

Task	Command (repo root)
Lint + fast tests (pre-PR)	<code>npm run verify:local</code>

6. What to ignore for now (keep it simple)

These exist for production and are **not needed to develop**: `docker-compose*.yaml`, `k8s/`, `terraform/`, `grafana/`, `prometheus/`, `loki/`, `logstash/`, `filebeat/`, `nginx/`, RabbitMQ, MongoDB replica sets, PM2. Ignore them until you deploy. Local dev is just **Node + MongoDB + two** `npm run dev s`.

See also: [MEDRUSH PHARMACY MARKETPLACE PLAN.md](#) (roadmap) and [MEDRUSH DATA MODEL.md](#) (collections/tables & how to add one).

Data model

How MedRush stores its data, why it's shaped this way, and how to extend it. It's the single reference for the pharmacy marketplace and the home-health data layer.

MongoDB, not SQL. "Collections" are our "tables". A collection $\approx$ table, document $\approx$ row, field $\approx$ column. Collections appear on first insert, and their shape is enforced by the Mongoose schema in `models/`. The only thing we "migrate" is **indexes** (`npm run db:indexes`, which also runs as a one-off ECS task on every deploy).

0. Design principles (learned from quick-commerce leaders)

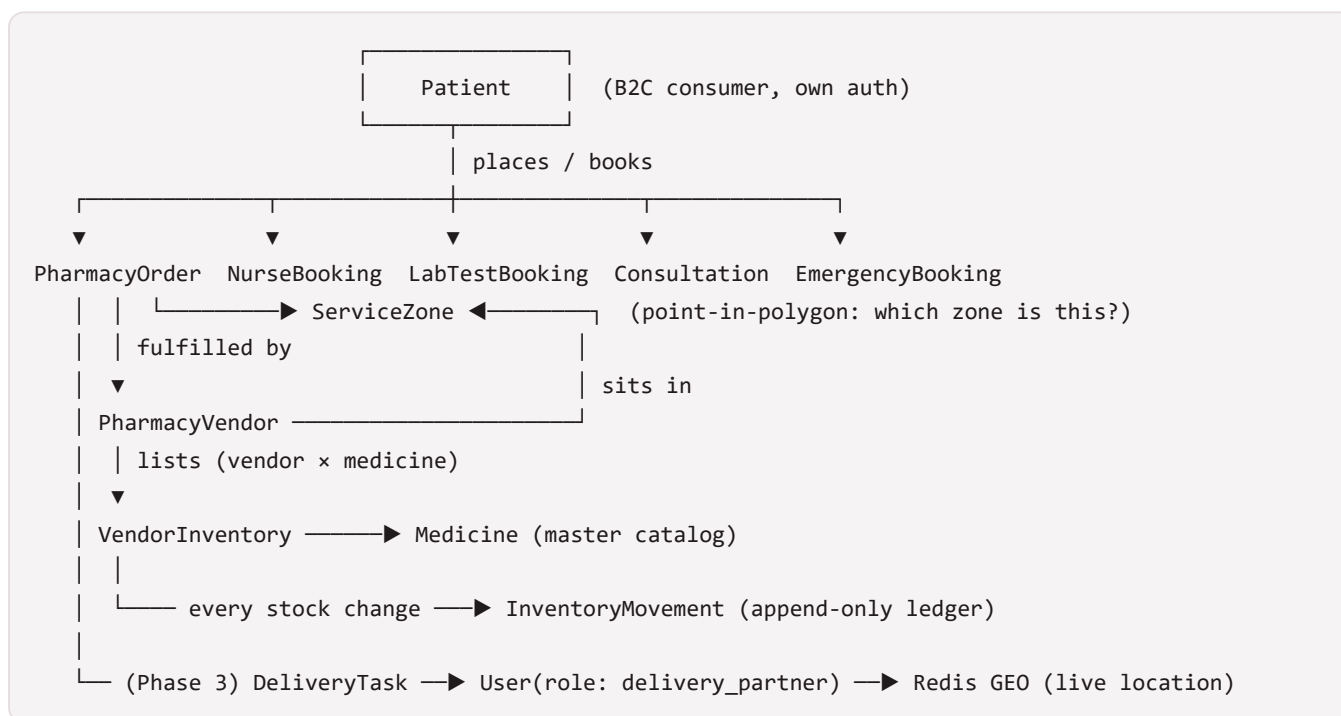
MedRush is a Swiggy-style **aggregator of local pharmacies**, not a Zepto/Blinkit dark-store chain. Most of their data lessons still apply:

#	Principle	Who does it	How MedRush applies it	Status
1	Serviceability is a zone decision, not just a radius. Resolve the customer to an ops-drawn polygon (point-in-polygon), with a radial fallback	Swiggy serviceability platform	<code>ServiceZone</code> polygons + <code>serviceabilityService</code>	✓ built
2	Shrink serviceability under stress (rain, rider shortage) instead of breaking promises	Swiggy ("reduce last-mile distance")	<code>ServiceZone.stress</code> → radius multiplier; <code>isActive:false</code> pauses a zone	✓ built
3	Hierarchical cell ids for hyperlocal keys (cache, heatmaps, analytics)	Swiggy geohash, Uber H3	<code>geohash</code> on vendors, <code>deliveryGeohash</code> on orders	✓ built
4	ETA is a sum of legs: assignment + first mile vs prep, then last mile. Store the promise <i>and</i> the actual per-leg timestamps so each leg can be modelled	Swiggy ETA models (O2A, FM, WT, LM)	<code>order.eta</code> (promise per leg) + <code>order.milestones</code> (actuals)	✓ built
5	"Listed" $\neq$ "in stock"	Blinkit	<code>VendorInventory.isAvailable</code> = vendor's intent; stock is <code>stockQty</code> ; orderable = both	✓ built (fixed a bug)
6	Reserve stock atomically at checkout; release on cancel/expiry	Zepto/Blinkit OMS	atomic <code>\$inc</code> with a <code>stockQty >= qty</code> guard; unpaid orders expire and restock	✓ built
7	Every stock change is ledgered (explainable inventory, reconciliation)	standard OMS/WMS practice	<code>InventoryMovement</code> (append-only)	✓ built
8	Strict order state machine + event timeline	Zomato	<code>VENDOR_STATUS_TRANSITIONS</code> , <code>timeline[]</code>	✓ built

#	Principle	Who does it	How MedRush applies it	Status
9	Snapshot everything the customer agreed to (price, fee, address, ETA)	all	item price snapshots, amounts, deliveryAddress, eta	✅ built
10	Delivery is its own entity , separate from the order (assignment, batching, re-assignment)	Swiggy, Zomato	DeliveryTask	➡ Phase 3 (\$5)
11	Live rider location lives in Redis GEO, not the database (pings every few seconds)	Zomato/Swiggy tracking pipelines	Redis GEOADD + sampled trail	➡ Phase 3 (\$5)
12	Money moves through a ledger (commission, payouts, refunds)	Zomato commission ledger	SettlementEntry	➡ Phase 5 (\$5)

What we deliberately **don't** copy: dark-store inventory ownership (our pharmacies own their stock) and per-SKU warehouse bins and pick paths (that's the pharmacy's own process).

1. The big picture (ERD)



Two identity collections, kept separate on purpose:

- **Patient**: end users, with their own login (`middleware/patientAuth.js`).
- **User**: every staff/partner role (`pharmacy_vendor`, `delivery_partner`, `medical_staff`, `phlebotomist`, doctors/nurses, admins), with their own login (`middleware/auth.js`).

2. Collections (data dictionary)

Legend: **FK** = ObjectId reference. `[Point]` = GeoJSON `{ type: 'Point', coordinates:[lng,lat] }` (2dsphere).

⚠️ Never give a GeoJSON `type` a default: a coordinate-less `{type: 'Point'}` breaks 2dsphere inserts.

ServiceZone • `models/serviceZone.js` (NEW)

An ops-drawn delivery polygon ("cluster").

Field	Type	Notes
code	String, unique	KORAMANGALA-1
name, city	String	
boundary	GeoJSON Polygon/MultiPolygon	2dsphere; MongoDB rejects invalid rings
isActive	Boolean	false = deliveries into the zone paused
maxLastMileKm	Number	hard cap on store→customer distance here
stress	{level, radiusMultiplier, reason, until, updatedBy}	NORMAL / HIGH (0.7)/ SEVERE (0.4); until auto-expires

Effective radius of a store for a customer = `min(store.serviceRadiusKm, zone.maxLastMileKm) × stressMultiplier`. With no zone drawn at that point, it's just `store.serviceRadiusKm` (radial fallback). Admin API: GET/POST `/pharmacy/admin/zones`, PATCH `/pharmacy/admin/zones/:id` (stores are re-linked automatically when a boundary changes).

PharmacyVendor • `models/pharmacyVendor.js`

The local pharmacy (the marketplace seller).

Field	Type	Notes
name, slug, owner (FK User)		owner = <code>pharmacy_vendor</code> login
drugLicenseNumber, gstin, documents		KYC
address, location [Point]		2dsphere
serviceRadiusKm	Number	store's own max reach (now enforced)
zone	FK ServiceZone	set on create / when zones change
geohash	String	derived from <code>location</code> (precision 7 ≈ 150 m), never hand-set
operatingHours, isOpen, avgPreparationMinutes		prep time feeds ETA
deliveryFee, minOrderValue	Number	
status	Enum	PENDING / APPROVED / SUSPENDED / REJECTED
payout	Object	settlement details (<code>accountNumber</code> is <code>select:false</code>)

Medicine • `models/medicine.js`

Global master catalog: one row per sellable product, de-duplicated across vendors. `name`, `genericName` / `composition`, `form`, `strength`, `packSize`, `scheduleType`

(OTC / PRESCRIPTION / SCHEDULE_H / H1 / X), category , requiresPrescription , referenceMrp . Has a text index for search.

VendorInventory • models/vendorInventory.js

A vendor's listing of a Medicine (join: vendor × medicine, unique).

Field	Notes
mrp, sellingPrice, discountPercentage	discount recomputed on save
stockQty	on-hand units. Changes only via atomic \$inc (orders) or explicit vendor counts
isAvailable	LISTED : the vendor's intent to sell. Stock level never changes it
lowStockThreshold, batchNumber, expiryDate	

Orderable = isAvailable && stockQty > 0 . Previously, hitting 0 stock delisted the item and every cancellation re-listed it, overriding a vendor who had switched it off.

InventoryMovement • models/inventoryMovement.js (NEW, append-only)

Field	Notes
vendor, medicine	FKs
type	ORDER_RESERVED (-) / ORDER_RELEASED (+) / ADJUSTMENT (±)
delta, balanceAfter	signed units, stock after the move
order	FK PharmacyOrder (for order-driven moves)
actor {kind, id}, reason	PATIENT / VENDOR / ADMIN / SYSTEM

VendorInventory.stockQty stays the source of truth. The ledger is the audit trail: it explains every unit and supports reconciliation. Linked to order → patient → prescription, it's also the raw material for the **Schedule H1 register**. Never update or delete rows; correct mistakes with a new ADJUSTMENT . Retention: keep at least as long as the drug-register rules require (plan for ≥ 3 years; confirm with compliance).

PharmacyOrder • models/pharmacyOrder.js

A patient's order to one vendor. Everything the customer agreed to is **snapshotted**.

Field	Notes
orderNumber, patient, vendor	
items[]	price/name/pack snapshots per line
requiresPrescription, prescription {key, verified...}	key = private S3 object under prescriptions/<patientId>/
status, timeline[]	state machine (§3) + full event log
deliveryAddress, deliveryLocation [Point]	

Field	Notes
deliveryGeohash, zone, distanceKm	serviceability snapshot at checkout
eta {promisedAt, prepMinutes, assignmentMinutes, lastMileMinutes}	the promise, per leg
milestones {placedAt, paidAt, acceptedAt, readyAt, riderAssignedAt, pickedUpAt, deliveredAt, cancelledAt}	first time each stage was reached (actuals per leg)
rider, deliveryStatus	superseded by <code>DeliveryTask</code> in Phase 3
amounts {itemsSubtotal, deliveryFee, tax, discount, total}	INR snapshot
paymentMode, paymentStatus, paymentExpiresAt, razorpay{...}	prepaid flow (see payments section in <code>context.md</code>)

Leg metrics fall straight out of `milestones` : `prep` = `readyAt` - `acceptedAt` , `last mile` = `deliveredAt` - `pickedUpAt` , `promise kept` = `deliveredAt` ≤ `eta.promisedAt` .

User / Patient (extended)

User: `pharmacyVendor` FK, `currentLocation` [Point] (last *known*, throttled, not every GPS ping), `isOnline` , `isAvailable` , `vehicle` , `servicesOffered[]` . Patient: `emergencyContacts[]` , `preferredLanguage` .

Reused as-is

`Review` , `Notification` , `ServiceCatalog` , `NurseBooking` . (The legacy `Payment` model is for staff payouts and is **not** used by pharmacy orders.)

3. State machines

PharmacyOrder, with vendor transitions enforced by `VENDOR_STATUS_TRANSITIONS` :

```

PLACED → ACCEPTED → PREPARING → READY_FOR_PICKUP → OUT_FOR_DELIVERY → DELIVERED
|           |           |
└─ REJECTED ─┬─ CANCELLED ◀ (both release stock → ORDER_RELEASED)

```

- Prepaid orders are invisible to the vendor until `paymentStatus` = `PAID` . Unpaid ones expire (`SYSTEM` cancel + restock).
- Patients may cancel only while `PLACED` / `ACCEPTED` . Cancelling a paid order refunds it.
- Each transition stamps its milestone once. `timeline[]` keeps every event.

Sample (`SAMPLE_STATUSES`): `SCHEDULED` → `COLLECTED` → `IN_TRANSIT` → `AT_LAB` → `PROCESSING` → `REPORT_READY` → `DELIVERED` . **Consultation**: `REQUESTED` → `ACCEPTED` → `IN_PROGRESS` → `COMPLETED` (+ `CANCELLED` / `NO_SHOW`).

4. Where each kind of data lives

Data	Store	Why
Orders, catalog, inventory, users, zones, ledger	MongoDB Atlas (system of record)	documents + geo + transactions
Prescriptions, reports, photos	S3 (private, encrypted)	files never go in the DB
Rate limits, caches (e.g. serviceability by geohash)	Redis	disposable, rebuildable
Live rider positions (Phase 3)	Redis GEO	a ping every ~3 s × riders would crush Mongo; only the latest value matters
Analytics (later)	S3 + Athena/warehouse via change streams	keep heavy queries off the OLTP DB

Containers store nothing: see `docs/MEDRUSH_AWS_DEPLOYMENT.md` §2.

5. Target model for the next phases (designed, not built)

DeliveryTask (Phase 3)

The delivery job, separate from the order (Swiggy/Zomato), so one order can be re-assigned and, later, two orders can be **batched** for one rider.

```
order (FK), batchId?, rider (FK User), status (DELIVERY_STATUSES),
pickup {location, vendor}, drop {location, addressSnapshot},
assignment {attempts:[{rider, offeredAt, response, respondedAt}], assignedAt},
distanceKm, eta {firstMileMinutes, lastMileMinutes},
proofOfDelivery {otpHash, deliveredAt, photoKey?}, codCollected?
indexes: {rider, status}, {order}, {status, createdAt}
```

Assign the rider *during* prep, not at order time, so they don't idle at the counter (Swiggy's O2A/FM vs prep insight). Only offer riders in the store's geohash neighbourhood (`GEOSEARCH` on Redis).

Rider location (Phase 3)

- Hot path: app → API (rate-limited ≈ 1 ping / 3 s) → `GEOADD riders:<city> lng lat riderId`
 - `HSET rider:<id> {lat,lng,speed,battery,at}` with a short TTL (offline if it expires).
- Tracking: Socket.IO room per order; clients receive pushes instead of polling.
- Mongo gets **sampled** points only (`User.currentLocation` updated every ~30 s, plus a per-task trail capped at N points) for disputes and ETA training.

SettlementEntry (Phase 5: vendor payouts)

Double-entry style ledger, one row per money movement per party: `party {type: VENDOR|RIDER|PLATFORM, id}, order, kind (ORDER_REVENUE, COMMISSION, DELIVERY_FEE, REFUND_REVERSAL, PAYOUT), amount (paise,`

signed), payoutBatch?, createdAt . Balances are sums, and payouts are just entries, so it reconciles with Razorpay settlements.

Also planned

- **PaymentTransaction**: one row per gateway attempt, refund and webhook event (today the refs sit on the order). Needed for partial refunds and webhook idempotency.
- **Server-side Cart**: cross-device carts for the mobile app. Price re-validated at checkout.
- **Promotion / Coupon** with usage caps and per-user redemption rows.
- **Order events outbox** → notifications/analytics (the repo already uses outbox workers).

Scaling notes

- **Money in paise (integers)** for new money fields (`SettlementEntry`). Existing rupee fields are converted with `toPaise` at the gateway boundary.
- **Hot counters**: inventory stays per-(vendor, medicine) documents with atomic `$inc` . No global counters.
- **Sharding (when needed)**: orders by `{vendor: 1, createdAt: 1}` or zone, and inventory by `{vendor: 1}` , so each store's traffic stays on one shard.
- **Archival**: delivered orders > 18 months → Atlas Online Archive (queryable, cheap).

6. Indexes (the only thing we "migrate")

Declared in the schemas and built by `scripts/add-indexes.js` (explicit `createIndexes()` for every MedRush model, which fails the deploy if one can't be built). Key ones:

- `PharmacyVendor` : `location 2dsphere` , `{status,isActive,isOpen}` , `{zone,status}` , `geohash`
- `ServiceZone` : `boundary 2dsphere` , `code unique`
- `VendorInventory` : `unique {vendor,medicine}` , `{vendor,isAvailable}`
- `InventoryMovement` : `{vendor,medicine,createdAt}` , `{order}` , `{vendor,type,createdAt}`
- `PharmacyOrder` : `{patient,createdAt}` , `{vendor,status,createdAt}` , `deliveryLocation 2dsphere` , `{paymentMode,paymentStatus,paymentExpiresAt}` (sweeper), `{zone,status,createdAt}` , `{deliveryGeohash,createdAt}`
- `Medicine` : text index (search). `User` : `currentLocation 2dsphere` , `{role,isOnline,isAvailable}`

7. How to add a model (step-by-step)

1. **Vocabulary first**. Enums in `constants/enums.js` (roles in both `constants/roles.js` and `STAFF_ROLES`). Then re-copy the file to `apps/patient-health/constants/` (mirror test).
2. **Schema** in `models/<name>.js` , with no-arg hooks only and the export guard `mongoose.models.X || mongoose.model('X', S)` .
3. **Service** holds the logic and throws `utils/errors.js` types. The **controller** is thin (`responseHelper`). **Routes** use express-validator plus `protect / authorize / protectPatient` , wired in `routes/v1/index.js` .
4. **Types + client** in `frontends/shared/src/{types,api}.ts` .

5. **Indexes:** add the model to the MedRush loop in `scripts/add-indexes.js`.
 6. **Tests:** DB-free unit tests for pure logic, plus `tests/integration/*` against a real MongoDB for anything geo/atomic (see `medrush-data-model.test.js`).
-

Sources

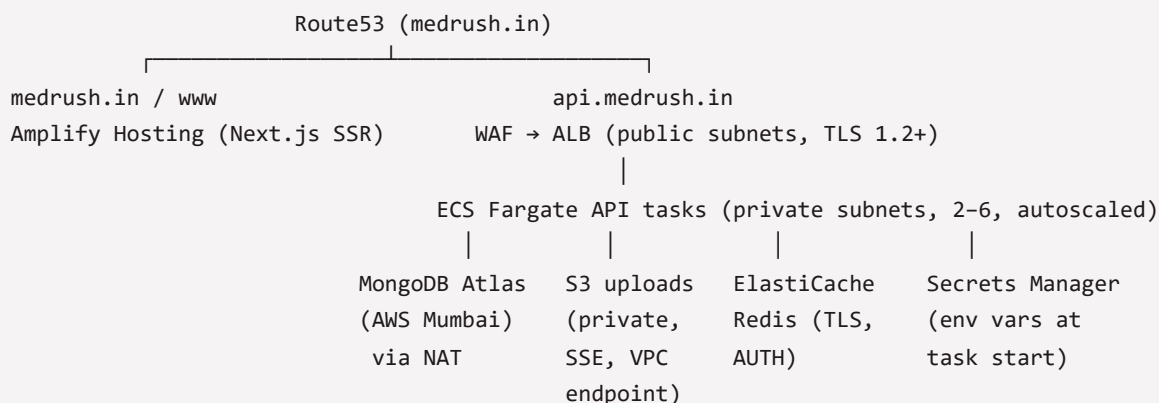
- Swiggy Bytes: [Designing the Serviceability Platform at Swiggy, Part 1](#), [Part 2](#), [What serviceability means at Swiggy](#)
- Swiggy Bytes: [How ML powers "When is my order coming?", Part II](#), [Predicting food delivery time at cart](#), [Delivery partners app architecture](#)
- Uber Engineering: [H3: Uber's Hexagonal Hierarchical Spatial Index](#)
- Real-time rider tracking pattern (Redis GEO + pub/sub + WebSockets): [How Zomato/Swiggy update rider location in real time](#), [Zomato-style data model](#)
- Blinkit listed vs in-stock and inventory planning: [EcomSarathi: Blinkit inventory planning](#), [Inc42: Blinkit inventory model](#)
- Zepto OMS/WMS (serviceability → reserve → pick → dispatch): [How Zepto delivery works](#)

Secondary sources (non-engineering blogs) informed patterns only. Every design choice above is verified in our own tests, not taken on trust.

AWS deployment

Region: **ap-south-1 (Mumbai)**, so health data (prescriptions) stays in India. Infrastructure is code in [terraform/](#). CI/CD is in [.github/workflows/deploy.yml](#).

1. Architecture



Web (`medrush.in`) and API (`api.medrush.in`) share one registrable domain, so the API's `SameSite=strict` `httpOnly` auth cookies work from the web app with no CORS cookie workarounds. `ALLOWED_ORIGINS` is set by Terraform.

2. Where is the data stored?

Containers keep no data. An API task can be killed, replaced or scaled at any moment. Anything written to its disk is lost and invisible to the other tasks. So every piece of state lives in a managed service:

Data	Where (AWS)	Where (local Docker)	Durability
Documents: users, patients, vendors, catalog, inventory, orders	MongoDB Atlas cluster in AWS <code>ap-south-1</code>	<code>mongo:7</code> container, named volume <code>mongodb_data</code>	Atlas replica set + continuous backups
Files: prescriptions, reports, photos	S3 <code>medrush-<env>-uploads-<acct></code> (private, encrypted, versioned, TLS-only)	MinIO (S3-compatible) container, named volume <code>minio_data</code>	11 nines; old versions kept 90 days
Cache, rate-limit counters	ElastiCache Redis (TLS + AUTH)	<code>redis:7</code> container, named volume <code>redis_data</code>	Rebuildable. The app runs without it
Secrets (DB URI, JWT, keys)	Secrets Manager <code>medrush/<env>/app</code>	<code>.env</code> file (git-ignored)	Versioned, KMS-encrypted
Logs	CloudWatch Logs <code>/ecs/medrush-<env>/api</code>	<code>docker logs</code> / <code>./logs</code>	30-day retention

Cross-platform local dev: Docker Compose uses **named volumes**, which Docker manages inside its own VM/storage on Windows, macOS and Linux alike. There are no host paths, drive letters or file-permission differences. Data survives `docker compose down` and is removed only by `docker compose down -v`.

```
docker compose up -d mongodb redis minio minio-init
# then in .env: STORAGE_PROVIDER=s3, S3_UPLOADS_BUCKET=medrush-local-uploads,
#               S3_ENDPOINT=http://localhost:9000, AWS_ACCESS_KEY_ID/SECRET=minioadmin
npm run dev
```

MinIO console: <http://localhost:9001>. With `STORAGE_PROVIDER` empty the API falls back to `./uploads` on disk. That's fine for quick hacking, but it isn't the AWS code path.

3. First-time setup (once per AWS account)

Prereqs: AWS account + admin CLI profile, Terraform ≥ 1.5 , a domain, a GitHub repo, a MongoDB Atlas account.

3.1 Terraform state

```
cd terraform/bootstrap
terraform init && terraform apply      # S3 state bucket + DynamoDB lock in ap-south-1
```

3.2 MongoDB Atlas

1. Create a project → cluster on **AWS / Mumbai (ap-south-1)**. Use M10+ for production (dedicated, backups, VPC peering/PrivateLink available); M0/M2 is fine for staging.
2. Database user `medrush-app` with `readWrite` on the `medrush` database only.
3. Enable **Continuous Cloud Backup** (production).
4. Network access: add the NAT IPs from `terraform output nat_public_ips` after step 3.3. Upgrade path: Atlas **PrivateLink** to keep DB traffic off the internet entirely.

Why Atlas, not DocumentDB: the app uses `$near` + `2dsphere` (nearby pharmacies), `$text` search and MongoDB 7 semantics. Atlas runs real MongoDB inside AWS, so nothing needs rewriting.

3.3 Infrastructure

```
cd terraform
cp environments/production.tfvars.example environments/production.tfvars # edit domain, repo, email
export TF_VAR_github_access_token=<GitHub token for Amplify> # only needed on first apply

terraform init -backend-config=backend/production.hcl
# ECR first, so the service has an image to pull:
terraform apply -var-file=environments/production.tfvars -target=aws_ecr_repository.api
# → push an image tagged `bootstrap`, or just run the deploy workflow once after the full apply
terraform apply -var-file=environments/production.tfvars
```

If `create_hosted_zone = true`, delegate the printed `hosted_zone_name_servers` at your registrar (ACM validation waits for DNS).

3.4 Secrets

Terraform seeds `medrush/<env>/app` with generated `JWT_SECRET`, `ENCRYPTION_KEY` and the Redis credentials, plus a **placeholder** `MONGODB_URI`. Tasks refuse to start until you replace it:

```
aws secretsmanager get-secret-value --secret-id medrush/production/app --query SecretString --output text > s.json
# edit s.json: set MONGODB_URI (Atlas SRV string); add RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET when ready
aws secretsmanager put-secret-value --secret-id medrush/production/app --secret-string file://s.json
&& rm s.json
```

Adding a new key (e.g. Razorpay)? Also add it to `extra_app_secret_keys` in the tfvars and `terraform apply`. ECS won't start a task that references a missing key.

3.5 GitHub

Create GitHub **environments** `staging` / `production` and add the variables from `terraform output github_environment_variables` (`AWS_DEPLOY_ROLE_ARN`, `ECR_REPOSITORY`, `ECS_CLUSTER`, `ECS_SERVICE`, `ECS_CONTAINER_NAME`, `ECS_SUBNETS`, `ECS_SECURITY_GROUP`, `DEPLOY_HEALTH_URL`, `AWS_REGION`). No AWS keys are stored in GitHub: the workflow assumes the role via OIDC.

4. Deploying

API: Actions → *Deploy* → environment + `deploy`. The workflow:

1. builds the image, scans it with Trivy (fails on HIGH/CRITICAL), and signs it with cosign
2. mirrors it to ECR by digest
3. registers a new task definition
4. runs `node scripts/add-indexes.js` as a one-off task, which creates the legacy indexes and the MedRush model indexes; the deploy stops if this fails
5. updates the service as a rolling deploy with a circuit breaker that auto-rolls back
6. smoke-tests `/api/v1/health`

Rollback: run the workflow with `action=rollback` and a previous `image_tag` (`sha-...`).

Web: Amplify builds automatically on every push to `web_branch`.

5. Operating

Task	How
Logs	CloudWatch → <code>/ecs/medrush-<env>/api</code> (JSON lines)
Shell into a task	<code>aws ecs execute-command --cluster medrush-production --task <id> --container medrush-api --interactive --command sh</code>

Task	How
Seed / one-off script	<code>aws ecs run-task</code> with a command override, e.g. <code>["node", "scripts/seedAdmin.js"]</code> (same image, secrets and network)
Alarms	5xx rate, unhealthy targets, CPU → SNS → <code>alarm_email</code>
Scale	autoscaling 60% CPU / 75% memory between <code>min_capacity</code> and <code>max_capacity</code>
Backups	Atlas continuous backups; S3 versioning (90 days of prior versions)

Background workers (refund outbox, unpaid-order sweeper) run inside every API task. They're safe to run concurrently because every state change is a compare-and-set update.

6. Security baseline (on by default)

- Private subnets for tasks and Redis; only the ALB is public; WAF with AWS managed rules + per-IP rate limit.
- TLS 1.2+ at the ALB; Redis in-transit TLS + AUTH; S3 denies non-TLS; Atlas enforces TLS.
- No static AWS keys anywhere: ECS task role (S3), execution role (secrets, ECR), GitHub OIDC role (deploy).
- Least privilege: the task role can only touch its own uploads bucket.
- Prescriptions are private objects under `prescriptions/<patientId>/`, readable only through `GET .../orders/:id/prescription`, which checks access and then issues a 5-minute presigned URL.

7. Rough monthly cost (production, low traffic)

Item	≈ USD/month
Fargate 2 × (0.5 vCPU, 1 GB)	30
ALB	20
NAT gateways × 3 (per-AZ HA)	100 + data
ElastiCache t4g.micro × 2	25
WAF	10
Atlas M10	60
Amplify, S3, CloudWatch, Secrets	10–20

NAT gateways are the biggest line item. Staging uses one. Adding Atlas PrivateLink plus VPC endpoints for ECR, Secrets Manager and Logs can cut NAT data costs later.

Staging infrastructure (Terraform)

Part	Where
Website (Next.js)	App Runner <code>nabz-staging-web</code> , e.g. <code>https://79fkmxu8w3.ap-south-1.awsapprunner.com</code>
API (Express)	App Runner <code>nabz-staging-api</code> , e.g. <code>https://tiuh3tvxsa.ap-south-1.awsapprunner.com</code>
Database	MongoDB Atlas <code>nabz-staging</code> (AWS Mumbai), database <code>nabz</code>
Images	ECR <code>nabz-staging-api</code> / <code>nabz-staging-web</code> , built by CodeBuild <code>nabz-staging-images</code>
Uploads	private S3 <code>nabz-staging-uploads-<account></code>
Terraform state	S3 <code>nabz-tfstate-<account></code> (key <code>staging/apprunner.tfstate</code>)

The browser only talks to the website; `/api/*` is proxied to the API (see `frontends/web/next.config.mjs`). That keeps the API's `SameSite=strict` cookies first-party without a shared domain. The mobile apps call the API URL directly.

Secrets (AWS Secrets Manager, region ap-south-1)

- `nabz/staging/MONGODB_URI` : Atlas connection string, created by hand (never in git or chat).
- `nabz/staging/JWT_SECRET` , `JWT_REFRESH_SECRET` , `ENCRYPTION_KEY` : generated by Terraform.
- `nabz/staging/ADMIN_PASSWORD` : staging admin password (read it in the console: Secrets Manager → the secret → *Retrieve secret value*). Admins must set up two-step verification (authenticator app) at first sign-in.

Deploy a new version

```
export AWS_PROFILE=nabz AWS_REGION=ap-south-1
# 1. zip the working tree (no node_modules, no .env) and upload
#   (see the zip command used in the deploy session, or `git archive`)
aws s3 cp source.zip s3://nabz-staging-build-<account>/source.zip
# 2. build; App Runner redeploys automatically when :latest changes
aws codebuild start-build --project-name nabz-staging-images \
  --environment-variables-override name=TARGET,value=api,type=PLAINTEXT
aws codebuild start-build --project-name nabz-staging-images \
  --environment-variables-override name=TARGET,value=web,type=PLAINTEXT \
  name=API_ORIGIN,value=https://<api host>,type=PLAINTEXT
```

First-time setup (already done once)

1. State bucket `nabz-tfstate-<account>` (versioned, encrypted, private).
2. `terraform init -backend-config=backend.hcl`
3. `terraform apply` → registries, buckets, CodeBuild, secrets.
4. Build the api image → `terraform apply -var create_api=true` .

5. Build the web image with `API_ORIGIN = api URL` → `terraform apply -var create_api=true -var create_web=true` (this also sets the API's `ALLOWED_ORIGINS` to the website URL).
6. Database: run `scripts/add-indexes.js`, `seedServiceCatalog.js`, `seedPharmacyMarketplace.js` (needs `SEED_DEMO_PASSWORD`), `seedAdmin.js` from a laptop with `MONGODB_URI` from the secret (insert `/nabz` as the database path for these scripts).

Always pass `-var create_api=true -var create_web=true` on later applies, or Terraform will delete the services.

Known limits (staging)

- ⚠ Atlas network access is `0.0.0.0/0` because App Runner has no fixed egress IP. Protected by the database password + TLS. Production: fixed egress (VPC connector + NAT) or Atlas PrivateLink.
- Website logins are rate-limited per client IP as seen through the website's proxy; heavy login testing from many users at once may hit the 5-per-15-minutes auth limit sooner.
- Adding a domain later: App Runner → service → *Custom domains* (both services), then update `ALLOWED_ORIGINS` / `APP_URL` and rebuild the web image with the new API origin.

Mobile apps

One app for **customers** (browse → cart → Cash-on-delivery order, prescription photo) and **pharmacy stores** (incoming-order queue with loud new-order alerts). Talks to the Express API (/api/v1) with bearer tokens and shares types via @medrush/shared .

1. Start the backend (laptop)

```
# repo root – MongoDB must be running (local mongod or Docker)
npm run dev           # API on :5000, listens on all interfaces
npm run db:seed:pharmacy # demo stores + logins (prints them)
```

Demo logins (LOCAL DEV ONLY):

Who	Email	Password	Pick on login screen
Customer	customer@nabz-staging.test	SEED_DEMO_PASSWORD	 Customer
Pharmacy store	pharmacy@nabz-staging.test	SEED_DEMO_PASSWORD	 Pharmacy / staff

2. Which server URL?

Where the app runs	API URL
Android emulator	http://10.0.2.2:5000 (the emulator's alias for your laptop) — the default
Physical phone (same Wi-Fi)	http://<laptop Wi-Fi IP>:5000 — find it with <code>ipconfig</code> → "IPv4 Address"
Phone via USB, any network	<code>adb reverse tcp:5000 tcp:5000</code> , then <code>http://localhost:5000</code>

For a physical phone over Wi-Fi, allow the port once (PowerShell **as Administrator**):

```
New-NetFirewallRule -DisplayName "MedRush API 5000" -Direction Inbound -Protocol TCP -LocalPort 5000
-Action Allow -Profile Private
```

In dev and preview builds the login screen has a **Server** box with **Test connection**, so one APK works on the emulator and on phones.

3. Run during development (Expo Go or emulator)

```
cd frontends/mobile
npm install
npx expo start           # press a → emulator, or scan the QR with Expo Go on the phone
```

Emulator: Android Studio → Device Manager → create a Pixel device (API 35 image is installed), start it, then press `a` .

Expo Go doesn't include your Firebase config, so **server** push can't reach it. In-app alerts and the test alert still work. Test closed-app push on the APK (steps 5–6).

4. What to test (checklist)

1. **Login:** customer and store both log in; a wrong password shows an error; kill and reopen the app → still logged in (token in SecureStore); Log out → back to login.
2. **Permissions:** Home → Permissions → *Allow all*. Location, Notifications (Android 13+ prompt), Camera and Photos should all show . Deny one → the button switches to *Settings*.
3. **Store notification** (two devices, e.g. emulator = store, phone = customer):
 - Store: log in → *Incoming orders* → tap **Send test alert** → you should see and hear it.
 - Customer: Pharmacy → pick the same store → add items → address + pincode → **Place order**.
 - Store: within about 10 s a **"New order MR-..."** notification appears (sound and vibration). The order card is highlighted → Accept → Start packing → ... → Delivered.
 - With Firebase configured (step 6) the alert also arrives when the store app is **closed**.
4. **Rx items:** an item needing a prescription asks for a photo (Camera/Gallery) before ordering.
5. **CORS** doesn't apply to the native app (no browser). The web app's CORS is covered by `tests/integration/medrush-mobile-flow.test.js` (backend repo).

5. Build an installable APK

Local build (this laptop has the Android SDK):

```
cd frontends/mobile
$env:JAVA_HOME = "C:\Program Files\Android\openjdk\jdk-21.0.8"
$env:ANDROID_HOME = "$env:LOCALAPPDATA\Android\Sdk"
$env:EXPO_PUBLIC_ALLOW_SERVER_OVERRIDE = "true"    # Server box on the login screen
$env:MEDRUSH_ALLOW_CLEARTEXT = "true"             # allow http:// to your laptop
npx expo prebuild --platform android --clean
cd android; .\gradlew.bat assembleRelease
# → android\app\build\outputs\apk\release\app-release.apk
```

Install it with `adb install -r app-release.apk`, or copy it to the phone and open it (allow "Install unknown apps").

Cloud build (no local SDK needed): `npm i -g eas-cli && eas login && eas build -p android --profile preview` (this downloads an APK link).

 **Preview APKs are test builds.** They allow `http://` and a custom server and are signed with the debug key. For the Play Store use `eas build --profile production` (AAB, HTTPS API only, no server override) with your own upload key.

6. Server push when the app is closed (Firebase)

1. Firebase console → add an Android app with package `app.medrush.mobile` → download `google-services.json` into `frontends/mobile/` (it's picked up automatically) → rebuild the APK.
2. Backend: a service account with the **Firestore Cloud Messaging API** permission.
`GOOGLE_APPLICATION_CREDENTIALS=<path to key json>` and `FIREBASE_PUSH_ENABLED=true` in `.env`.
3. The store app registers its FCM token automatically (`POST /api/v1/mobile-devices`). Each new order then creates a `Notification` row and sends an FCM push (`services/pharmacyNotificationService.js`).

Without Firebase, the store app still alerts in-app by polling every 10 s while it's open.

Files

- `app/login.tsx` : customer/store login, customer sign-up, server picker
- `app/vendor.tsx` : store order queue, status actions, new-order alerts
- `app/pharmacy.tsx` : nearby stores, storefront, cart, COD checkout, Rx photo
- `app/permissions.tsx` : every runtime permission the app uses, with status
- `lib/api.ts` , `lib/auth.tsx` , `lib/notifications.ts` , `lib/permissions.ts`
- `app.config.js` (permissions, plugins, build flags), `eas.json` (APK / AAB profiles)

Website

Patient storefront + vendor/doctor/admin dashboards. Talks to the existing Express API (`/api/v1`) and shares types via `@medrush/shared` .

Run

```
cd frontends/web
cp .env.local.example .env.local      # set NEXT_PUBLIC_API_BASE_URL
npm install
npm run dev                          # http://localhost:3000
```

The backend must be running (default `http://localhost:5000`) and seeded:

```
# from the repo root
npm run dev          # backend on :5000
npm run db:seed:pharmacy
```

Routes

- `/` — home, category tiles
- `/pharmacy` — nearby vendors + storefront + cart (uses browser geolocation, falls back to demo coords)
- `/vendor` — vendor dashboard (paste a `pharmacy_vendor` token for now)
- `/nursing` , `/lab-tests` , `/consult` , `/emergency` , `/doctor` , `/admin` — scaffolded, shipping in later phases

Notes

- `@medrush/shared` is imported directly from `../shared/src` via a tsconfig path alias + `experimental.externalDir` in `next.config.mjs` .
- Auth uses httpOnly cookies against the API (`credentials: 'include'`); a proper login flow lands in Phase 2.

Code context

Nabz: code context (27 Sep 2026)

Read this before writing a PRD, a plan or code for Nabz. It describes what is built and running today, so new work refines it instead of starting over. Branch `feature/nabz-care-marketplace` (PR #216), staging commit `2f849fa` (updated 27 Sep evening).

What Nabz is

Home healthcare on demand in Jaipur (launch city):

- **Home visits:** nurses and physiotherapists, booked now (ASAP) or scheduled.
- **Medicine delivery:** from nearby partner pharmacies, in about 30 minutes.
- **Home care packages:** for example 10 physio sessions, or 14 days of post-surgery care.

Two Android apps and a website:

Surface	Who	Where
Nabz app	Customers	<code>frontends/mobile</code> (Expo SDK 52, React Native 0.76), <code>APP_VARIANT</code> unset
Nabz Partner app	Nurses, physios, pharmacies (delivery and path labs: onboarding cards)	Same codebase, <code>APP_VARIANT=partner</code>
Website	Customers, partners, admins	<code>frontends/web</code> (Next.js 15)
API	All of the above	Repo root: Node 22, Express 5, Mongoose 9 (MongoDB Atlas)

Shared TypeScript API client and types for both frontends: `frontends/shared/src`.

Staging (live)

- **API:** `https://tiuh3tvxsa.ap-south-1.awsapprunner.com` (AWS App Runner, Mumbai).
- **Website:** `https://79fkmxu8w3.ap-south-1.awsapprunner.com` (proxies `/api/*` to the API).
- **Infrastructure as code:** `terraform/apprunner-staging`.
 - App Runner services and container images (built by CodeBuild).
 - Secrets in Secrets Manager.
 - An EventBridge minute tick that runs background jobs.
 - CloudWatch alarms.
- **Domain:** none yet. The default App Runner URLs are used.

Roles and what each can do today

Role	Status	Main code
Customer (patient)	Built	<code>models/patient.js</code> , <code>routes/patient.js</code> , app tabs <code>app/(tabs)/*</code>
Nurse / physiotherapist (<code>nurse</code> , <code>physiotherapist</code>)	Built	<code>app/staff.tsx</code> , <code>services/dispatchService.js</code> , <code>services/bookingService.js</code>
Pharmacy (<code>pharmacy_vendor</code>)	Built	<code>app/vendor.tsx</code> , <code>services/pharmacyService.js</code> , <code>docs/pharmacy/</code>
Platform admin (<code>platform_admin</code>)	Built (web)	<code>frontends/web/app/admin</code> , admin sign-in with 2-step verification
Path lab (<code>lab_partner</code>)	Waitlist (apply)	Partner app shows how the lab flow works; lab module lives on branch <code>feature/medrush-core</code> (see below)
Phlebotomist, PRP technician, hospital / nursing home	Waitlist (apply)	Partner applications (<code>services/partnerApplicationService.js</code>)
Delivery rider (<code>delivery_partner</code>)	Waitlist (apply)	Pharmacies deliver themselves or nurses pick up

Customer

- **Accounts:** sign in with phone OTP, Google or email.
- **Home visits:** book an ASAP or scheduled visit. Customers can prefer a female or male professional, **choose a specific verified professional** (or best available, with or without a substitute), see their profile and verification badges, and give them a 4-digit visit code at the door.
- **Packages:** book every session at once (chosen weekdays and time). The same professional comes to every session; "Change professional" moves the upcoming ones.
- **Book for someone else:** a parent in another city, picked from phone contacts or recent people; the professional calls the on-site contact. Works for medicine orders too.
- **Saved preferences:** address, gender preference, favourite professional, substitute choice.
- **Referral codes:** a customer can enter a Nabz partner's code before their first order.
- **Visit tracking:** follow the visit live on a map, press SOS, and share a family tracking link.
- **Cancelling and rebooking:** cancelling is free until the professional is on the way, then ₹100. If nobody is free, the customer picks another time.
- **Medicines:** order from nearby stores with a prescription upload.
 - The customer pays with the Nabz payment sheet (UPI apps, card or cash) or cash on delivery.
 - They give the rider a 4-digit delivery code at handover.
- **Nabz Plus:** a membership with free delivery and no visit platform fee.
- **Account deletion:** health history is kept; identity and contact details are erased (app or `/account/delete`).
- **Terms:** consent checkbox at sign-in; `/terms` page (draft for legal review).

Nurse / physio (Partner app and website /staff , /partner/account)

- **Going online:** only verified staff (ID, police check, council registration) can go online.
- **Visit requests:** while online, the app rings loudly and vibrates for a new request nearby, even in the background (foreground location service).
- **Offers:** accept or decline. Going on the way, starting the visit (needs the patient's code) and completing it (with the cash collected) are the steps.
- **Dropping a visit:** hands it back to matching. Three drops in a week takes them offline.
- **Account page:** profile, verification, rating, earnings (today / week / all time), commission tier this month, referral code, payouts.
- **Payouts:** add a UPI ID or bank account (encrypted), withdraw the available balance (earnings minus cash held, minimum ₹100); admin pays and marks paid with the UTR.

Pharmacy (Partner app / website)

- **Order flow:** accept or decline an order (with a reason), verify the prescription, then pack, hand over and deliver (the customer's code is required).
- **Stock:** stock by batch and expiry, shelf-count confirmation, CSV import from billing software, low-stock alerts, and demand from customers nearby.
- **Compliance:** Schedule H1 register, risk flags, and monthly caps on sensitive medicines.

Admin (website)

- **Stores and catalogue:** approve and suspend stores; manage the medicine catalogue.
- **Partner applications:** approving a nurse, physio or pharmacy creates their login and emails a set-password link.
- **Revenue:** revenue and pending partner payouts, netted against cash they collected.
- **Withdrawals:** reveal payout details and mark paid (fresh 2-step code); physio/nurse M : F mix against the 2 : 8 target.
- **Users:** customers and partners with email and phone masked. Showing full contact details, and suspending or restoring an account, need a fresh 2-step code and a reason; both go to the security audit log.
- **Verification:** the queue of nurses and physios missing ID, police or council checks. Ticking or removing a check is audited; removing a core check takes them offline and reassigns their upcoming visits.
- **Payments:** every money movement in a period: online payments, cash collected, refunds, failed payments and partner withdrawals, with gateway references.
- **Live logs:** recent server log lines, refreshed every 3 seconds. Secrets are removed and emails and phone numbers masked before a line is stored. They are held in memory per API instance; older logs are in CloudWatch.
- **Campaigns:** offers and announcements for customers, all partners, nurses and physios, or pharmacies. They can be scheduled, carry an offer code and an in-app button, and show in the "Offers & updates" feed on the website and in both apps. An optional push notification is sent once at the start time (needs Firebase; otherwise the campaign is marked "push skipped").

Key flows and where they live

Flow	Code
Visit booking, prices, cancellation fees, time rules	<code>services/bookingService.js</code> , <code>services/careVisitPolicy.js</code>
Matching (nearest verified, free, preferred gender; offer; next)	<code>services/dispatchService.js</code> , <code>services/staffAvailabilityService.js</code>
Background jobs (offers, pharmacy timeouts, refunds)	<code>services/cronService.js</code> , called by <code>POST /api/v1/internal/tick</code> each minute
Pharmacy search across stores ("who has this brand near me")	<code>services/pharmacyAvailabilityService.js</code> , <code>docs/pharmacy/STORE_NETWORK_PLAN.md</code>
Pharmacy order state machine, store reassignment on decline or timeout	<code>services/pharmacyService.js</code> , <code>services/pharmacyAssignmentService.js</code>
Payments (Razorpay orders, verification, refunds)	<code>services/pharmacyPaymentService.js</code> , <code>services/paymentService.js</code> , <code>frontends/shared/src/payments.ts</code>
Revenue ledger (commission, delivery fee, payouts, cash held)	<code>services/settlementService.js</code> , <code>docs/NABZ_REVENUE_MODEL.md</code>
Commission: monthly tiers for visits (1–10: 20%, 11–30: 15%, 31+: 12%), pharmacy flat 10%, referral credit = min(5%, tier)	<code>services/commissionService.js</code> , <code>config/revenue.js</code>
Referrals (codes, rewards, credits)	<code>services/partnerReferralService.js</code>
Choose a professional, packages as linked sessions, change professional	<code>services/dispatchService.js</code> (<code>findCandidate</code> , <code>lockSeriesProvider</code>), <code>bookingService.createPackageSeries</code> , <code>changeSeriesProvider</code>
Partner account and withdrawals	<code>services/partnerAccountService.js</code> , <code>services/payoutService.js</code> , <code>models/withdrawalRequest.js</code>
Partner onboarding (apply, approve, invite)	<code>services/partnerApplicationService.js</code> , <code>routes/partners.js</code>
Admin panel operations (users, reveal, suspend, verification queue, payment log)	<code>services/adminOpsService.js</code> , <code>routes/adminOps.js</code> (<code>/api/v1/admin/ops</code>)
Live logs (redacted in-memory buffer)	<code>utils/logBuffer.js</code> , <code>utils/logRedaction.js</code> , wired in <code>utils/logger.js</code>
Campaigns and the Offers & updates feed	<code>services/campaignService.js</code> , <code>models/campaign.js</code> , <code>GET /patients/me/offers</code> , <code>GET /partners/me/updates</code> , tick step <code>campaignPush</code>
Service catalogue (visit types, prices, packages)	<code>models/serviceCatalog.js</code> , <code>constants/careServices.js</code> , <code>scripts/seedServiceCatalog.js</code>

Data model (MongoDB collections)

Collection	Holds
<code>patients</code>	Customers, including medical history and dues
<code>users</code>	Staff, partners and admins (with <code>role</code>), including verification in <code>careProfile</code>
<code>nursebookings</code>	Home visits: status, dispatch state, pricing, payment, visit code, service location with on-site contact person and phone, <code>patientDetails</code> (who the visit is for)
<code>servicecatalogs</code>	Visit types and packages with prices
<code>pharmacyvendors</code> , <code>vendorinventories</code> , <code>inventorybatches</code> , <code>medicines</code>	Stores, their stock and batches, and the shared medicine catalogue
<code>pharmacyorders</code> , <code>pharmacycheckouts</code>	Medicine orders (one checkout can span several stores)
<code>settlemententries</code>	Money ledger for every paid event
<code>partnerapplications</code>	Partner sign-ups (with gender, referral code, terms consent) and review state
<code>withdrawalrequests</code>	Partner withdrawals and the ledger entries they settle
<code>campaigns</code>	Admin offers and announcements: audience, schedule, push status, opens
<code>securityauditevents</code>	Audit trail, including admin contact reveals, suspensions, verification changes and campaigns
<code>notifications</code> , <code>pushtokens</code> , <code>jobleases</code> , <code>ratelimitcounters</code>	Supporting data

Built on another branch, not deployed

Branch `feature/medrush-core` (commit `b32a60a`, with uncommitted work in the main checkout) has:

- `models/labTest.js` and `models/labTestBooking.js`: a lab test catalogue, and home sample collection bookings with a phlebotomist.
- `scripts/seedLabCatalog.js`: sample tests and prices. These are placeholders, not a negotiated rate list.
- Consultation and emergency modules.

These must be ported to this branch (and adapted to its conventions) before labs or phlebotomists can go live.

Conventions that matter for new work

- Routes:** every route lives under `/api/v1`; add routers in `routes/` and wire them in `routes/v1/index.js`.
- State changes:** compare-and-set updates (`findOneAndUpdate` with the expected current status), so two people can't change the same record at once.

- **Staging-only switches:** the demo area and test radius must never be set in production. `config/validateEnv.js` refuses to start if they are.
- **Commits:** Conventional Commits. Never push to `main` or `develop` directly.
- **Tests:**
 - `npm test` runs the fast suite.
 - Integration tests need a MongoDB (`MONGODB_URI`).
 - Frontends type-check with `npx tsc --noEmit`.
- **Deploying:**
 - CodeBuild project `nabz-staging-images` (`TARGET=api|web`) builds from a source zip of the commit.
 - APKs are built by `C:\mrb\build-cloud.ps1` (`-Variants customer|partner`).

Known gaps (from `docs/AUDIT_2026-09-24_ALL_ROLES.md`)

- The Atlas database password must be rotated, and network access restricted.
- Move to MongoDB M10 for backups.
- No SMTP yet, so partner invites and password-reset emails aren't sent.
- No alert email subscribed to the CloudWatch alarms.
- Razorpay keys aren't set on staging, so only cash on delivery works there.

Project context (handoff brief)

Read this first. It is a **self-contained** brief: what the project is, where it lives, how it's wired, what's built, the bugs already fixed (don't reintroduce them), how to run it, and what's next. Deep detail is in the repo under `docs/`.

Last updated: 2026-09-23 (Razorpay checkout, AWS deployment, quick-commerce data model).

0. Where the code is

- **Repo (worktree currently in use):**

`C:\Users\ASUS\OneDrive\Desktop\personal\project\.claude\worktrees\pharmacy-vendor-app-website-12957b`

- Branch: `claude/pharmacy-vendor-app-website-12957b`. **Nothing committed yet.**
- Backend deps resolve from the parent checkout's `node_modules` (this worktree has none of its own). `node`, `npx eslint`, `npx jest work`; `npm test` **does NOT** (its script hardcodes `./node_modules/jest`). Use `npx jest --config jest.fast.config.js` instead. Known worktree-only failure: `shared-package-lazy-load.test.js` (`@nocturnal/shared` resolves to the parent checkout's `packages/shared`). Passes in CI.
- Docs: `docs/MEDRUSH_PHARMACY_MARKETPLACE_PLAN.md` (roadmap), `docs/MEDRUSH_DATA_MODEL.md` (**data model + quick-commerce design principles**), `docs/MEDRUSH_AWS_DEPLOYMENT.md` (**AWS runbook**), `docs/MEDRUSH_GETTING_STARTED.md`.
- Integration tests (`tests/integration/*`) need a real MongoDB via `MONGODB_URI` and skip otherwise. `mongodb-memory-server` is in the parent `node_modules` (not a declared dep). Start one, then set `MONGODB_URI` and run `npx jest --config jest.config.js tests/integration/medrush-data-model.test.js`.

⚠️ Competing uncommitted work in the MAIN checkout

The parent checkout (`...\project`, branch `main`) has **staged, uncommitted** Phase-4-style work that was NOT made in this worktree: `lab/consult/emergency models+routes`, `staffLocation`, `geoDispatchService`, `socketTrackingService`, `prescriptionService`. It overlaps this branch (`constants/enums.js`, `models/user.js`, `models/patient.js`, `routes/v1/index.js`) and **contradicts decisions/fixes here**:

- makes `medical_staff / phlebotomist` **self-registerable** (adds them to `REGISTRATION_ROLES`). ⚠️ Security: these roles must stay admin-verified.
- gives `User.currentLocation` `default: 'Point'` + `coordinates` `default [0,0]`, which puts every user at Null Island and pollutes nearest-staff queries (see bug #1).
- adds roles to `enums.js` but **not** `constants/roles.js` (so `authorize()` rejects them).
- different enum values (`MISSED` vs `NO_SHOW`, `HORMONES` vs `HORMONE`, no `CANCELLED` sample status), `availabilityStatus` vs `isAvailable`, `servicesOffered` `ObjectId` vs `String`. Reconcile before merging either line. Don't commit that work to `main` (hooks block it anyway).

1. What we're building

A quick-commerce **healthcare super-app + website**. Fulfilment is a **marketplace of local pharmacy vendors** (aggregator like Swiggy, NOT company-owned dark stores like Zepto/Blinkit). Verticals: **pharmacy medicine delivery** (primary) + home nursing + lab tests + doctor consult + emergency SOS.

- **Backend:** existing "Nocturnal" **Node/Express 5 + Mongoose 9 + MongoDB** monolith (reused).
- **Web:** **Next.js 15** (App Router, TS) — `frontends/web`.
- **Mobile:** **React Native via Expo** (expo-router, TS) — `frontends/mobile` (scaffolded, not run yet).
- **Shared:** `@medrush/shared` typed API client + domain types — `frontends/shared`.

Locked decisions: full breadth, web+mobile in parallel, vendors **admin-seeded + vendor dashboard**, payment prepaid (SOS post-pay), lab catalog in-house for MVP, consult chat/audio first (video later).

2. Status

DONE — Phase 1 (backend foundation + scaffolds)

- New roles in **both** `constants/roles.js` and `constants/enums.js` `#STAFF_ROLES`: `pharmacy_vendor`, `delivery_partner`, `medical_staff`, `phlebotomist` (NOT self-registerable — `/auth/register` validates `REGISTRATION_ROLES`).
- New enums in `constants/enums.js`: `medicine forms/schedule/categories`, `pharmacy order + vendor statuses`, `delivery statuses`, `lab categories/sample types/statuses`, `consultation types/statuses`.
- New models: `models/{pharmacyVendor,medicine,vendorInventory,pharmacyOrder}.js`.
- Extended (additive): `models/user.js` (`pharmacyVendor` ref, `currentLocation [2dsphere]`, `isOnline/isAvailable`, `vehicle`, `servicesOffered`), `models/patient.js` (`emergencyContacts[]`, `preferredLanguage`). The `apps/patient-health` mirror of `constants/{enums,roles}.js` + `models/patient.js` is synced (enforced by `app-mirror-integrity.test.js`; re-copy whenever you edit those root files).
- `services/pharmacyService.js` (atomic stock reservation + rollback, price snapshots, `VENDOR_STATUS_TRANSITIONS` state machine), `services/pharmacyAdminService.js`.
- `controllers/pharmacyController.js`, `routes/pharmacy.js` wired at `/api/v1/pharmacy` in `routes/v1/index.js`.
- Frontend scaffolds: `frontends/shared`, `frontends/web`, `frontends/mobile`.
- Seeds: `scripts/seedPharmacyMarketplace.js` (`npm run db:seed:pharmacy`; vendor login `vendor.koramangala@medrush.test`; password = `SEED_DEMO_PASSWORD` from your local `.env`).

DONE — Phase 2 (auth + checkout + admin), verified live in a browser

- **Patient auth (cookie session):** `frontends/web/lib/auth.tsx`, `app/login`, `app/register`, nav in `app/_components/SiteNav.tsx`.
- **Cart:** `lib/cart.tsx` (single-vendor, `localStorage`) wired into `app/pharmacy`.
- **Checkout:** `app/checkout`: address prefilled from profile, prescription upload for Rx carts, then **online (Razorpay) or COD**.
- **Orders:** `app/orders` (list) + `app/orders/[id]` (status timeline, polling, cancel, **Pay now**).

- **Vendor dashboard:** `app/vendor/login` + `app/vendor` (cookie/staffMe-gated, accept/reject/prepare/ready orders).
- **Admin console:** `app/admin/login` + `app/admin` (approve/suspend vendors, add medicines to master catalog).
- **Backend:** patient prescription upload `POST /api/v1/pharmacy/prescriptions` (`protectPatient` + `uploadPrescription` multer middleware in `middleware/upload.js`).
- Seeds: `scripts/seedAdmin.js` (`npm run db:seed:admin`).

DONE — Plan step 1: Razorpay prepaid checkout (unit-tested; not yet run against live Razorpay test keys)

- `utils/razorpayGateway.js` : lazy SDK client, same feature flag as `/payments-b2c` (`RAZORPAY_KEY_ID` + `SECRET` set, `RAZORPAY_ENABLED !== 'false'`), constant-time HMAC signature check, `toPaise` .
- `services/pharmacyPaymentService.js` : create/reuse the Razorpay order, verify, record failure, refund, and the unpaid-order expiry sweeper (`startExpiryWorker` , started and stopped in `server.js` next to the refund-outbox worker, gated on DB readiness).
- Rules (all compare-and-set updates):
 - A PREPAID order reserves stock and sets `paymentExpiresAt` (`PHARMACY_PAYMENT_TTL_MINUTES` , default 15).
 - It is **hidden from the vendor** (list, detail, and status changes) until `paymentStatus=PAID` .
 - Verify = order-id match + HMAC + server re-fetch of the payment (order_id/amount/currency/status). An `authorized` payment is **captured** before it is marked PAID.
 - The sweeper **asks Razorpay first**: a paid-but-unverified order becomes PAID. Otherwise it is cancelled (`cancelledBy: SYSTEM`) and restocked. If the gateway is unreachable, the order is skipped for that round, not cancelled.
 - Cancel or reject of a PAID order → automatic full refund. A PAID-after-cancel payment is also refunded. Refund failure → `REFUND_PENDING` + `razorpay.refundError` (manual reconcile) and a monitoring alert.
 - PREPAID is refused with a clear message when Razorpay is off, so COD is the only option.
- `PharmacyOrder` gained `paymentExpiresAt` , `razorpay{orderId,paymentId,paidAt,failureReason,refundId,refundedAt,refundError}` , `paymentStatus` `REFUND_PENDING` , plus 2 indexes (run `npm run db:indexes`). The legacy `Payment` model is for staff payouts (requires duty/doctor/hospital), so it is **not** linked.
- Web: `lib/razorpay.ts` (`payForOrder`), checkout payment choice, order page **Pay now** + payment/refund status.
- Tests: `tests/unit/pharmacy/pharmacy-payment-service.test.js` (20 tests); `server-startup-readiness.test.js` covers the sweeper lifecycle.

DONE — AWS-ready from day one (validated, not yet applied to a real account)

- Target: **ap-south-1**, ECS Fargate API behind ALB + WAF, **MongoDB Atlas on AWS** (not DocumentDB: we need `$near` / `$geoNear` / `$text`), S3 uploads, ElastiCache Redis (TLS+AUTH), Secrets Manager, **Amplify Hosting** for Next.js, GitHub OIDC deploy role (no static keys).

- `terraform/` rewritten (old `main.tf` was never applied and couldn't plan). Split files, per-env backends (`backend/*.hcl`) and `environments/*.tfvars.example` . `terraform validate` passes for the main stack and bootstrap. Runbook: `docs/MEDRUSH_AWS_DEPLOYMENT.md` .
- **S3 upload backend** (`config/storage.js` , `STORAGE_PROVIDER=s3` + `S3_UPLOADS_BUCKET`): streams through the magic-byte validator, SSE-S3/KMS, 5-min presigned reads, task-role credentials. Local parity: **MinIO** in `docker-compose.yml` (named volumes, cross-platform).
- Deploy workflow: OIDC auth, image mirrored GHCR→ECR by digest, `scripts/add-indexes.js` run as a one-off ECS task before rollout (now also builds all MedRush model indexes). Dockerfile copies `scripts/` .
- Containers are stateless: data = Atlas (docs), S3 (files), Redis (cache), Secrets Manager.

DONE — Quick-commerce data model (see `docs/MEDRUSH_DATA_MODEL.md` §0)

- `ServiceZone` (polygons + stress lever: rain/rider shortage shrinks radius; pause zone) + `services/serviceabilityService.js` (`$geoNear`; **store's own radius now enforced**; radial fallback). Admin: `GET/POST /pharmacy/admin/zones` , `PATCH /pharmacy/admin/zones/:id` .
- `geohash` on vendors / `deliveryGeohash` on orders (`utils/geohash.js`).
- Orders snapshot `zone` , `distanceKm` , `eta` (per-leg promise) and `milestones` (per-leg actuals). Checkout rejects out-of-range delivery points (web now sends the customer's real GPS fix).
- `InventoryMovement` append-only stock ledger (reserve / release / adjustment).
- Tests: `tests/unit/pharmacy/serviceability.test.js` , `tests/integration/medrush-data-model.test.js` (7, real Mongo).

DONE — Mobile app (Expo) + vendor notifications (2026-09-23)

- `utils/mobileAuth.js` : Expo native clients (`X-Nocturnal-Mobile: expo` , **no** `Origin` / `Sec-Fetch-*` headers) get bearer tokens in the login/refresh body. Browsers never do (cookies only).
- `services/pharmacyNotificationService.js` : new order → `Notification` (type `PHARMACY_ORDER_NEW`) + FCM push to every store account. Fired for COD at placement and for PREPAID when payment is captured. Never throws.
- CORS: a disallowed origin now gets **403** (was 500 plus an error log).
- `frontends/mobile` : login (customer/store) + sign-up, `SecureStore` session, silent token refresh (access tokens last 15 min), store order queue with alerts (10 s polling + FCM), COD checkout + Rx photo, a permissions screen, server picker (preview builds only). `app.config.js` (replaces `app.json`), `eas.json` (preview APK / production AAB). README = testing guide.
- Seed adds demo patient `customer@nabz-staging.test` (LOCAL DEV ONLY; password = `SEED_DEMO_PASSWORD`).
- Tests: `tests/integration/medrush-mobile-flow.test.js` (login, CORS, notification; real Mongo).
- Local APK build: copy `frontends/{mobile,shared}` to a short non-OneDrive path (C:\mrb), JDK 21 (C:\Program Files\Android\openjdk\jdk-21.0.8) , `expo prebuild` + `gradlew assembleRelease` . Java can't download Gradle here (timeouts), so the wrapper uses the cached 8.14.3. AVD `MedRush_API_35` was created by hand (no Android Studio/cmdline-tools installed).

DONE — Home-care booking ↔ pharmacy link, separate portals, Uber-style UI (2026-09-23)

- **Supplies:** `ServiceCatalog.supplies[]` (linked to pharmacy products by `medicineSlug`). Per item the patient picks `PATIENT_HAS` / `STAFF_BRINGS`. `STAFF_BRINGS` → `services/careSuppliesService.js` creates a `PharmacyOrder` with `fulfilment: STAFF_PICKUP + careVisit` (no delivery fee, no basket minimum, COD), linked from `NurseBooking.supplies`. The store is notified ("Nurse pickup"), and the nurse sees the pickup store in `/bookings/provider/me`. Cancelling the booking cancels and restocks the order. Public API: `GET /care/services`, `GET /care/supplies/quote`. Test: `tests/integration/medrush-care-booking.test.js`.
- **Bug fixed:** `createBooking` read `serviceLocation.city` (never sent), so every booking in a city-limited service failed "not available in the requested city".
- **Separate logins** (`constants/portals.js`): `/auth/login` takes `portal` (`staff | pharmacy | lab | rider | admin`) and returns 403 for other roles *before* any session/cookie. New role `lab_partner`. `medical_staff` can use the provider booking routes. Customers stay on `/patients/login` (separate Patient model). Partner accounts are NOT self-registrable (admin-created after verification).
- Demo logins (LOCAL DEV ONLY, `npm run db:seed:pharmacy`): customer@nabz-staging.test, nurse@nabz-staging.test, lab@nabz-staging.test, pharmacy@nabz-staging.test; all use `SEED_DEMO_PASSWORD` from your local `.env`.
- **Mobile UI:** royal-blue theme, bottom tabs (Home = book medical staff with a Leaflet/OSM map in a WebView, Pharmacy, Bookings, Account), booking flow `app/book.tsx` (supplies checklist → date/slots → details → confirm modal → booked/failed screen). Partner areas: `/vendor`, `/staff`, `/lab`. Icon fonts didn't render in release builds, so the app uses emoji and text glyphs.
- **Web UI:** same tokens; home = booking panel + OSM map; `/nursing`, `/staff`, portal logins (`/staff/login`, `/vendor/login`, `/lab/login`, `/admin/login`) via `app/_components/PortalLogin.tsx`.
- OSM tiles are fine for dev/pilot only. Use a paid tile provider (or Google Maps with a key) before launch.

DONE — Brand: Nabz (2026-09-23)

- User-facing name is **Nabz** (नब्ज़ = pulse). The internal code name stays MedRush (packages `@medrush/*`, env vars, Android package `app.medrush.mobile`); change the package only when creating the Play Store listing.
- Logo: `frontends/web/public/brand/nabz-{icon,logo,logo-dark}.svg` (map pin + pulse line, royal blue #1f45e0, coral dot #ff5a3c). App icon/splash PNGs: `frontends/mobile/assets/` (rendered from the SVGs with headless Edge). Review page: `/brand`. Before launch, have a designer outline the wordmark (it uses live text) and run a trademark search.
- No emojis in the UI: Lucide line icons (`lucide-react` / `lucide-react-native` + `react-native-svg`) in tinted tiles (`frontends/web/app/_components/icons.tsx`, `frontends/mobile/lib/icons.tsx`).

DONE — Matching, trust layer, sign-in, two apps, premium UI (2026-09-23)

- **Two apps, one codebase** (`frontends/mobile`): `APP_VARIANT` unset → **Nabz** (customers, `app.medrush.mobile`); `APP_VARIANT=partner` → **Nabz Partner** (staff/pharmacy/lab, `app.medrush.partner`, midnight+gold icon). `lib/variant.ts` (`IS_PARTNER_APP`). EAS profiles `preview` / `partner-preview` / `production` / `partner-production`.

- **Uber-style matching** (`services/dispatchService.js`): Book now → offer to nearest discoverable staff for 45 s → accept/decline (CAS) → next; sweeper every 10 s; scheduled visits dispatch 60 min before; NO_STAFF after 8 tries/10 min. Routes: `GET /bookings/offers/me` , `POST /bookings/:id/offer/{accept,decline}` .
- **Trust layer**: 4-digit visit code (`visitOtp` , `select:false` , patient-only in `/tracking` ; nurse must enter it to start, locks after 5 misses), `careProfile` + verification badges (admin: `PATCH /partners/admin/staff/:id/verification`), SOS `POST /bookings/:id/sos` (alerts platform_admins), public family link `GET /care/track/:token` (first name only), preferred gender, review tags.
- **Customer sign-in**: phone OTP + Google (`routes/socialAuth.js` , `services/socialAuthService.js`); customers only. Production needs `SMS_PROVIDER` (dev logs the code) and `GOOGLE_OAUTH_CLIENT_IDS` + app `EXPO_PUBLIC_GOOGLE_*_CLIENT_ID` .
- **Partner onboarding**: `POST /partners/apply` → admin review queue; logins still created by ops (no self-assigned roles).
- **Home feed** `GET /care/home` (banners only for real offers, packages, popular); **staff dashboard** `GET /care/staff/dashboard` .
- **App UI**: midnight `#0a0f24` / ivory `#f6f7fb` , Instrument Serif + Manrope embedded (`@expo-google-fonts/*`), dark mode follows the phone at launch (`lib/theme.ts`), Hindi/English (`lib/i18n.tsx`), haptics/skeleton/radar (`lib/motion.tsx`). Screens: `welcome` , `phone` (OTP + profile), `partner` , `partner-apply` , redesigned Home, `book` (now/schedule, preference, bill), `track` (radar → nurse card + code + badges + route + share + SOS → rating), `staff` (online switch, earnings, offer card with countdown, demand, code entry, complete).
- **Web**: `/track/[token]` family tracking page.
- Tests: `tests/integration/nabz-matching-trust.test.js` (phone OTP, Google disabled, dispatch offer/decline/accept, visit code gate, SOS, family link, partner apply/review, dashboard, home feed).

DONE — Admin two-step login (2026-09-23)

- `admin` / `platform_admin` : password → 5-min challenge (no session) → authenticator code (TOTP, `utils/totp.js` , RFC 6238 tested) or one-time recovery code → session with `mfa: true` + `authTime` claims (`utils/authTokens.js`).
- `services/adminMfaService.js` , routes `routes/adminMfa.js` at `/auth/admin-mfa` (rate-limited). First sign-in enrolls (QR drawn in the browser, secret AES-GCM encrypted at rest, 10 HMAC-hashed recovery codes).
- Replay-protected codes; 5 wrong codes → 15-min lock; admin sessions end after `ADMIN_SESSION_MAX_HOURS` (8); `requireRecentAuth` step-up (`ADMIN_STEP_UP_MINUTES` , 15) on partner approval, staff verification, vendor create/status.
- Enforced in `middleware/auth.js#protect` , `patientAuth.js#protectBoth` and `/auth/refresh` . Off only under `NODE_ENV=test` unless `ADMIN_MFA_REQUIRED=true` (the integration test opts in).
- Fixed: `protectBoth` ignored `sessionVersion` / `passwordChangedAt` (revoked sessions kept working on `/webauthn`).
- Web: `_components/AdminMfa.tsx` (setup / code / recovery) + `StepUpDialog` in the admin console. Mobile tells admins to use the website. Tests: `tests/unit/security/totp.test.js` , `tests/integration/nabz-admin-mfa.test.js` .

DONE — AWS staging live (2026-09-24), no domain needed

- AWS account `554433963412`, CLI profile `nabz` (user `nabz-deployer`), region `ap-south-1`.
- Website `https://79fkmxu8w3.ap-south-1.awsapprunner.com`, API `https://tiuh3tvxsa.ap-south-1.awsapprunner.com` (App Runner). Stack + runbook: `terraform/apprunner-staging/` (README). Images built by CodeBuild from a zip in S3.
- Atlas `nabz-staging` (M0, Mumbai), db `nabz` via `MONGODB_DB_NAME` (the stored string has no db path).
⚠ Atlas allows `0.0.0.0/0` (App Runner has no fixed IP) and the DB password was pasted in chat: rotate before real users.
- Fixes found while deploying: health check exempt from HTTPS redirect (`app.js`); Next.js `HOSTNAME=0.0.0.0` (App Runner overrides `HOSTNAME`); `add-indexes` creates model indexes one by one (email index name conflict used to skip `users.currentLocation 2dsphere`).
- Staging admin password lives only in Secrets Manager `nabz/staging/ADMIN_PASSWORD`.

DONE — Forgot password + website redesign (2026-09-24, commit `8e3cf82`, deployed)

- `/auth/password/{forgot,check,reset}` (`routes/passwordReset.js`, `services/passwordResetService.js`): one-time SHA-256-hashed tokens (30 min, TTL), no account enumeration, 3 links/hour/account + 3/hour/IP, reset bumps `sessionVersion` + revokes refresh sessions. Email via `services/mailer.js` (nodemailer; `SMTP_HOST/PORT/USER/PASSWORD`, `EMAIL_FROM`; dev logs the mail, production without SMTP sends nothing). Link = `${WEB_APP_URL}|APP_URL}/reset-password`.
- Fixed: `protectPatient` ignored `sessionVersion`; `errorHandler` crashed on service `ValidationError` (empty 500).
- Website in the app theme (Manrope + Instrument Serif via `next/font`, `app/nabz.css`): landing `/`, `/login`, `/signup` (`/register` redirects), `/forgot-password`, `/reset-password`, `/partners`; old home moved to `/book`; site footer; phone tab bar for signed-in customers. Signed-in visits to `/` go to `/book`.
- `WEB_REDIRECT_URL` (backend, optional): browser visits to the API host redirect to the website (set to `http://localhost:3000` in local `.env`; the legacy Nocturnal pages stay for their contract tests).
- Apps: "Forgot password?" → `app/forgot.tsx`; the emailed link opens the website.

DONE — Render removed, PR to main (2026-09-24, commit `ef80e36`)

- Deleted `render.yaml`, `.github/workflows/render-smoke.yml`, `docs/ops/render-post-deploy-smoke.md`; CORS no longer trusts `.onrender.com / RENDER_`; health `deploymentCommit = DEPLOYMENT_COMMIT` (CodeBuild passes the SHA).
- PR #216 feature/`nabz-care-marketplace` → main (needs CODEOWNERS review). Render services themselves still run until deleted in the Render dashboard; legacy Nocturnal Android wrapper still points at `nocturnal-api.onrender.com`.
- Apps: a saved test-server override is dropped when the build's default server changes (`lib/api.ts`).

NOT done yet

- SMTP not configured anywhere yet (reset emails are not delivered until it is).
- Admin passkey (WebAuthn) as a login factor; new-admin-sign-in email alert (needs SMTP).
- Health vault, referrals, tips (need backend + payments), 3D icon pack, paid map tiles, web redesign to the new palette.

- **Razorpay webhook** (`payment.captured / refund.processed`). The sweeper's reconcile covers closed tabs within the TTL. A webhook would also cover a patient who cancels an unpaid order while a payment is in flight and never returns (money captured, no verify → needs manual refund today).
- Mobile: online payment (react-native-razorpay) not in the app yet (COD only). Server push when the app is closed needs Firebase: `frontends/mobile/google-services.json` + backend `FIREBASE_PUSH_ENABLED=true` + `GOOGLE_APPLICATION_CREDENTIALS` . Patients get no status push yet.
- Real-time (Socket.IO) — NOT in this branch; order tracking uses polling.
- Lab / consult / emergency models + UIs in this branch (see the main-checkout note in §0).
- Integration/load tests against a real MongoDB.

Admin login (LOCAL DEV ONLY)

Email/password come from `ADMIN_EMAIL` / `ADMIN_PASSWORD` in your local `.env` (role `platform_admin`), seeded by `scripts/seedAdmin.js` . ⚠ Change before any public deploy.

3. 🐛 Bugs already found & fixed — DO NOT reintroduce

1. **GeoJSON partial default** — giving a GeoJSON sub-field's `type` a `default: 'Point'` creates a coordinate-less `{type: 'Point'}` on every doc without a location, which a `2dsphere` index rejects (breaks inserts). Keep `type` / `coordinates` with **no defaults**; set both together or omit the field. (Applies to `User.currentLocation` , `PharmacyVendor.location` , `PharmacyOrder.deliveryLocation` .)
2. **Mongoose pre-hooks take NO next here** — use synchronous/ `async` (no-arg) hooks. `function hook(next){...next()}` throws `next is not a function` under `insertMany` / `.create()` . (See `models/{medicine,vendorInventory,pharmacyOrder}.js` .)
3. **Bundler imports must be extensionless** in `frontends/shared/src/*` (use `./types` , not `./types.js`) — Next/Metro can't resolve `.js` against `.ts` .
4. **Native `fetch` must be bound to global** — the shared client binds `fetch.bind(globalThis)` ; calling `this.fetchImpl(...)` unbound throws "Illegal invocation" in the browser.
5. **Timer workers must never throw** — an error thrown from a `setInterval` callback's catch block (e.g. a mocked `monitoring` without `trackError`) is an unhandled rejection that kills the process. Guard error reporting in workers.
6. **Jest `resetMocks: true`** (`jest.config.js`) wipes `jest.fn(() => ...)` factory implementations before every test. Re-arm mock return values in `beforeEach` .
7. **"Listed" ≠ "in stock"**: stock level must never flip `VendorInventory.isAvailable` (the vendor's listing intent). The old save hook delisted items at 0 stock, and `restockOrder` re-listed on every cancel, overriding vendors.
8. **Serviceability must enforce each store's `serviceRadiusKm`** . The old `$near` query only used the search radius, so a 3 km store showed for a customer 7 km away.
9. **Prescriptions**: orders carry the upload's storage **key** (validated as the patient's own, `prescriptions/<patientId>/...`) and are read via access-checked `.../orders/:id/prescription` . The old `isURL()` check rejected the relative upload URL (every Rx order 404'd), and the generic file route 404'd prescriptions for everyone.

10. **Don't mix Mongoose `autoIndex` with explicit `createIndexes()`** after `models/user.js` is loaded: it can stall. Scripts and tests that build indexes connect with `autoIndex: false`. Regression tests: `tests/unit/pharmacy/*.test.js`, `tests/unit/infrastructure/s3-upload-storage.test.js`, `tests/integration/medrush-data-model.test.js`.
-

4. Architecture facts you must know

- **All API routes** are under `/api/v1`, composed in `routes/v1/index.js`. Add a per-domain router in `routes/`, wire it there — **never mount in `app.js`**.
 - **Two identities:** `Patient` (B2C, `middleware/patientAuth.js` → `protectPatient`) and `User` (all staff/partner roles, `middleware/auth.js` → `protect` + `authorize(...roles)`). `authorize` validates roles via `constants/roles.js`.
 - **Enums/roles are the single source of truth** — edit `constants/enums.js` (+ `constants/roles.js` for roles) and both Mongoose and express-validator stay in sync.
 - **Thin controllers** → **services**. Services throw typed errors from `utils/errors.js` (`NotFoundError` / `ValidationError` / `ConflictError` / `AuthorizationError` / `PaymentError`). Controllers respond via `utils/responseHelper` (`sendSuccess` / `sendCreated` / `handleServiceError`). `sendSuccess` **spreads** the data object into the top-level JSON.
 - **Model export guard:** `module.exports = mongoose.models.X || mongoose.model('X', S)`.
 - **Circular require:** `pharmacyService` imports `pharmacyPaymentService`; the latter lazy-requires `pharmacyService` (for `restockOrder`) inside the sweeper. Keep it that way.
 - **Frontends isolated** under `frontends/` (NOT in root npm `workspaces`) to keep the CI-gated backend clean. `@medrush/shared` imported from source: web via `tsconfig` paths + `experimental.externalDir`; mobile via `babel-plugin-module-resolver` + `metro watchFolders` + `tsconfig` paths.
 - **DB = MongoDB.** "Tables" = collections; a Mongoose schema is the table (no migration to create it). Only **indexes** are migrated (`npm run db:indexes`).
 - Redis / Razorpay / Firebase / S3 are optional and degrade gracefully. On AWS, S3 is required in practice (container disks are ephemeral); `STORAGE_PROVIDER=s3` is set by Terraform.
 - **Mirrors:** `apps/patient-health/` holds copies of `constants/{enums,roles}.js`, `models/patient.js`, `controllers/patientAnalyticsController.js` and `services/investigationReportService.js`. Apply every edit to both (`app-mirror-integrity.test.js`).
-

5. Data model (collections)

- **PharmacyVendor** — store: name, owner(User), drugLicense/gstin, address, location[GeoJSON 2dsphere], serviceRadiusKm, operatingHours, isOpen, status (PENDING/APPROVED/SUSPENDED/REJECTED), deliveryFee, minOrderValue, rating.
- **Medicine** — master catalog: name, genericName, form, strength, packSize, scheduleType (OTC/PRESCRIPTION/...), category, requiresPrescription, referenceMrp.
- **VendorInventory** — join (vendor×medicine): mrp, sellingPrice, stockQty, isAvailable; unique `{vendor,medicine}`.

- **PharmacyOrder** — patient, vendor, items[] (price-snapshotted), prescription, status (PLACED→ACCEPTED→PREPARING→READY_FOR_PICKUP→OUT_FOR_DELIVERY→DELIVERED , or REJECTED / CANCELLED which restock), deliveryAddress, rider, amounts, paymentMode (PREPAID/COD), paymentStatus (PENDING/PAID/FAILED/REFUND_PENDING/REFUNDED), paymentExpiresAt, razorpay{...}, timeline[].
- **User** (extended), **Patient** (extended). Reused: Review, Notification, ServiceCatalog, NurseBooking. Planned: LabTest, LabTestBooking, Consultation, EmergencyBooking.

Order transitions enforced in `services/pharmacyService.js#VENDOR_STATUS_TRANSITIONS` . Patients may cancel only while `PLACED` / `ACCEPTED` .

6. API surface (`/api/v1/pharmacy`)

- Public: `GET /vendors/nearby?lat&lng&radiusKm` , `GET /medicines/search?q&vendorId&category` , `GET /vendors/:vendorId` , `GET /payment-options` ({online, razorpayKeyId, currency, paymentWindowMinutes}).
- Patient (`protectPatient`): `POST /prescriptions` (multipart), `POST /orders` , `GET /orders` , `GET /orders/:id` , `POST /orders/:id/cancel` , `POST /orders/:id/payment` (create/reuse Razorpay order), `POST /orders/:id/payment/verify` (razorpay_order_id/payment_id/signature), `POST /orders/:id/payment/failure` .
- Vendor (`authorize('pharmacy_vendor')`): `GET /vendor/orders` , `GET /vendor/orders/:id` , `PATCH /vendor/orders/:id/status` , `GET|PUT /vendor/inventory` , `PATCH /vendor/profile` . (Unpaid PREPAID orders are invisible here.)
- Admin (`authorize('admin','platform_admin')`): `GET|POST /admin/vendors` , `PATCH /admin/vendors/:id/status` , `GET|POST /admin/medicines` , `PATCH /admin/medicines/:id` .
- Auth used by the frontends: patient `POST /patients/{register,login}` , `GET /patients/me` ; staff/vendor/admin `POST /auth/login` , `GET /auth/me` , `POST /auth/logout` . All set httpOnly cookies.

7. How to run (with a real MongoDB — the normal path)

```
# from the repo root
npm run setup:env           # generates .env (JWT_SECRET + ENCRYPTION_KEY)
# start MongoDB (local service, Atlas via MONGODB_URI, or: docker run -d -p 27017:27017 mongo:7)
npm run db:indexes
npm run db:seed:admin       # admin from ADMIN_EMAIL / ADMIN_PASSWORD in .env
npm run db:seed:pharmacy    # 3 vendors + medicines + inventory (+ a vendor login)
npm run dev                 # API on http://localhost:5000 (health: /api/v1/health)
```

Online payment (optional): add Razorpay **test** keys `RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` to `.env` (see `docs/MEDRUSH_GETTING_STARTED.md` → "Online payment").

Web:

```
cd frontends/web
cp .env.local.example .env.local      # NEXT_PUBLIC_API_BASE_URL=http://localhost:5000
npm install
npm run dev                          # http://localhost:3000
```

Mobile (later): `cd frontends/mobile && npm install && npx expo start`. Shared (first time, for typecheck): `cd frontends/shared && npm install`.

No MongoDB installed? A throwaway in-memory Mongo works for a demo: `npm i -g mongodb-memory-server` (or local), start it, set `MONGODB_URI` to its uri + `JWT_SECRET / ENCRYPTION_KEY`, then `require('./server.js').startServer()` and seed. Data resets on restart — install a real MongoDB for anything persistent.

Only Node ≥ 22 + MongoDB + npm are required. Docker/k8s/Grafana/etc. in the repo are production tooling — ignore them for local dev.

8. Next steps (recommended order)

1. **Razorpay checkout** ✅ done. Follow-ups: live test with Razorpay test keys; add a Razorpay **webhook** (`payment.captured`, `refund.processed`, verify `X-Razorpay-Signature` against a separate webhook secret); an admin view for `REFUND_PENDING` orders.
2. **Mobile app** — `npm install` in `frontends/mobile`; extend `utils/mobileAuth.js` so the Expo app receives body tokens (store via `expo-secure-store`, set with `lib/api.ts#setAuthToken`); build patient screens (checkout via `expo-camera` for Rx; Razorpay via `react-native-razorpay` reusing `startPayment / verifyPayment`).
3. **Real-time (Phase 3)**: follow `docs/MEDRUSH_DATA_MODEL.md` §5. Build a `DeliveryTask` collection, rider pings in Redis GEO (not Mongo), Socket.IO rooms per order, rider assignment during prep, and stamp `milestones.riderAssignedAt/pickedUpAt`. Add FCM/expo push. 3b. **AWS go-live**: follow `docs/MEDRUSH_AWS_DEPLOYMENT.md` §3 (bootstrap state → Atlas in ap-south-1 → terraform apply → set `MONGODB_URI` in Secrets Manager → GitHub env vars → deploy). Draw the first `ServiceZone` polygons for launch neighbourhoods.
4. **Medical breadth (Phase 4)** — reconcile the main checkout's staged `LabTest / LabTestBooking / Consultation / EmergencyBooking` work (§0) against this branch's enums/roles, then routes + UIs; nursing reuses `NurseBooking`.
5. **Hardening (Phase 5)** — integration + load tests (k6/artillery) against a real Mongo, Redis geo-cache, deploy (Vercel web + EAS/Play Store mobile).

9. Conventions

- Conventional Commits (`feat: / fix: /...`); branch from `develop`; never push to main/develop directly.
- `npm test` = fast suite; `npm run test:all` = full; `npm run verify:local` = lint + fast tests before a PR (in this worktree use `npx jest ...` — see §0).
- Keep new routes under `/api/v1` via `routes/v1/index.js`; keep controllers thin; flag security issues with a `WARNING` comment and never implement insecure patterns.

PART 4

Audits and working notes

Audit of all roles (24 Sep 2026)

Nabz risk audit: all roles (24 Sep 2026)

This audit covers customers, nurses and physios, pharmacies, lab and delivery partners, admins, and the platform itself. Each finding says what goes wrong, where in the code, how to fix it, and whether the fix needs a new app build. The pharmacy order flow was hardened earlier today ([pharmacy/STORE_NETWORK_PLAN.md](#)), so it only appears here where it touches other roles.

Severity:

- **Critical** means money, safety or trust breaks in normal use.
- **High** means it will happen at modest scale.
- **Medium** means it bites later or needs specific conditions.

Status (25 Sep 2026)

Plan steps 1 and 2 are done and deployed to staging (commit `32d441c`).

- **Fixed:** C1–C6, H1–H7, M1, M2, M4, P3.
 - The minute tick is confirmed running on staging.
 - The `background-tick` lease records each run and its result.
- **Also fixed:** "API rate limit exceeded" for normal use.
 - Limits are now per signed-in account.
 - The website forwards the visitor's real IP in a signed header (`PROXY_SHARED_SECRET`), so visitors no longer share one limit.
- **Step 3 (25 Sep 2026):** M3, M5, M6, M7 and P4 are fixed.
 - **M3:** login, sensitive-action and password-reset limits count in MongoDB, so they hold across instances. They fall back to memory if the database is down.
 - **M5:** customers can delete their account in the app (Account → Delete account) or at `/account/delete` .
 - Personal data is erased and every session is revoked.
 - Order, visit and invoice records stay anonymised.
 - Deletion is refused while a visit or order is in progress, or while dues are unpaid.
 - Health records, metrics and the medical history on the account are kept as the patient's history (decided 25 Sep 2026), attached to the anonymous account.
 - **M6:** approving a nurse, physio or pharmacy application creates the login with verification pending and emails a 72-hour set-password link.
 - Pharmacies get a PENDING store that isn't visible to customers.
 - On staging without SMTP, the admin panel shows the one-time link instead of emailing it. Production never shows it.
 - **M7:** lab and delivery logins are hidden. Their applications are a waitlist.

- **P4:** CloudWatch alarms fire when the tick stops or fails, and when the API returns 5xx errors. They publish to the SNS topic `nabz-staging-alerts`.
 - Set `alert_email` in Terraform to receive them by email.
- **Still open, needs you:**
 - **P2:** rotate the Atlas password and restrict network access. This is the most urgent.
 - **P1:** move to MongoDB M10 for backups.
 - Choose an email address for the alarms.
 - SMTP credentials, so partner invites and password resets actually get emailed.

Critical

C1. Background jobs stall when nobody is using the app

App Runner throttles a service's CPU when it isn't handling requests. The `setInterval` workers started in `server.js` all live inside the web service:

- dispatch offer expiry and scheduled-visit dispatch;
- pharmacy acceptance timeouts and reassignment;
- partial-refund retries;
- unpaid-order expiry;
- batch quarantine.

At quiet hours (night) an expired nurse offer or an unaccepted pharmacy order can sit untouched until the next request wakes the instance.

Fix:

- An EventBridge schedule (every minute) calls a new internal endpoint `POST /api/v1/internal/tick`, secured with a secret header checked in constant time.
- The endpoint runs every sweep once, holding a short MongoDB lease so two instances never run it together.
- The in-process timers stay as a backup.
- Terraform adds the EventBridge connection, API destination and rule.

App rebuild: no.

C2. Unverified nurses can go online and take visits

`staffAvailabilityService.setAvailability` and dispatch's candidate filter never check `careProfile.verification`, and admin `assignProvider` doesn't either. The site and app promise that every professional is ID, council and police verified before their first visit.

Fix:

- Going online, receiving offers and being assigned all require ID + council + police verified.
- The staff app shows "Verification pending" instead of the Go-online switch.
- Revoking a verification takes the person offline and re-dispatches their upcoming visits.

App rebuild: yes (banner). The backend alone already blocks it.

C3. Visit status changes can overwrite each other

`bookingService.updateStatus` and `cancelBooking` read the booking, check it, then `save()`. A customer cancelling while the nurse taps Start can end with a cancelled visit marked `IN_PROGRESS`, or a started visit cancelled with its supplies order released. This is the same class of bug fixed in pharmacy today.

Fix: every transition becomes one compare-and-set on the current status, and side effects (supplies cancel, notifications) run only for the winner.

App rebuild: no.

C4. Customers can cancel a visit that is under way, for free

`cancelBooking` only refuses `COMPLETED` / `CANCELLED`. A customer can cancel after the nurse has travelled (`EN_ROUTE`), or even during the visit (`IN_PROGRESS`, so pay-after-visit is never paid).

Fix:

- Free cancellation until the nurse is on the way.
- After `EN_ROUTE`, a cancellation fee (configurable) that is paid to the nurse.
- No cancellation once `IN_PROGRESS`; the customer uses SOS or reports a problem instead.

App rebuild: yes (cancel sheet shows the fee). Enforcement is server-side.

C5. Cash is never recorded, so payouts double-count

Visits are "pay after the visit" and pharmacy orders can be cash on delivery, but:

- nothing records the cash as collected (`payment.status` stays `PENDING`);
- settlement books `PROVIDER_PAYOUT` / `VENDOR_PAYOUT` as money the platform owes, even though the partner already holds the cash, so paying those out means paying twice.

Fix:

- The nurse and the pharmacy confirm "cash collected ₹X" at completion or delivery.
- The ledger gets `CASH_COLLECTED` entries.
- Payouts are netted: the partner owes our commission and fee.
- The admin revenue view shows cash still to collect from each partner.

App rebuild: yes (collect-cash step).

C6. Pharmacies mark their own orders delivered

The store moves an order to `DELIVERED` with no proof. That releases its payout, and prescription medicines can be handed to anyone.

Fix:

- A 4-digit delivery code (like the visit code): the customer's app shows it, and the store or rider enters it to complete.
- If the code can't be used, "delivered without code" needs a reason, and the order is flagged.

App rebuild: yes (code on the customer screen, entry on the store screen).

High

#	Problem	Where	Fix	Rebuild
H1	Nurse cancels or can't make it: the customer's visit is just cancelled	<code>updateStatus</code> → <code>CANCELLED</code>	Provider cancellation re-dispatches to the next nurse (back to searching), records reliability, and auto-pauses after repeated drops, as pharmacy does	No
H2	Scheduled visits start dispatch only 60 min before and give up after 10 min, then sit in <code>NO_STAFF</code> . The customer isn't told and ops aren't alerted	<code>dispatchService</code>	Start offers 12 h before (or at booking if sooner); keep retrying until 30 min before; on <code>NO_STAFF</code> notify the customer (reschedule / cancel free) and ops	Yes (reschedule prompt)
H3	Any confirmed visit (even tomorrow) makes a nurse "busy" all day, and nothing stops two overlapping visits	<code>findCandidate</code> busy check, <code>assignProvider</code>	Busy = a visit overlapping this one's time window ($\pm$ duration + travel); the same check on accept and on admin assign	No
H4	Health-record access: admin-assign grants it with no expiry; app-accept grants none	<code>assignProvider</code> , <code>dispatchService.accept</code>	Grant on accept too; expire 24 h after the visit ends; revoke on cancel	No
H5	Visit dates aren't validated (past dates, years ahead)	booking validation	"Now" visits use server time; scheduled visits must fall between now + 30 min and 30 days ahead	No
H6	A deactivated or suspended nurse keeps their upcoming visits	admin deactivate	Re-dispatch their future visits and take them offline	No
H7	Nurses see the customer's phone and email forever	<code>getBookingById</code> populate	Hide email; show the phone only from confirmation until 2 h after completion; masked calling later	Yes (small)

Medium

#	Problem	Fix
M1	Booking admin checks use <code>role === 'admin'</code> only; the real admin role <code>platform_admin</code> can't act as admin there	Use the shared <code>ADMIN_ROLES</code> list
M2	Admin assignment leaves the dispatch record in OFFERED/SEARCHING (stray offer, wrong state)	Set dispatch MATCHED and clear the offer on assign
M3	Rate limits and some stores are in-memory (Redis off). With 2+ App Runner instances, login and OTP limits multiply	Keep staging at max 1 instance; move limits to Redis or MongoDB before scaling
M4	A phone OTP challenge is saved before the SMS is sent; a failed send still counts toward the hourly cap	Delete the challenge if sending fails

#	Problem	Fix
M5	No "delete my account" or data export (India's DPDP Act 2023)	Delete-account flow: anonymize personal data, keep legally required records (H1 register, invoices)
M6	Approving a partner application doesn't create their login	Approval sends an invite link to set a password, with the role taken from the application; verification starts as pending
M7	Lab and delivery partners can log in to empty dashboards; no rider flow	Hide them from login until built, or build the rider flow alongside C6
M8	Monthly medicine caps can be beaten by two orders at the same instant	Accepted risk (flags + H1 register); a per-patient lock if it's ever seen
M9	Customers aren't notified when a nurse cancels, or when a scheduled visit finds nobody	Covered by H1/H2 notifications

Platform and operations

#	Problem	Fix
P1	MongoDB M0 (free) has no backups and small limits	Move to M10 before real users (daily backups + point-in-time restore)
P2	Atlas password was shared in chat; network access is 0.0.0.0/0	Rotate the password; restrict access to App Runner's fixed egress (VPC connector + NAT)
P3	Staging-only switches must never reach production	A startup check that refuses to boot in production when <code>SERVICEABILITY_TEST_RADIUS_KM</code> / demo flags are set
P4	<code>monitoring.triggerAlert</code> only logs	CloudWatch alarms on error-log patterns and on a worker heartbeat (from C1)

Plan

1. Backend safety, no app rebuild:

- C1 tick endpoint + EventBridge;
- C2 verification gate;
- C3 compare-and-set transitions;
- C4 cancellation rules;
- H1–H6;
- M1, M2, M4, P3.

Tests for each race and rule.

2. App and website changes, then rebuild both APKs:

- C5 collect-cash step + ledger;
- C6 delivery code;
- cancel sheet with fee;

- reschedule prompt;
- "verification pending" banner;
- H7 phone visibility.

3. **Before real users:** M5, M6, M7, P1, P2, P4.

Sources

- [App Runner FAQs](#)
- [App Runner scaling for background services \(AWS re:Post\)](#)
- [App Runner roadmap: support worker services](#)
- [Managing App Runner automatic scaling](#)

Build prompts

Paste a prompt into Claude or another assistant together with [CODE CONTEXT.md](#) (and, for the PRD, `docs/AUDIT_2026-09-24_ALL_ROLES.md`). Both prompts forbid invented numbers, because the outputs go to a team and to investors.

1. PRD prompt

You are a senior product manager for Nabz, a home-healthcare-on-demand startup in Jaipur, India. Attached: CODE_CONTEXT.md (what is built and live on staging) and the risk audit.

Write the Product Requirements Document for Phase 1.1. Phase 1.1 has two tracks.

HUMAN track (partner supply we are signing up now):

1. Pharma stores and path labs, plus freelance delivery riders.
2. Drug delivery live; 5 path labs onboarded on one common rate list (same price for a test at every partner lab).
3. Phlebotomists (home blood-sample collection for the labs).
4. PRP technicians.
5. Physiotherapists, hiring toward a 2 : 8 male : female ratio; customers can book a physio "from a distance" (for a parent or relative at another address, e.g. from another city).
6. Home care packages (multi-session physio, post-surgery, elderly care).
7. Hospital or nursing-home partners.

TECH track:

1. Demo of the Partner app and the customer app, with real-looking medication data.
2. Domain and hosting (move from default AWS URLs to our own domain).

For each item, write:

- Problem and who it is for (customer, partner, ops), with the user story.
- What already exists in the code (quote CODE_CONTEXT.md; say "not built" when it isn't).
- Requirements: must-have for this phase vs later, as numbered, testable statements.
- Edge cases and failure handling (no partner free, partner cancels, payment fails, sample damaged, wrong address, customer not home, disputes, refunds).
- Compliance and safety for India: licences to verify (drug licence, NABL/lab registration, council registration, police check), DPDP Act 2023 for health data, what a PRP technician may and may not do without a doctor (flag anything that needs a medical or legal decision instead of assuming).
- Success metrics with a target to be set by the team (write "target: TBD by team"; never invent numbers).
- Dependencies and open questions.

Then add:

- A release plan: what ships in Phase 1.1 vs 1.2, ordered by dependency.
- A table of partner types → onboarding documents → verification step → app/role.
- Risks, and what we will not do in this phase.

Rules: use only facts from the attachments and this prompt. Do not invent market sizes, prices, partner names, customer counts or growth numbers; mark anything unknown as "[to confirm]". Plain English, short sentences, tables where they help.

2. Investor pitch prompt

You are helping the founders of Nabz prepare an investor pitch (pre-seed / seed).

Attached: CODE_CONTEXT.md (the product that is built and live on staging) and the Phase 1.1 plan.

Nabz in one line: home healthcare on demand in Jaipur: nurses, physiotherapists and medicines at home, booked in minutes from an app, with verified professionals.

Produce two things.

A) A 12-slide deck outline, one slide per section, each with a title, 3 to 5 bullet points, the visual to show, and speaker notes:

1. Problem (families managing care at home; elderly parents living in another city).
2. Solution and demo moment (book a verified female physio for a parent from another city; medicines from the nearest store with the brand in stock).
3. Product today: what is live (use CODE_CONTEXT.md: two apps, website, live tracking, SOS, verification, pharmacy network, payments). Screenshots to capture.
4. Why now.
5. Market: TAM / SAM / SOM for home healthcare and e-pharmacy in India and Tier-2 cities.
6. Business model: commission on visits and medicines, delivery fee, Nabz Plus membership, packages (see the revenue model in CODE_CONTEXT.md; percentages as placeholders).
7. Supply and operations moat: verified partner network, common lab rate list, female-majority physio supply (2 : 8 M : F target), partner app with loud dispatch alerts.
8. Go-to-market in Jaipur, then the next cities.
9. Traction and pipeline.
10. Competition (home-care and e-pharmacy players in India) and how Nabz differs.
11. Team.
12. The ask: amount, use of funds, milestones for 18 months.

B) A one-page investor memo in prose (problem, solution, why us, model, ask).

Rules:

- Never invent numbers. Every market size, growth rate, competitor fact or price must either come with a named, checkable source (report title, publisher, year) or be written as "[number: source needed]". Traction and team sections are placeholders for the founders to fill: "[partners signed: __]", "[pilot bookings: __]".
- Be honest about stage: staging product, pilot in Jaipur.
- Flag regulatory items investors will ask about (pharmacy licences, DPDP Act, PRP needs a doctor) and how we handle them.
- Plain, confident language. No hype words.

3. Code context document prompt (to regenerate CODE_CONTEXT.md after changes)

You are a staff engineer documenting the Nabz codebase for new engineers and for AI coding assistants. Read the repository (start with CLAUDE.md, docs/ARCHITECTURE.md, routes/v1/index.js, models/, services/, frontends/mobile/app, frontends/web/app, frontends/shared/src, terraform/apprunner-staging) and write docs/product/CODE_CONTEXT.md.

Sections, in this order:

1. What Nabz is (one paragraph) and the surfaces (customer app, Partner app, website, API) with their folders and tech stack.
2. Staging environment: URLs, AWS services, how deploys and APK builds work.
3. Roles and what each can do today (customer, nurse/physio, pharmacy, delivery, path lab, phlebotomist, admin), marking each "built", "waitlist" or "not built", with the main files.
4. Key flows and where they live (booking, dispatch, cancellations, pharmacy orders and stock reservation, payments and refunds, settlement and payouts, referrals, partner onboarding, account deletion), as a table: flow → files.
5. Data model: collections and what they hold.
6. Work on other branches not yet merged.
7. Conventions that matter (API versioning, compare-and-set state changes, transactions need primary reads, staging-only switches, commits, tests).
8. Known gaps and open decisions (from docs/AUDIT_*.md and docs/plans/).

Rules: describe only what the code does today; quote file paths; say "not built" rather than guessing; no marketing language; tables where they help; under 250 lines.



Nabz Care

Features, models and edge cases

Physiotherapy, home care, nursing and path labs as a marketplace, on the apps and the website, with the admin panel and ads.
How each part works, why it fits Nabz, and every edge case it handles.

Inside

1. The idea in one page
2. What each person can do
3. How the models work
4. Why this design suits Nabz
5. Edge cases covered
6. Design and motion
7. Tests and quality
8. What is left, and settings to confirm

1. The idea in one page

Nabz Care works like Zomato or Swiggy, for health care at home. Each physiotherapist, home-care agency, nurse or path lab is a **shop**. Each shop has its own **rate card** (its menu), with its own price for each service. The customer compares shops, picks one, chooses home or clinic, picks how many sessions or shifts, sees the full bill and books. Nabz runs the shared catalog, the booking, the payment, the safety layer and the payouts.

Zomato / Swiggy	Nabz Care
Restaurant	A physio on their own, a physio clinic, a home-care agency, a nurse, or a path lab
Dish on the menu	A service on the rate card: "Knee pain therapy, 45 min", "12-hour attendant shift", "Thyroid profile"
Same dish, different price per restaurant	Same catalog service, each shop sets its own price inside a Nabz range
Delivery or dine-in	Home visit or at the clinic / lab
Delivery fee by distance	Travel fee: each shop picks ₹10–15 per km; Nabz measures the distance
Quantity	Number of sessions (1–30) or shifts, on the days the customer picks
One restaurant per cart	One shop per booking; for labs, all tests go to one lab
"Closed now"	Paused, on leave, outside hours, or fully booked
Restaurant offers and ads	New-customer offers (shop-funded, admin approved) and labelled sponsored slots

Four kinds of shop share one engine

Kind	Unit	How it books	Special rules
Physiotherapy	Session (30–90 min)	Home or clinic. One visit or a plan of up to 30 sessions on chosen weekdays.	Plan suggestion after the first visit; multi-session discounts.
Home care	Shift (4, 8, 12 or 24 h)	At home only. Days of the week and a start time; the same named caregiver every day.	Night shifts cross midnight; live-in care charges travel once.
Nursing (planned)	Visit	Same as physio. Urgent "now" nursing keeps the Uber-style matching on the Home tab.	Both models share shops and rate cards.
Path labs	Test or package	Pick tests, compare labs, home collection or walk-in, one time slot.	Fasting tests only in the morning; collection code; report upload; late-report credit.

Everything in this guide is live on the AWS staging servers, in the Nabz customer app, the Nabz Partner app and the website, with the admin controls on the website admin panel.

2. What each person can do

Every customer and partner feature ships on both the apps and the website. The admin panel is on the website.

Customers (Nabz app and website)

Feature	What it does
Care hub	A new Care tab: animated heart hero, active plans with the next session, plan suggestions, Nabz credit, the three kinds, a sponsored spotlight, popular services and "how booking works".
Compare shops	Filter by service, home or clinic, sort by recommended / price / nearest / rating, female or male professional. Each card shows price per session, travel fee and km, rating, verified and NABL badges, offers, and a clear "Sponsored" label on ads.
Shop page (the menu)	Full rate card with clinic and home prices, session length, plan discounts and offers; badges, rating, years of experience and languages.
Booking flow	Home or clinic, visit address (saved or current location), for me or someone else, number of sessions with quick picks, weekdays, a 14-day date strip and free times, pay per visit or upfront.
Live bill	Re-priced by the server on every change: service × sessions, discount, offer, travel × visits, Nabz fee, GST, total, credit to be used, the dates, and "price locked until".
Plans	Progress track with a heart marker, every session with Move / Cancel / Report, Pay Now while times are held, refund status, change the address for upcoming visits, cancel the rest, book again.
Lab tests	Search and pick tests, compare labs (price incl. collection, report time, distance, NABL, missing tests named), book a time (morning only for fasting), pay at collection or online.
Lab orders	Collection code, fasting reminder, five-step progress with times, report viewer, change time, cancel, free re-collection after a rejected sample, late-report credit.
Saved addresses	Home, work, parents: pinned on the map from the phone location or found from the typed address; default address; edit and delete.
Nabz credit	Wallet with history. Credit is used automatically on the next booking.
Plan suggestions	A professional's suggested plan opens the shop with service, mode and sessions filled in; decline in one tap.

Partners (Nabz Partner app and website)

Feature	What it does
My Shop: profile	Name, solo or clinic / agency, council or NABL number, bio, qualification, years, languages, gender, address and map pin (private if home visits only).
Hours	Clinic hours and home-visit hours per weekday, up to two shifts a day, "copy Monday to all days", night shifts (close 23:59, open 00:00).
Home visits	Radius up to 25 km, ₹/km in the 10–15 band, minutes between visits, how many visits at once (teams).
Rate card	For each catalog service: clinic on/off and price, home on/off and price, session or shift length (up to 24 h), live-in, up to three plan discounts, a new-customer offer for review. Warnings when booked sessions are affected.

Feature	What it does
Team	Clinics and agencies add verified Nabz partners by phone or email; removing one hands their days back to the families.
Leave and pause	Pause new bookings in one tap; add days off (booked sessions go back to customers to move or cancel for free); reliability strikes shown.
Plans and suggestions	See booked plans; "Suggest a Plan" on an in-progress or finished visit.
Ads	Ad wallet with top-up, campaigns (sponsored listing, home spotlight, service banner) with views, taps, CTR, bookings and spend; pause or resume.
Lab desk	The day's collections, "Collect Sample" with the customer's code and cash / UPI, Reached Lab, Testing, upload report (photo in the app, PDF on the web), reject sample with a reason.

Admin (website admin panel)

Area	Controls
Overview	Counts for every queue: shops to review, refunds, reports, sessions needing action, today's sessions, late lab reports, ads and settings waiting, offers.
Shops	Approve, reject, suspend or reinstate with a reason (booked sessions are handed back to customers), approve prices outside the range, add strikes.
Catalog	Add and edit services and tests: price floor and ceiling, home / clinic allowed, default length, lab details.
Money	Refund queue for prepaid plans and lab orders (through Razorpay or a bank reference), customer credit with a reason.
Operations	Reported problems (extra cash, no-show) with outcomes, the "needs action" queue, today's sessions board with arrival flags, lab board with late reports.
Offers and ads	Approve or reject offers and ads, pause any ad, house ads, ad wallet adjustments, daily ad report.
Settings	Fees, travel band, plan rules, GST switch and ad placements, changed by propose → approve (maker-checker) with a full history.

3. How the models work

Each part below is a small, separate record with one job. Together they make the booking engine.

Model	Holds	Key rules
CareStore (shop)	Kind, format (solo, clinic, lab), owner, team, registration, map point, clinic and home settings, hours, leave, pause, status, rating, strikes	One shop per owner per kind; one shop per registration number; a map index for "near me"; only safe fields are editable by the shop.
RateCardItem	Shop + catalog service; clinic and home on/off and price; length; plan discounts; offer; lab report time; version	Prices must sit inside the catalog floor and ceiling unless an admin approves; every change bumps the version.
ServiceCatalog	The services and tests shops can list, with marketplace and lab details	Shops cannot invent services; the catalog decides home / clinic allowed, fasting, prescription needed.
CareQuote	The locked bill: every line, totals, schedule, dates, address and travel, expiry	Valid for 15 minutes and usable once. Booking always uses the quote, never prices sent by the app.
CarePlan	N sessions of one service with one shop: mode, payment mode, locked prices, credit used, status, refund	Each session is a normal visit linked to the plan, so tracking, visit codes, SOS and reviews keep working.
SlotReservation	One counter per resource and 15-minute block	An atomic "add one if below capacity" so two people can never take the last place.
LabOrder	Tests, lab, home or walk-in, slot, collection code, payment, timeline, report, rejection, late credit	One lab per order; fasting before 10:00; a duplicate booking is refused.
WalletAccount / WalletEntry	Nabz credit balance and every movement	Each credit or spend has a reference, so it can never apply twice; reversals are exact.
PlanProposal	A professional's suggested plan for a customer after a visit	Expires; the customer books at the shop's current prices or declines.
AdCampaign, AdWallet, AdStatDaily, AdViewerMark	Campaigns, ad money, daily counts, who saw what	Labelled slots, rating floor, budgets, frequency cap, one charge per viewer per day.
PlatformSetting / SettingChange	Admin-controlled numbers and switches, and every proposed change	Propose → approve by a second admin; bounds on every number.

Calendars: who is free, and when

- Time is cut into 15-minute blocks. A visit reserves every block it covers. A home visit also reserves the travel buffer the shop chose (default 30 minutes), so the next booking is always reachable.
- **Person calendars:** a named professional (a solo physio, or the caregiver an agency assigns) has one calendar across every shop they work for, keyed by the person. They can never be in two places at once.
- **Pool calendars:** a clinic with 3 beds has capacity 3 for clinic visits; an agency's home visits count against how many caregivers it can send.
- **Long shifts** (6 hours or more) hold no travel buffer; **night shifts** run across midnight into the next day's hours.
- Leave days, pause, outside hours and full slots are removed before the customer ever sees a time.

Prices: the bill, step by step

All amounts are worked out in **paise** (whole numbers) per session and then multiplied, so a plan's total always equals the sum of its sessions.

Line	How it is worked out
Service	The shop's price for home or clinic × sessions.
Plan discount	Only when paid upfront: the best tier the shop set for that many sessions (for example 5% from 5, 10% from 10).
Offer	New customers only, first session only, capped at the shop's maximum; only after admin approval.
Travel	Straight-line km × 1.3 road factor, rounded up to a whole km, × the shop's ₹/km, at least ₹30. Per home visit; once for live-in care; free for a second family member at the same address on the same day.
Nabz fee	15% of the service (never of the travel); waived for Nabz Plus members.
GST	Today on the whole bill. A switch moves it to the Nabz fee only (health care exemption) once the CA confirms.
Credit	Nabz credit pays first, automatically.

Worked example: 10 home sessions of knee therapy, paid upfront

Line	Amount
Knee therapy at home, ₹600 × 10	₹6,000
10% plan discount	– ₹600
Travel: 4.8 km straight × 1.3 = 6.2 km → 7 km × ₹12 = ₹84 × 10	₹840
Nabz fee: 15% of ₹540 = ₹81 × 10	₹810
GST (today's rule, 18% of ₹540 + ₹84 + ₹81 = ₹126.90 × 10)	₹1,269
Total (₹831.90 per session)	₹8,319

Payments, refunds and payouts

- **Pay per visit:** no discount; each visit is paid after it happens (cash, UPI or online).
- **Pay upfront:** the plan discount applies. The times are held for 20 minutes while the customer pays through Razorpay. The server checks the payment signature before the plan becomes active. Unpaid holds free their times automatically. A payment that arrives after the hold ran out is refunded in full.
- **Lab tests online:** the same flow (new in this release): pay right after booking, or later from the order page while the time is held.
- **Refund of unused sessions:** what was paid minus completed sessions at the **regular** price, so "buy 10 for the discount, use 3" does not pay. If the provider was at fault, used sessions count at the discounted price instead. Credit used is returned as credit.
- **Payouts:** prepaid money is held and paid to the professional after each completed session through the settlement ledger. Travel goes to the professional in full with no commission. Labs are paid when they upload the report.

Path lab order, start to finish

Step	What happens
Booked	The customer gets a 4-digit collection code. Fasting tests remind the customer to fast.
Collected	The phlebotomist enters the code (5 tries) and the cash or UPI amount if it was pay-at-collection.
At lab → Testing	The lab moves the sample along; the customer sees each step with its time.
Report ready	The lab uploads the report; the customer opens it through a short-lived private link. The lab's payout is recorded.
Sample rejected	The lab gives a reason; the customer books a free re-collection.
Late report	If the report misses its promised time, the customer gets 10% of the tests as credit (₹50–200), once.

Ads: how a sponsored slot is chosen

- **Gates first:** the shop must be live, open, near enough, offer the service, and be rated 4.0+ with 10+ reviews. Ads can never buy their way past these.
- **Then score = bid × quality** (rating and booking rate). The winner pays just enough to beat the next ad (second price), never more than its bid.
- **Placement:** positions 2 and 6 of search, never position 1; at most 2 sponsored in the first 10; always labelled "Sponsored"; no ads on a shop's own page, checkout, tracking, SOS or reports.
- **Clicks:** each tap carries a signed token tied to the viewer. Forged tokens are refused; the same viewer is charged once a day; the same ad is shown at most 3 times a day to one person.
- **Money:** ads spend from a prepaid ad wallet; they pause when it runs out. A booking within 7 days of a tap is counted for the campaign.
- **Admin:** review, pause, house ads, wallet adjustments, a kill switch, and every number above is a setting.

4. Why this design suits Nabz

Choice	Why it is right for Nabz
Shops with their own rate cards	Physios and labs already set their own prices offline. Forcing one Nabz price either loses the best professionals or overcharges in cheaper areas. Comparison builds trust: customers see exactly what they pay and why.
A shared catalog with price ranges	Customers compare like with like ("Knee pain therapy, 45 min" means the same thing everywhere). Floors stop ₹1 bait listings followed by cash demands; ceilings stop gouging.
Locked quotes	The app can never decide a price. A price change during checkout is caught and shown, not silently charged.
Plans made of normal visits	Everything built for visits (live tracking, visit codes, SOS, reviews, payouts, cancellation rules) works for every session of a plan with no second system.
Person calendars	Home care only works if "the same caregiver every day" is guaranteed. One calendar per person makes double-booking impossible, even across shops.
Travel by the km, measured by Nabz	Fair to both sides: near customers pay little, far ones pay for the real trip, and the professional earns the full travel fee. Customers cannot type a distance.
Paise everywhere	The bill, the payment and the payout always match to the paise.
Wallet with references	Compensation (no-shows, late reports) and refunds can be automatic without ever paying twice.
Ads behind quality gates	Health care must not let money outrank quality. Ads only help good shops get seen, always labelled, never at checkout or safety screens.
Maker-checker settings	Fees and rules change without a code release, but never by one person alone; every change is recorded.
Uber-style "now" kept for urgent nursing	An injection or IV now is about the nearest free nurse, not comparison. Both models share the same shops and rate cards.

5. Edge cases covered

Each row is handled in the product today. Rows marked **(tested)** have an automated test against a real database.

Prices and rate cards

Situation	What happens
Shop changes a price while a customer is checking out	Booking uses the locked quote; a changed price returns "price changed", shows the new bill and asks again. (tested)
Shop changes a price in the middle of a plan	Booked sessions keep their price.
Price outside the catalog range	Refused with the allowed range unless an admin approved it for that shop. (tested)
Same person opens two shops to appear twice	One shop per registration number and one per owner per kind. (tested)
Shop stops offering home visits with sessions booked	Booked sessions stand; the shop is warned with the count while editing.
Offer abuse	Offers apply only after admin approval, only for a new customer, only on the first plan, capped in rupees. (tested)

Home visits and distance

Situation	What happens
Address outside the shop's radius	Home is not offered for that address; the card says so. (tested)
Customer moves mid-plan	Change the address for upcoming visits; travel is worked out again; out-of-range is refused; any difference is added to the next bill or returned at the end. (tested)
Customer fakes the map pin to pay less	At arrival the professional's location is recorded; more than 2 km from the booked address flags the visit for ops.
Two family members at one address on the same day	Only one travel fee.
Live-in care	Travel charged once for the whole booking. (tested)
Saved address without a map pin	Marked "No map pin"; the typed address is found on the map when saved.

Calendars, plans and home care

Situation	What happens
Two customers tap the last slot at the same moment	Exactly one gets it; the other gets "that time was just taken" and fresh times. (tested)
A clinic with 2 beds	Takes exactly 2 bookings at a time. (tested)
Double tap on Book	Books once. (tested)
Single session on a leave day	The exact date chosen is used (an earlier bug moved it a week; fixed).

Situation	What happens
Same caregiver every day for an agency booking	One named caregiver is locked for every day; when nobody is free for all days, the customer is told before paying. (tested)
Caregiver also has a solo shop	Their one person calendar stops double-booking across both. (tested)
Night shift 20:00–08:00	Runs across midnight into the next day's hours. (tested)
Shop removes a caregiver or takes leave	Their days go back to the families to move or cancel for free. (tested)
Professional releases a session at short notice	Session goes back to the customer, not to dispatch; under 24 hours adds a strike; three strikes in 30 days pause the shop. (tested)
Professional doesn't come	Customer reports it; confirmed cases give ₹100 credit and a strike.
Asked to pay extra cash	Report button on the session; confirmed cases mean a strike and credit.
Session completed	Its time is released so the calendar stays accurate.
Diagnosis changes after the first visit	The professional sends a plan suggestion; nothing changes without the customer booking it. (tested)
Plan not used in time	Plans expire after sessions × 2 weeks (at least 4 weeks); unused sessions are refunded by the rule.

Labs

Situation	What happens
Chosen lab offers 4 of 5 tests	The card names the missing test; booking is blocked until it is removed or another lab is chosen. (tested)
Fasting test	Only times up to 10:00 are offered, with a fasting note. (tested)
Test that can't be collected at home	Lab visit only. (tested)
Test needing a prescription	Booking asks for it first.
Duplicate booking	The same collection twice is refused. (tested)
Wrong collection code	Refused; 5 tries then locked.
Sample rejected	Free re-collection; the report time restarts. (tested)
Report late	Automatic credit, once. (tested)
Paid online, then the hold ran out	Full refund queued for admin. (tested)
Report privacy	Only the patient gets a short-lived link; the lab never sees the collection code.

Payments and money

Situation	What happens
Forged payment signature	Refused; logged as a security event. (tested)
Paying someone else's order	Not found. (tested)
Customer closes the payment screen	The booking stays with "Pay Now" until the hold ends, then frees its times. (tested)

Situation	What happens
Buy 10 at a discount, use 3, refund 7	Used sessions are charged at the regular price. (tested)
Credit spent on a plan that is then cancelled	The credit comes back exactly once.
Rounding	All maths in paise. (tested)

Ads

Situation	What happens
Low-rated shop tries to buy the top	Cannot advertise below the rating and review floor. (tested)
Click fraud	Signed tokens tied to the viewer; one charge per viewer per day; forged tokens refused. (tested)
Ad wallet runs out	Ads pause and resume after a top-up.
Misleading claims	Words like "cure" or "guaranteed" are refused; every ad is reviewed.
Something goes wrong	Admin kill switch turns all ads off at once. (tested)

6. Design and motion

The apps and the website follow the CardioLife direction you shared: a crimson and wine hero, white rounded cards, red pill buttons, chips, and a dark pill bottom bar with a red active tab.

- **Colours:** crimson for actions and heroes, deep wine for depth, green only for success (paid, done, credit). Light and dark themes.
- **Type:** Outfit for headings, Manrope for text in the apps; the same pairing on the website.
- **Animated SVG art:** a heart that beats lub-dub while a heartbeat line draws itself; a knee swinging through its range for physio; a home with a beating heart and a ticking clock for home care; a test tube with rising bubbles for labs; soft pulse rings, orbiting dots and floating plus signs behind each.
- **Motion that respects people:** every animation stops when the phone or browser has "reduce motion" on. Animations run on the native thread where possible so scrolling stays smooth.
- **Tab bar:** a floating dark pill; the active tab grows into a red pill with its name (Home, Care, Pharmacy, Bookings, Account).
- **Booking screen:** a dark pill footer shows the live total with "Confirm Booking" or "Book & Pay".
- **Accessibility:** labelled buttons, 44-point touch targets, live regions for the bill and errors, screen readers skip the decorative art.

7. Tests and quality

Check	Result
Care marketplace automated tests (unit + end-to-end on a real MongoDB replica set)	67 tests: 20 pricing, 27 marketplace part 1, 20 marketplace part 2 (labs, home care, offers, suggestions, ads, settings, lab online payment). All pass.
Wider server suites run with this work (mirror, pricing, physio, matching)	221 tests passed in the last full run.
Load test (earlier this cycle)	10,000 bot users: data rules held. 100,000 bots on a single process: data rules held but the one process was overloaded; production needs several instances (PM2 cluster or more App Runner instances). Details in the Testing and Security Report.
Website	Type-checked and linted; booking flow verified end to end on phone and desktop sizes.
Apps	Type-checked; the Android bundle builds; customer and partner APKs built from the same code.

8. What is left, and settings to confirm

Item	Status
GST on health care	The switch exists and is off. Turn it on only after the CA confirms the exemption (notification 12/2017, entry 74).
Real road distance	Today: straight line × 1.3. Connecting Google Distance Matrix or OSRM makes travel exact.
Razorpay live keys	Online payment for plans, lab tests and ad top-ups is built and verified with test signatures; it needs the live keys on the production server.
PDF reports from the app	Labs upload report photos in the app; PDFs from the website.

Item	Status
Moving old physio packages	The old fixed packages (5, 10, 15 sessions) still exist next to plans; a one-time move of existing physios into shops is the last step.
Scale for launch	Run several API instances and Redis-backed rate limits before a large launch (see the load test).

Testing and Security Report

Staging build · prepared 29 September 2026, 17:12 IST · branch feature/nabz-care-marketplace

Summary

We attacked the Nabz platform the way real users and bad actors would: thousands of scripted "bot" users signing in, booking, buying and withdrawing at the same moment, plus password guessing, code guessing, forged logins and malformed data. Every problem found was fixed, proved fixed by the same bots, pushed to GitHub and deployed to staging on AWS.

Question	Answer
Does any data rule break under load?	No. At 500, 1,000, 2,000, 2,500, 5,000, 10,000 and 100,000 simultaneous bots, no duplicate visit, no oversold stock, no double withdrawal and no cross-account access.
Where does it slow down?	One server process serves about 2,500 simultaneous users without errors. At 10,000 it slows but stays correct. At 100,000 at once it is overloaded (waits of 9 to 16 minutes); that needs about 40 servers. Staging adds servers automatically (up to 4).
Can passwords or codes be brute-forced?	No. Tested with 40,000+ parallel password guesses and thousands of parallel code guesses: accounts lock after 10 wrong passwords, codes after 5 guesses.
Problems found	18 issues (security loopholes and robustness bugs) found by the bots. All fixed, re-tested and deployed.
Final hard test (this run)	All rules held

1. How we tested

Method	What it does
Bot test suites (28 scripted scenarios)	Scripted customers, nurses, pharmacies and attackers run against the real API and a real database: access control, races, out-of-order steps, impossible times, malformed input, uploads, privacy, sign-in.
Bot swarm (500 to 100,000 bots)	All bots at once: 60% customers (sign in, double-tap Book, race for a medicine with 50 units), 20% attackers (injection, forged tokens, snooping), 10% nurses (take visits that are not theirs), 10% partners (triple-tap Withdraw). Afterwards the rules are checked.
Brute-force lab	Credential stuffing (20 wrong passwords per account, all accounts at once), password spraying (one common password on every account), parallel visit-code guessing.
Soak test	Steady traffic for several minutes while measuring speed and memory, to find slow leaks.
Security bots	Parallel code guessing, mass assignment (setting price, dues, status, roles), account discovery, password lockout, OTP guessing, SMS flooding.
Live AWS checks	Brute-force burst and attack payloads against staging (firewall and flood protection), security headers, CORS, sign-in smoke tests, log review.
Dependency audit	Every library checked against published vulnerability databases.

Load tests run on one laptop with the load generator and one server process sharing the same CPU, so the speed numbers are a pessimistic floor for a single server, not the capacity of the AWS setup.

2. Problems found and fixed

Each item was reproduced by a bot first, then fixed, then the same bot confirmed the fix.

#	Problem (how the bots proved it)	Fix	Status
1	4-digit visit code guessable: 60 parallel guesses were all checked and a visit was started without the patient.	Every guess reserves an attempt atomically before checking; at most 5 ever checked.	Fixed
2	Delivery code guessable the same way (59 guesses checked).	Same atomic attempt limit (5).	Fixed
3	Phone OTP guessable in parallel (40 wrong codes checked, then accepted).	Atomic attempt limit (5 per code).	Fixed
4	Admin two-step code: wrong attempts overwrote the counter, so the lock never triggered.	Atomic counter with 15-minute lock.	Fixed
5	No lockout on password guessing (12 wrong then right still worked; 30 parallel guesses all checked).	Account locks for 15 min after 10 wrong passwords, even in parallel; password reset unlocks.	Fixed
6	SMS cost abuse: one client triggered 40 SMS to different numbers in seconds.	Atomic per-network counter: 20 codes per hour.	Fixed
7	Referral self-dealing: a nurse could refer a relative and serve that visit herself to cut her commission.	Reward only when someone else served the first order.	Fixed
8	Two sign-ins of the same account in the same second failed (identical tokens).	Every token carries a random unique ID.	Fixed
9	Sign-in froze the server under load: 200 sign-ins took 26 s each and stalled other requests for 20 s.	Password hashing moved off the main thread (native bcrypt, same hashes): 4.3x faster.	Fixed
10	Double-tapping Book created duplicate visits (3 taps, 3 visits).	One live booking per customer, service, time and person cared for; freed on cancel, completion, reschedule.	Fixed
11	Booking for someone else lost the patient's name, age and gender.	Stored and shown to the professional.	Fixed
12	Visit time used the booker's phone time zone (a son abroad got an evening visit).	Visit times always India time.	Fixed
13	SOS worked on cancelled or weeks-old visits and repeat taps re-paged ops.	SOS only during a visit (+30 min); repeats recorded without paging.	Fixed
14	Crashes (500) on: unassigned nurse completing a visit, oversized or broken requests, rejected uploads.	Clean refusals (400/404/413).	Fixed
15	Server errors showed internal messages to users.	Generic message; details only in logs.	Fixed
16	Website sent no browser security headers (could be framed / clickjacked).	CSP, HSTS, no framing, nosniff, permissions policy.	Fixed
17	Redis down produced unhandled errors.	Handled; the app runs without Redis.	Fixed
18	Single pharmacy order had no quantity cap.	Capped at 100 like the cart.	Fixed

Tested and found already safe: mass assignment (customers cannot set price, dues, payment status or role; nurses cannot verify themselves), account discovery (sign-in and reset answer the same for unknown emails), cross-account access, forged unsigned tokens, CORS from other sites, sign-in cookies (HttpOnly, Secure, SameSite strict), stock overselling, double withdrawals, double completion payouts.

3. Load tests: raising the number of bots until it breaks

Bots at once	Requests	Sign-in rush	p95 wait	Data rules	Result
500	3,360	6.2 s	10.2 s	Held	No errors
1,000	6,720	12.1 s	19.1 s	Held	No errors
2,000	13,440	25.0 s	43.5 s	Held	No errors
2,500	16,800	29.6 s	47.1 s	Held	No errors
5,000	33,600	59.8 s	454 s	Held	Slow: 100 of 33,600 requests (0.3%) timed out after 2 min
10,000	67,200	118 s	209 s	Held	No errors, no timeouts; slow (waits up to ~3.7 min) but every rule held
100,000	582,706	1,201 s	939 s	Held	Overloaded: median wait 9 min; 374,109 requests refused because the sign-in expired while queued; 100 connections reset

At every size no data rule broke: no duplicate visits, stock never oversold, never two withdrawals for one partner (up to 10,000 bots: exactly 50 of 50 units sold). The only thing that breaks under extreme load is waiting time, which more servers fix.

4. Brute force and endurance

Attack	Scale	Result
Credential stuffing	40,000 wrong passwords on 2,000 accounts at once	Max 10 guesses per account; all 2,000 locked; 0 break-ins
Password spraying	"Password@123" on 2,000 accounts	0 accounts opened
Visit-code guessing	6,000 parallel guesses on 200 visits	0 visits over the 5-guess limit; 0 started
Soak	67,292 requests over 3 minutes (374/s)	p95 190-273 ms, flat; memory flat 1,230-1,250 MB (no leak)
Live AWS brute force	Rapid sign-in burst against staging	Blocked after ~80 attempts ("suspicious activity"); WAF limit behind it
Live AWS attack payloads	Log4j, SQL injection, script injection, no user-agent	All blocked by the WAF (403); normal traffic 200

5. Final hard test (this run)

Run log: START 2026-09-29T01:23:31+05:30 · BOTS_DONE 44s · SWARM_DONE exit=0 181s · ATTACK_DONE exit=0 1703s · END 2026-09-29T01:51:54+05:30 · SWARM10K_START 2026-09-29T01:52:13+05:30 · SWARM10K_DONE exit=0 501s

5.1 Bot test suites

Suites	Result	Server errors
Bots, round 2 and security bots (28 scenarios)	28 passed, 28 total	0

5.2 Swarms

Bots	Requests	Sign-in rush	p95 wait	Visits	Medicine sold	Outcome
2,500	16,800	30.9 s	53.7 s	3,000	50 / 50	none broken; 0 server errors; 0 timeouts
10,000	67,200	118 s	209.2 s	12,000	50 / 50	none broken; 0 server errors; 0 timeouts
100,000	582,706	1201.1 s	938.7 s	30,415	0 / 50	data rules held; 0 server errors; 100 connections reset under overload

5.3 Brute-force lab and soak

Attack	Scale	Result
Credential stuffing	50,000 guesses on 2,500 accounts (325.2 s)	Max 10 per account · 2,500 locked · owner during lock: HTTP 429
Password spraying	2,500 attempts	0 accounts opened
Visit-code guessing	6,000 guesses on 200 visits	0 over limit · 0 started
Soak (right after the attacks)	11,697 requests in 300 s (39/s)	p95 per 30 s: 229, 302 ms · memory 1192 to 1196 MB
Soak (on its own, clean database)	137,230 requests in 300 s (457.4/s)	p95 per 30 s: 186, 171, 166, 172, 199, 174, 176, 189, 209, 205 ms · memory 707 to 718 MB

The first soak ran straight after 50,000 password guesses on a database still full of locked accounts, on the same laptop, so it was slower. Run again on its own, the server handled over 10x the traffic with flat speed and flat memory: no leak.

All rules held in the brute-force lab.

5.4 One lakh (100,000) bots at once: the breaking point

All 100,000 bots ran against a single server process on one laptop: 60,000 customers, 20,000 attackers, 10,000 nurses and 10,000 partners, 582,706 requests in 38 minutes.

- **Sign-in:** 20 minutes for 92,000 sign-ins; 79,917 of 80,000 accounts signed in (the rest waited over 2 minutes and gave up).
- **Waiting time:** median 9 min, 95% within 16 min, slowest 16 min. One server is far past its limit here.
- **Expired sign-ins:** sign-in tokens last 15 minutes. Requests that queued longer were correctly refused as expired (374,109 requests). The real app renews its token automatically; the bots did not, so the medicine race and withdrawals barely ran at this size (they were fully tested at 10,000).
- **Connection resets:** 100 booking requests were cut off after waiting more than 5 minutes inside the server. No server errors (500s).
- **Data rules: held.** 30,415 live visits with no duplicates, stock never oversold, no double withdrawals, no customer read another's visit, every attack refused.
- **What it means:** one server handles about 2,500 people at the same second. 1 lakh people acting at the same second needs about 40 servers and a larger database (MongoDB Atlas M30 or above). 1 lakh registered users

(not all at the same second) is far lighter: typically 1 to 5% are active at once.

6. Security layers now in place

Layer	Protects against
AWS WAF firewall (both services)	Known attacker IPs, OWASP attacks, Log4j and other exploits, SQL injection, sign-in floods (300 per network per 5 min), request floods.
App flood protection	Rapid bursts from one network (blocked after ~80 fast sign-in attempts).
Per-account lockout	Password guessing from many networks: 10 tries, then 15-minute lock.
Atomic attempt limits	Parallel guessing of visit, delivery, OTP and admin codes (5 tries).
SMS network cap	SMS cost abuse: 20 codes per network per hour.
Admin two-step + fresh code for sensitive actions	Stolen admin passwords; every sensitive admin action audited.
Browser security headers	Clickjacking, script injection, protocol downgrade.
Private storage + signed links	Documents and photos only reachable by the right people, each view logged for documents.
Auto-scaling (up to 4 servers)	Traffic spikes such as a launch-day sign-in rush.

7. Open items

- **Tomorrow:** SMTP email, Razorpay keys, SMS provider (with DLT registration), alert email for the AWS alarms.
- **Mobile build tools:** the Expo SDK 52 command-line tools carry 24 known vulnerabilities (1 critical, in tar). They run only on the build computer, not in the app on phones. Fix: upgrade to Expo SDK 54 with a full app retest.
- **Stricter website script policy:** move the content security policy from inline scripts to per-request nonces.
- **Production capacity:** for launch, set a minimum of 2 servers and raise the maximum (about 40 for 1 lakh people at the same moment); move MongoDB Atlas to a dedicated tier (M10+, M30+ for 1 lakh at once), then repeat the load test on AWS.
- **DigiLocker partner credentials** for automatic Aadhaar verification.
- **Saved locations** (book for another address and keep a list of addresses): next feature.

8. Re-running the tests

All tools are in the repository. Local runs need a throwaway MongoDB replica set; they refuse any non-local database.

What	Command
All automated tests incl. bots	<code>npm test</code>
Bot suites only	<code>npx jest --config jest.config.js tests/bots --runInBand</code>
Swarm of N bots	<code>node scripts/bot-swarm.js 2000 (100,000: LOG_LEVEL=warn, about 40 min)</code>
Brute force + soak	<code>node scripts/attack-lab.js 2000 180</code>
Sign-in load	<code>node scripts/loadtest-signin.js 50 4</code>